

슈팅월드 학습 가이드 문서

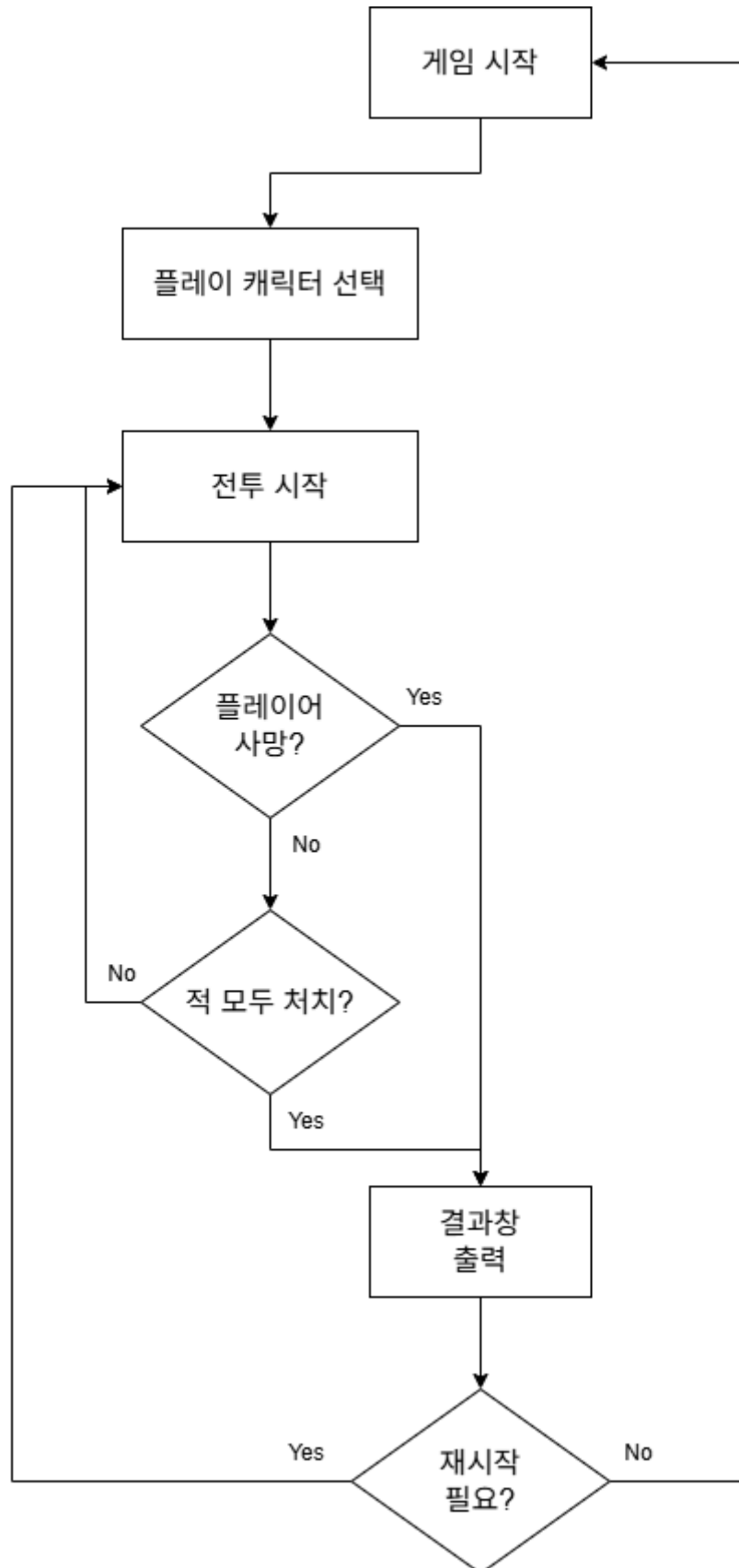
학습 목표

- 가이드 문서와 샘플 월드를 통하여 슈팅 게임의 핵심 기능들을 구현하는 방법에 대해 학습합니다.
 - 메이플스토리 월드에서 제작하려는 게임의 방식에 따라 맵을 세팅하는 방법에 대해 학습합니다.
 - Logic에 대해 이해하고 이를 사용하는 방법에 대해 설계할 수 있습니다.
 - 제작된 월드를 원하는 형태로 가공하고 제작할 수 있습니다.
-

학습 핵심 기능

- **DataSet 관리**: DataTableLogic을 활용하여 DataSet을 하나의 Logic이 관리하는 Manager 사용법을 학습합니다.
- **엔티티간 충돌 관리**: TriggerComponent의 Collision Group과 Matrix를 활용하여 충돌 조건을 설정하고, 사용할 수 있습니다.
- **오브젝트 풀**: 다수의 투사체를 관리하기 위해 오브젝트 풀 기법의 원리를 학습하고, 제작할 수 있습니다.

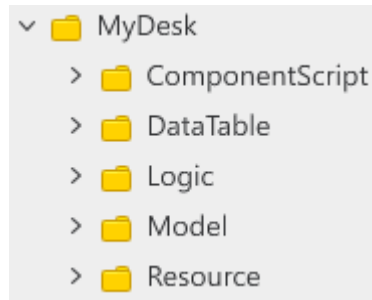
핵심 게임플레이 루프



1. **게임시작:** 타이틀 화면과 함께 게임 시작 또는 플레이 방법을 확인할 수 있습니다.
2. **플레이 캐릭터 선택:** 타이틀 화면에서 게임 시작 버튼을 눌렀을 경우 캐릭터를 선택하는 UI가 출력됩니다.
3. **전투시작:** 본격적으로 슈팅게임을 시작합니다. 몬스터의 스폰, 플레이어의 공격과 같은 슈팅 게임의 상호작용이 일어납니다.
4. **모든 적 처치 여부 검사:** 현재 플레이 중인 맵에서 클리어 조건을 충족하는 몬스터 수를 처치했는지 확인합니

- 다.
- 5. 결과 확인:** 플레이어가 모든 몬스터를 처치했는지, 혹은 플레이어가 패배했는지에 따라 결과 창을 제출합니다.
- 6. 재시작 여부 검사:** 플레이어가 재시작을 요청하는지, 타이틀로 돌아갈지에 따라 해당하는 화면으로 이동합니다.

폴더 구성



이름	설명
ComponentScript	샘플 월드 구현을 위해 엔티티에 부여한 컴포넌트들을 담은 폴더입니다.
DataTable	샘플 월드를 구성하는 데 필요한 DataSet을 담은 폴더입니다.
Logic	메이플스토리 월드의 기능인 Logic으로 제작된 스크립트를 담은 폴더입니다.
Model	Model화 한 엔티티들을 담은 폴더입니다.
Resource	게임 제작에 필요한 Resource를 담은 폴더입니다. 현재는 TileSet만 담겨 있으며, 추후 여러분이 월드 제작에 필요한 리소스를 담아서 이용하실 수 있습니다.

구현 순서

- 💡 **기초 학습:** 메이플스토리 월드의 기초적인, 또는 해당 장르의 가장 기초적인 기능을 담은 학습 항목입니다.
- 📖 **심화 학습:** 샘플 월드에 구현된 기능을 학습하고 구현하는 항목입니다. 해당 과정의 학습을 통해 장르의 기능 확장과 학습 이해도를 넓힐 수 있습니다.

[💡 기초 학습]1. 맵 설정하기

- 메이플스토리 월드에서 제공하는 맵의 종류를 학습합니다.
- BodyComponent와 맵의 종류의 관계에 대하여 학습합니다.
- KinematicBodyComponent에 대하여 학습합니다.
- RectTile을 배치하여 맵을 형성하는 방법을 알아봅니다.

[💡 기초 학습]2. Logic 선언하기

- Logic이 어떤 역할을 하는지에 대해 학습합니다.
- Manager에 대한 개념을 이해하고, 사용 방법을 학습합니다.
- GameLogic, DataTableLogic, UIManageLogic 구성 이유와 목적을 이해합니다.
- Logic의 장단점에 대하여 학습합니다.

[💡 기초 학습]3. Playercharacter 제작하기

- 플레이어를 구성하기 위한 DataSet 구성 요소를 학습합니다.
- DataTableLogic을 통해 DataSet을 불러오고, 사용하는 방법에 대해 학습합니다.
- 저장된 데이터를 담을 Component를 제작하고 적용하는 방법에 대해 알아봅니다.
- TriggerComponent를 부여하고, 편집하는 방법에 대하여 학습합니다.

[💡 기초 학습]4. Monster 제작하기

- 몬스터를 구성하기 위한 DataSet 구성 요소를 학습합니다.
- DataTableLogic을 통해 DataSet을 불러오고, 사용하는 방법에 대해 학습합니다.
- 저장된 데이터를 담을 Component를 제작하고 적용하는 방법에 대해 알아봅니다.
- TriggerComponent를 부여하고, 편집하는 방법에 대하여 학습합니다.
- 몬스터가 플레이어를 추적하기 위해 AIChaseComponent를 부여하고, 사용 방법에 대해 학습합니다.
- 제작한 몬스터를 재사용하기 위한 Model화 방법을 학습합니다.

[💡 기초 학습]5. 기본 공격 제작하기

- 투사체를 구성하기 위한 DataSet 구성 요소를 학습합니다.
- DataTableLogic을 통해 DataSet을 불러오고, 사용하는 방법에 대해 학습합니다.
- 저장된 데이터를 담을 Component를 제작하고 적용하는 방법에 대해 알아봅니다.
- TriggerComponent를 부여하고, 편집하는 방법에 대하여 학습합니다.
- Collision Group에 대해 학습하고, Matrix를 통한 편집 방법에 대해 학습합니다.
- 투사체가 지정된 방향으로 발사되도록 기능을 부여합니다.

[💡 기초 학습]6. 오브젝트 풀 생성하기

- 오브젝트 풀에 대한 개념을 학습하여 오브젝트 풀의 목적을 이해합니다.
- 오브젝트 풀 Component를 구현합니다.
- 오브젝트 풀의 생성과 반환을 통해 작동 방식을 직접 확인합니다.
- 플레이어와 몬스터의 투사체 발사 시스템을 구현하는 방법에 대해 학습합니다.

[📖 심화 학습]7. 스킬 구현하기

- 스킬을 구성하기 위한 스킬 DataSet 구성 요소를 학습합니다.
- DataTableLogic을 통해 스킬과 관련된 DataSet을 불러오고, 사용하는 방법에 대해 학습합니다.
- 저장된 스킬 데이터를 담을 Component를 제작하고 적용하는 방법에 대해 알아봅니다.
- 스킬 엔티티에 TriggerComponent를 부여하고, 편집하는 방법에 대하여 학습합니다.
- 제작한 엔티티를 Model로 전환하고 사용하는 방법을 학습합니다.
- 투사체형 스킬이 지정된 방향으로 발사되도록 기능을 부여합니다.
- 투사체형 스킬의 기능을 구현하기 위해 사용된 TriggerEvent에 대하여 학습합니다.

[📖 심화 학습]8. 아이템 구현하기

- 아이템을 구성하기 위한 스킬 DataSet 구성 요소를 학습합니다.
- DataTableLogic을 통해 아이템 DataSet을 불러오고, 사용하는 방법에 대해 학습합니다.
- 아이템 데이터와 아이템 스폰 데이터를 적용하기 위해 컴포넌트 추가 및 수정을 진행합니다.
- 아이템을 구성하는 Model을 제작하는 방법에 대해 학습합니다.
- 아이템의 스폰 확률과 아이템 스폰을 담당하는 ItemHelper Logic을 제작합니다.
- 아이템의 효과가 정상적으로 처리되도록 기능을 구현하는 방법에 대해 학습합니다.
- 아이템 스폰의 확률을 적용하기 위해 Monster를 스폰하는 방법에 대해서 학습합니다.

[📖 심화 학습]9. MonsterSpawn 구현하기

- Monster가 DataSet에 입력된 값에 따라 InGame에 등장하도록 기능을 구현합니다.
- DataSet의 구조를 학습하고 해당 구조를 사용하는 방법을 학습합니다.
- 스폰되는 몬스터마다 아이템 드랍 확률을 부여하여, 처치시 아이템이 확률에 맞춰 등장하도록 기능을 구현합니다.

[💡 기초 학습]10. UI 구성하기

- UIGroup과 구성 요소, 편집 방법에 대하여 학습합니다.
- 타이틀 화면을 제작하여 UI 편집 방법에 대해 학습합니다.
- 타이틀 화면을 제어하는 Component를 제작하여 해당 UI의 기능을 처리할 수 있도록 제작합니다.
- 플레이어가 캐릭터를 선택할 수 있도록 하는 SelectCharacterUI 제작 방법을 학습합니다.
- 인 게임 정보를 제어하는 DefaultUI를 구성하는 방법에 대해 학습합니다.
- DefaultUI를 제어하는 Component를 제작하여 실제로 체력, 스킬의 남은 횟수가 업데이트되도록 기능을 구현합니다.

[💡 기초 학습]맵 설정하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: 00:01:29

학습 목표

- 메이플스토리 월드의 맵 방식에 대하여 학습하고, 월드를 제작할 때 자신이 필요한 맵 방식이 무엇인지 선택할 수 있습니다.
- 각 맵 방식에 따라 맵의 발판인 타일들과 상호작용 할 수 있는 컴포넌트가 무엇인지 설명할 수 있습니다.
- RectTileMap과 상호작용을 하기 위한 KinematicBodyComponent에 대하여 학습합니다.
- RectTileMap에 타일을 직접 배치하는 방법에 대하여 학습하고, 맵을 형성할 수 있습니다.

해당 영상 시청 후 수행해 볼 내용

- 슈팅 월드 제작을 위해 Map의 형식을 RectTileMap으로 변경하기.

- TileSet 제작하기.
- 제작한 TileSet을 활용하여 실제로 상호작용이 가능한 Map 제작하기.

맵의 종류



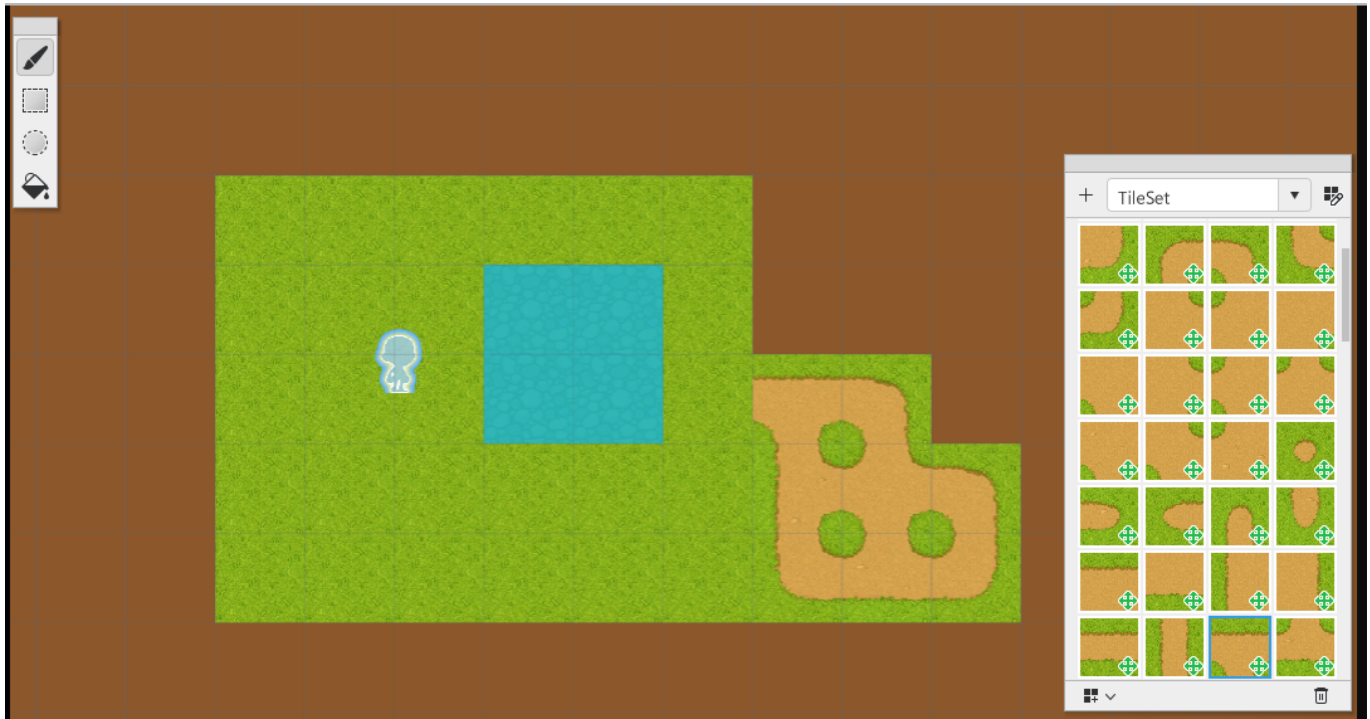
TileMap



- 메이플스토리 월드에서 처음으로 월드를 생성하면 기본 지정되는 맵 형식입니다.
- 메이플스토리의 느낌을 가장 잘 살릴 수 있는 방식의 월드입니다.

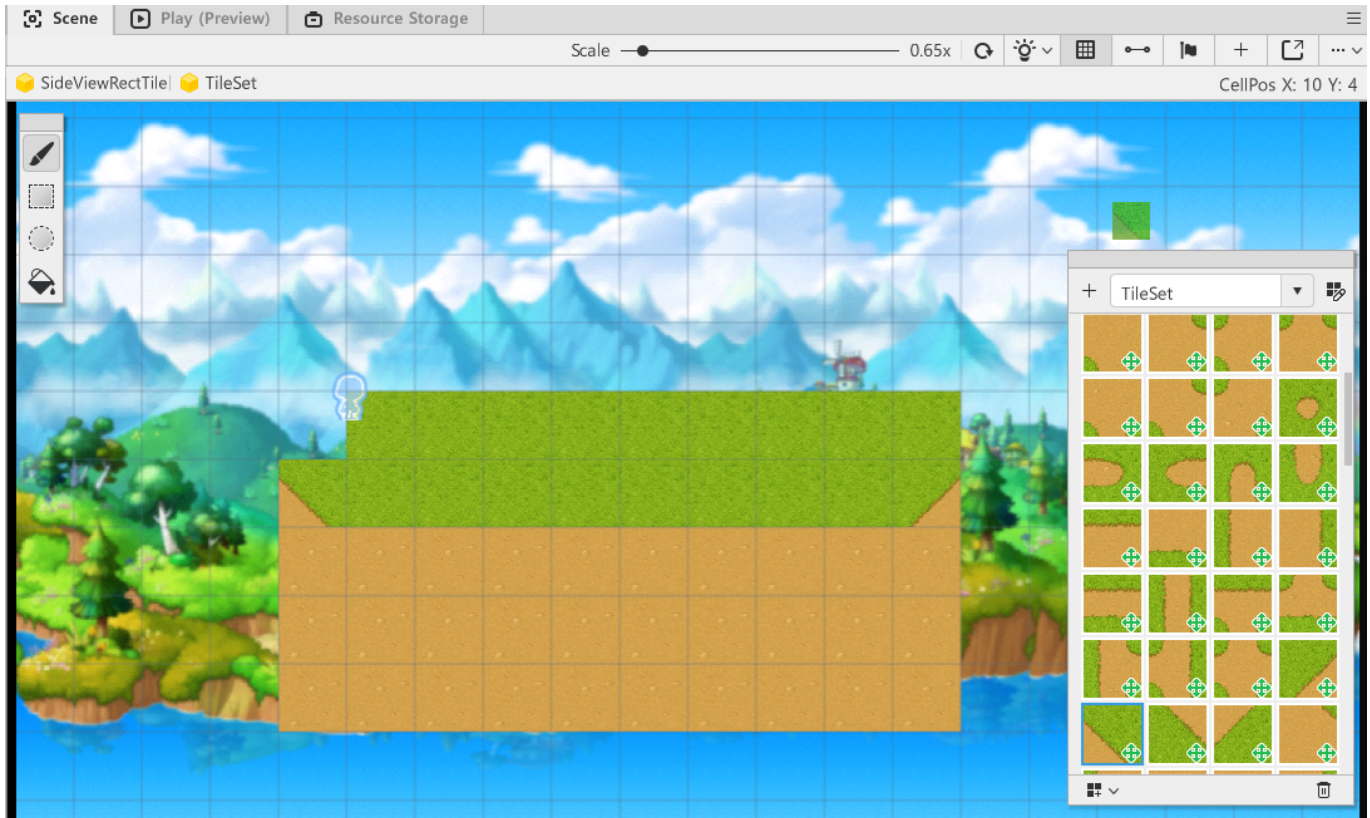
- 메이플스토리 월드가 제공하는 Tile을 활용하여 발판을 생성하고, 엔티티를 배치하여 맵을 형성할 수 있습니다.
- 타일의 배치에 따라 경사로를 제작할 수 있습니다.
- 횡 스크롤 형태의 게임을 제작할 때, 메이플스토리의 느낌을 살리고 싶을 때 유용한 맵 방식입니다.

RectTileMap



- 탑다운뷰 형태의 게임을 제작할 때 효과적인 방식의 맵 형식입니다.
- 직접 타일셋을 제작하거나, 메이플스토리 월드에서 제공하는 타일셋 리소스를 활용하여 맵을 형성할 수 있습니다.
- 바둑판 형태로 타일을 배치하여 맵을 형성할 수 있습니다.
- 타일마다 이동 가능과 이동 불가능한 성질을 부여할 수 있습니다.
- TileMap, SideViewRectTileMap 방식과 다르게 4방향, 8방향으로 이동하는 형태의 게임을 제작할 수 있습니다.

SideViewRectTileMap



- 횡 스크롤 방식의 게임을 제작할 때 효과적인 방식의 맵 형식입니다.
- 직접 타일셋을 제작하거나, 메이플스토리 월드에서 제공하는 타일셋 리소스를 활용하여 맵을 형성할 수 있습니다.
- 바둑판 형태로 타일을 배치하여 맵을 형성할 수 있습니다.
- RectTileMap과는 다르게 이동 불가능한 타일셋이 아닐 경우, 캐릭터가 타일을 **통과**합니다.
- 따라서 SideViewRectTileMap으로 맵을 형성 할 경우, 이동 불가능한 타일셋을 **발판**으로 제작해야 합니다.

BodyComponent

- 메이플스토리 월드에서 맵의 발판인 타일들과 상호작용 하기 위해서는 BodyComponent가 필요합니다.
- BodyComponent는 맵의 형식에 따라 다른 BodyComponent를 요구합니다.
- BodyComponent는 크게 3종류의 BodyComponent가 존재합니다.

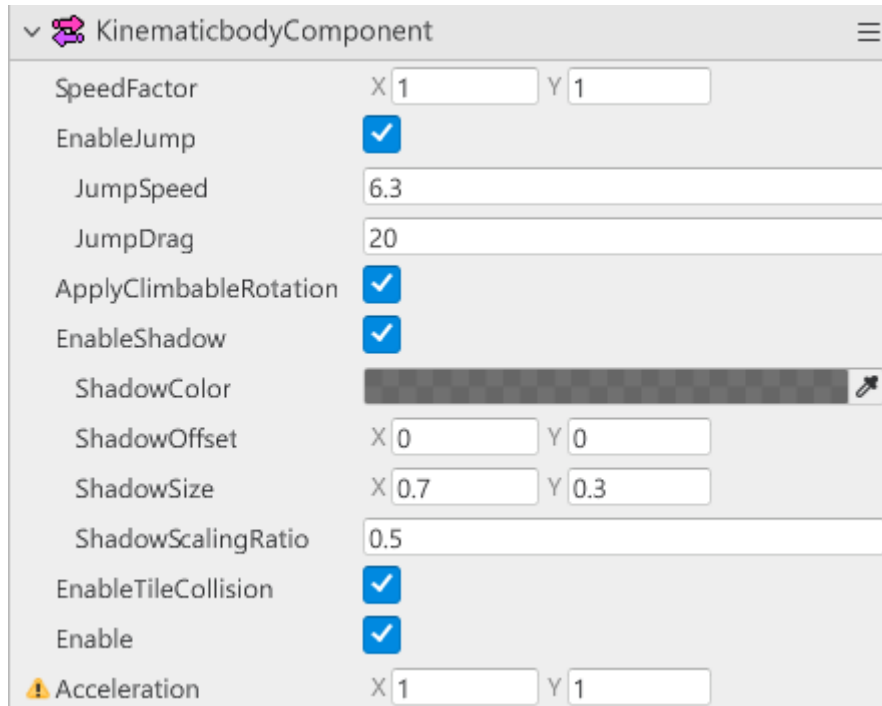
RigidBodyComponent

RigidbodyComponent

Mass	100
Gravity	1
JumpBias	0.03
WalkJump	1.23
DownJumpSpeed	2
WalkSpeed	1.4
WalkAcceleration	1
WalkDrag	1
WalkSlant	0.9
AirAccelerationX	1
AirDecelerationX	1
FallSpeedMaxX	1
FallSpeedMaxY	1
LayerSettingType	All
IsBlockVerticalLine	☐
IgnoreMoveBoundary	☐
ApplyClimbableRotation	☒
IsolatedMove	☐
KinematicMove	☐
Enable	☒

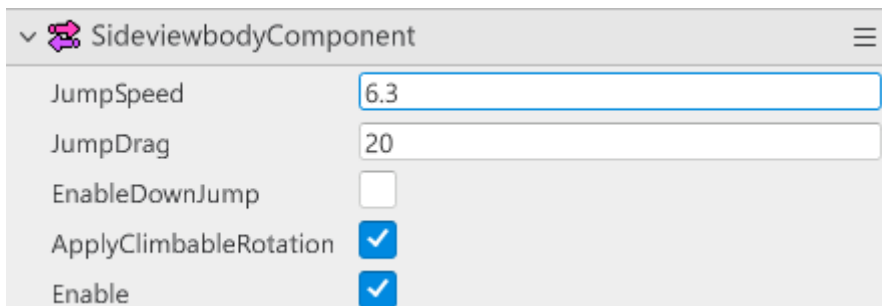
- 메이플스토리 월드의 TileMap 방식에서 캐릭터의 물리를 제어합니다.
- 맵이 TileMap인데 RigidbodyComponent가 없으면, 캐릭터가 타일과 상호작용을 할 수 없습니다.
- RigidbodyComponent내에 캐릭터의 점프 제어, 중력 제어 등 다양한 물리 프로퍼티값을 제공합니다.
- 해당 값을 변경하여 물리 방식을 변경할 수 있습니다.

KinematicBodyComponent



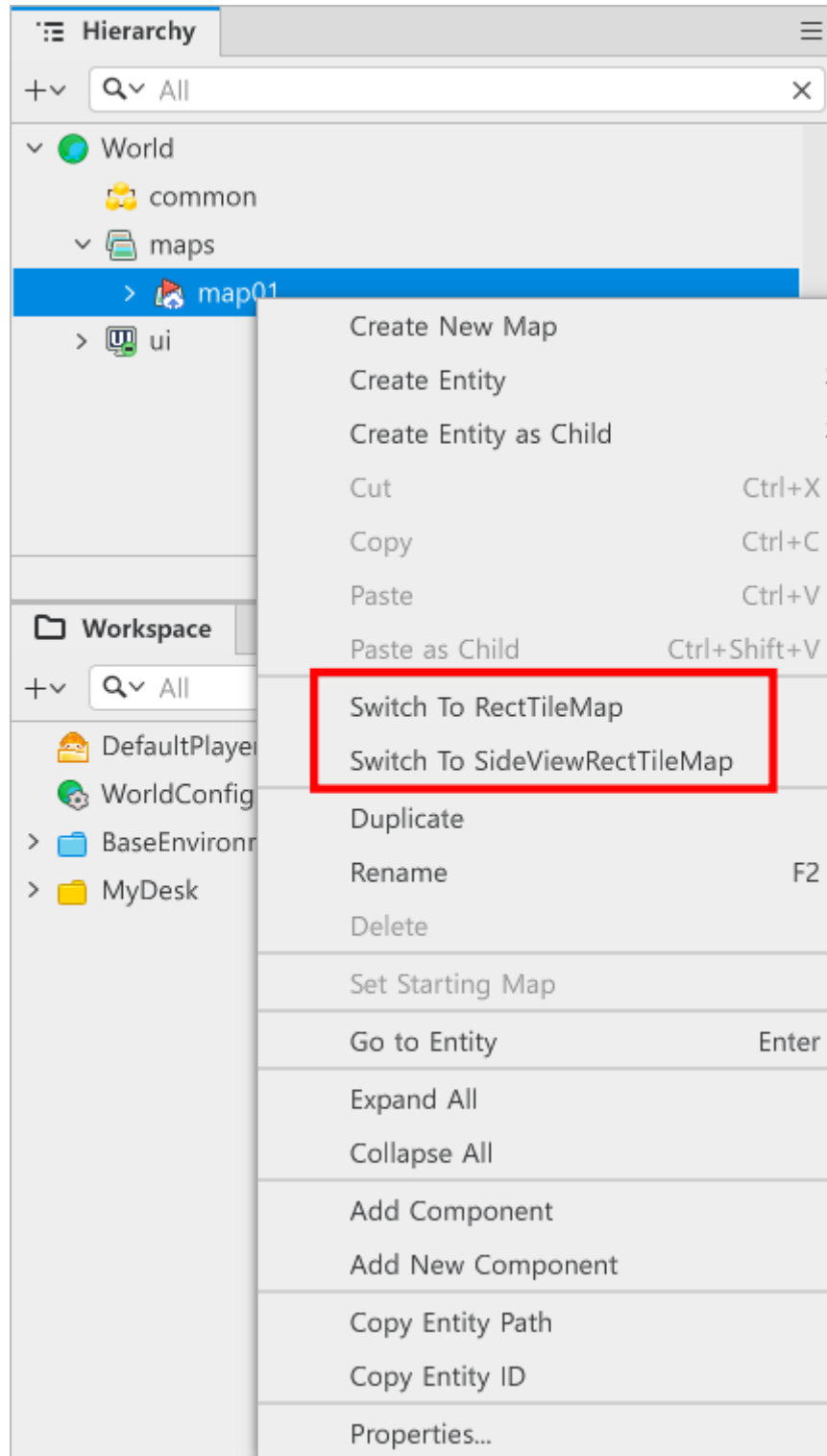
- 메이플스토리 월드의 RectTileMap 방식일 때 캐릭터의 물리를 제어합니다.
- 맵이 RectTileMap일 때 KinematicBodyComponent가 없으면, RectTileMap의 타일들과 상호작용이 일어나지 않습니다.
- KinematicBodyComponent에서 캐릭터의 이동 속도, 점프 여부, 그림자 여부 등을 조절할 수 있습니다.

SideViewRectTileMap



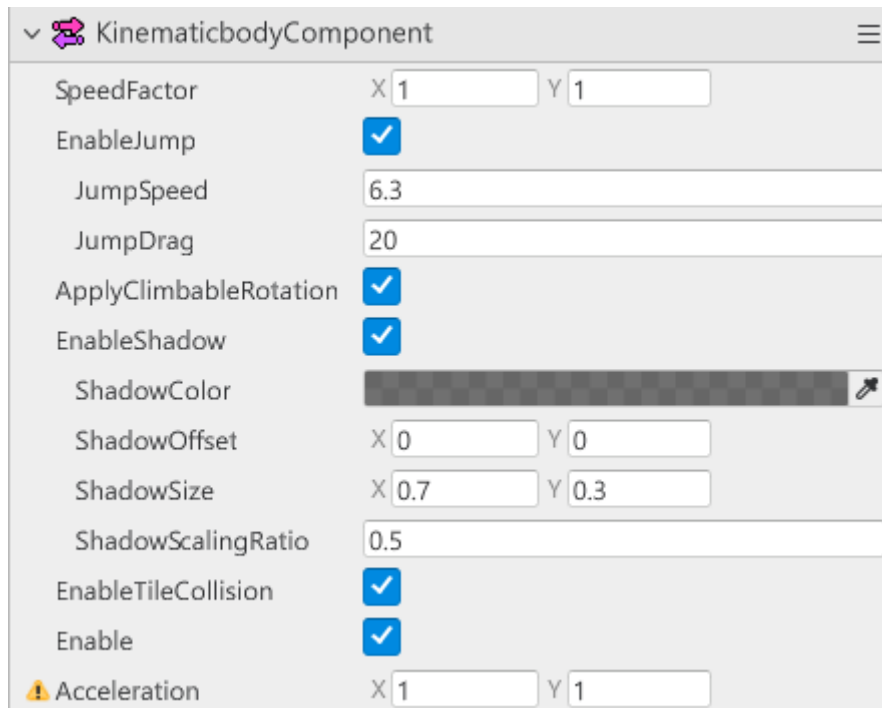
- 메이플스토리 월드에서 SideViewRectTileMap 방식을 사용할 경우, 캐릭터의 물리를 제어합니다.
- 해당 컴포넌트가 존재하지 않을 경우, 캐릭터는 맵의 타일들과 상호작용을 할 수 없습니다.
- 캐릭터의 점프 속도, 중력 가속도 등을 조절할 수 있습니다.

맵 형식 변경하기



- Hierarchy 창에서 변경하려는 맵을 우클릭합니다.
- Switch To RectTileMap, Switch To SideViewRectTileMap 중 하나를 클릭합니다.
 - 만약 현재 맵이 TileMap이 아닐 경우, Switch To TileMap이 출력되며, 현재 맵 상태로의 전환이 숨김 처리됩니다.
- 슈팅 월드 제작을 위해, Switch To RectTileMap을 클릭하여 맵을 RectTileMap으로 전환합니다.

KinematicBodyComponent 요소 살펴보기



- **SpeedFactor:** 캐릭터의 이동속도를 조절할 수 있는 값입니다. X, Y의 값에 따라 좌우, 또는 상하의 이동속도를 조절할 수 있습니다.
- **EnableJump:** 캐릭터의 점프 허용 여부를 결정할 수 있습니다. 해당 값이 체크가 해제되어 있을 때, 캐릭터가 점프할 수 없습니다.
- **JumpSpeed:** 캐릭터의 점프 속도를 조절할 수 있습니다. 값이 클수록 빠른 속도로 점프하여 더 높이 점프할 수 있습니다.
- **JumpDrag:** 캐릭터의 중력 가속도를 조절할 수 있습니다. 값이 작을수록 중력의 힘이 작아져 캐릭터가 공중에서 천천히 지상으로 복귀합니다.
- **ApplyClimbableRotation:** 캐릭터가 사다리, 밧줄과 같은 요소와 상호작용을 할 경우, 사다리 또는 밧줄의 회전 여부에 맞춰 캐릭터를 회전시킬지 결정하는 값입니다.
- **EnableShadow:** 캐릭터의 그림자 출력 여부를 결정합니다. 해당 값이 해제되어 있을 때, 캐릭터의 그림자 출력을 제거합니다.
- **ShadowColor:** 그림자의 색상을 변경할 수 있습니다.
- **ShadowOffset:** 그림자의 출력 위치를 변경할 수 있습니다.
- **ShadowSize:** 그림자의 크기를 설정할 수 있습니다.
- **ShadowScalingRatio:** 캐릭터가 점프했을 때, 그림자의 축소율을 변경할 수 있습니다. 값이 작을수록 그림자의 축소율이 작아집니다.
- **EnableTileCollision:** 캐릭터가 RectTileMap의 타일중, 이동 불가능한 타일을 무시하고 이동할 수 있습니다.

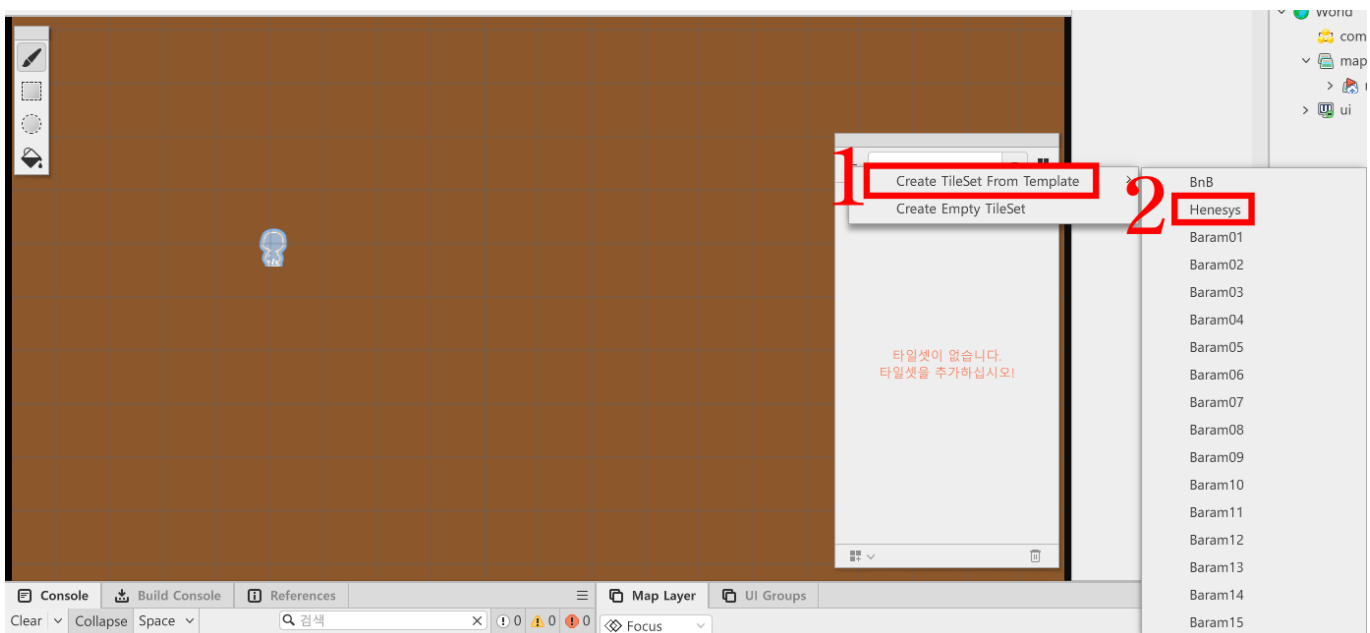
RectTileMap에 타일 배치하기

- 슈팅 월드의 이동 구역을 지정하기 위해 RectTileMap을 배치해야 합니다.
- 만약 처음 RectTileMap으로 변경했다면, 타일을 배치하기 위한 타일셋이 필요합니다.
- 샘플 월드에서는 메이플스토리 월드가 제공하는 리소스를 활용하여 타일셋을 제작하겠습니다.
- 만약 여러분들이 제작하거나 사용하려는 리소스가 이미 있다면, 직접 타일셋을 제작하여 사용할 수 있습니다.

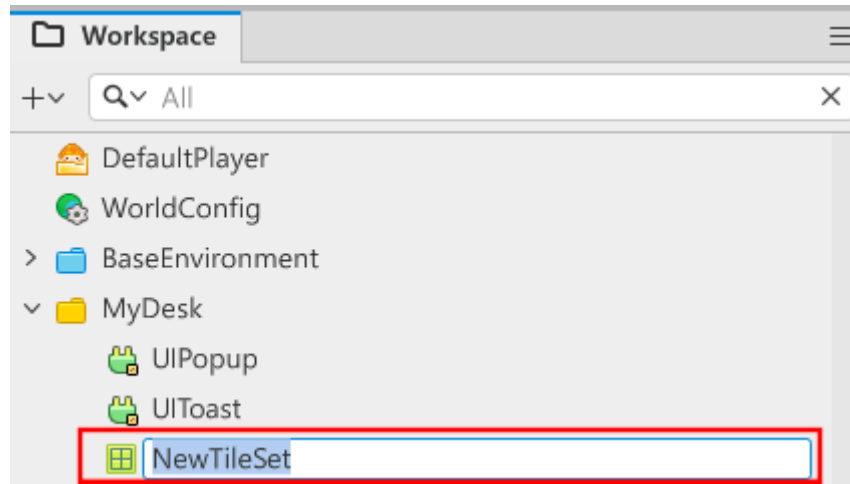
타일셋 제작하기



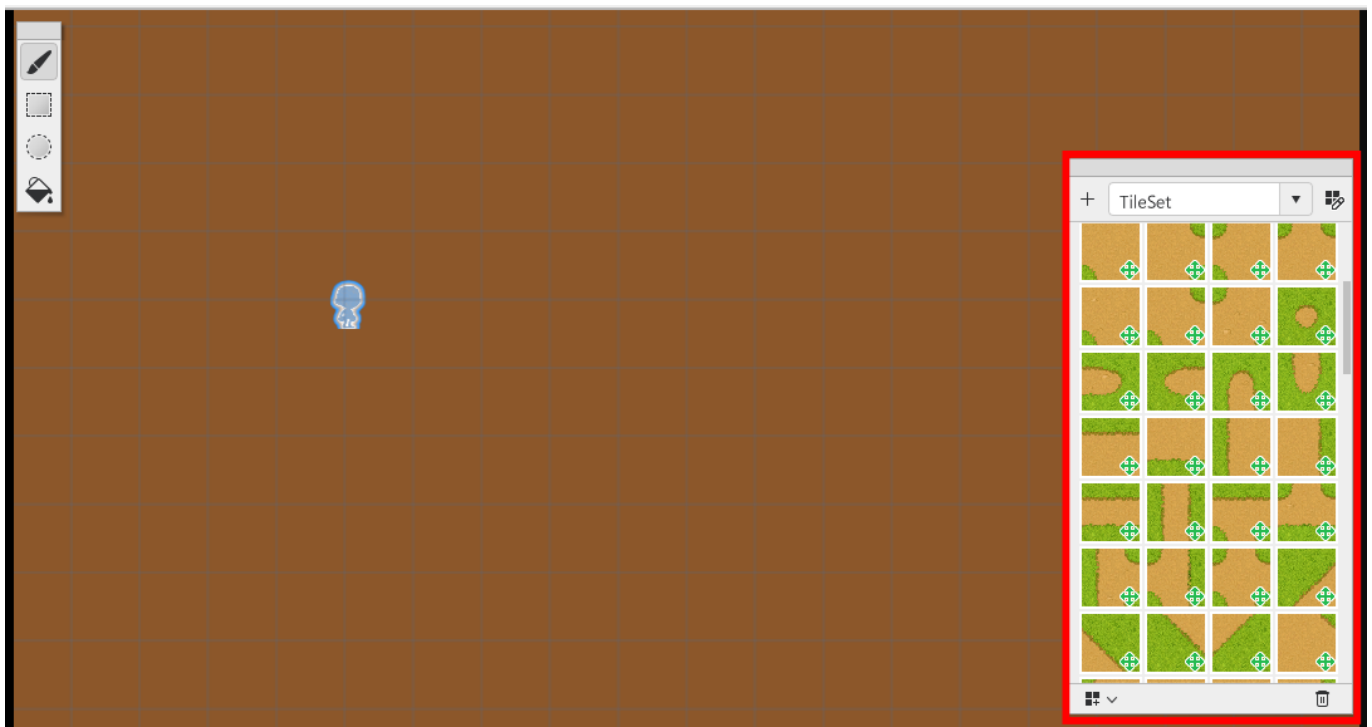
- RectTileMap 상태에서 출력되는 타일셋 에디터 화면의 +버튼을 클릭합니다.



- Create TileSetFrom Template를 클릭합니다.
- 이후 원하는 리소스를 선택합니다.
 - 샘플 월드에서는 Henesys를 사용하였습니다.
- 만약 타일셋으로 사용하려는 리소스가 따로 있을 경우, Create Empty TileSet을 통해 타일셋을 직접 제작하시면 됩니다.



- Workspace에 추가된 NewTileSet의 이름을 변경하여 타일셋 제작을 마무리합니다.

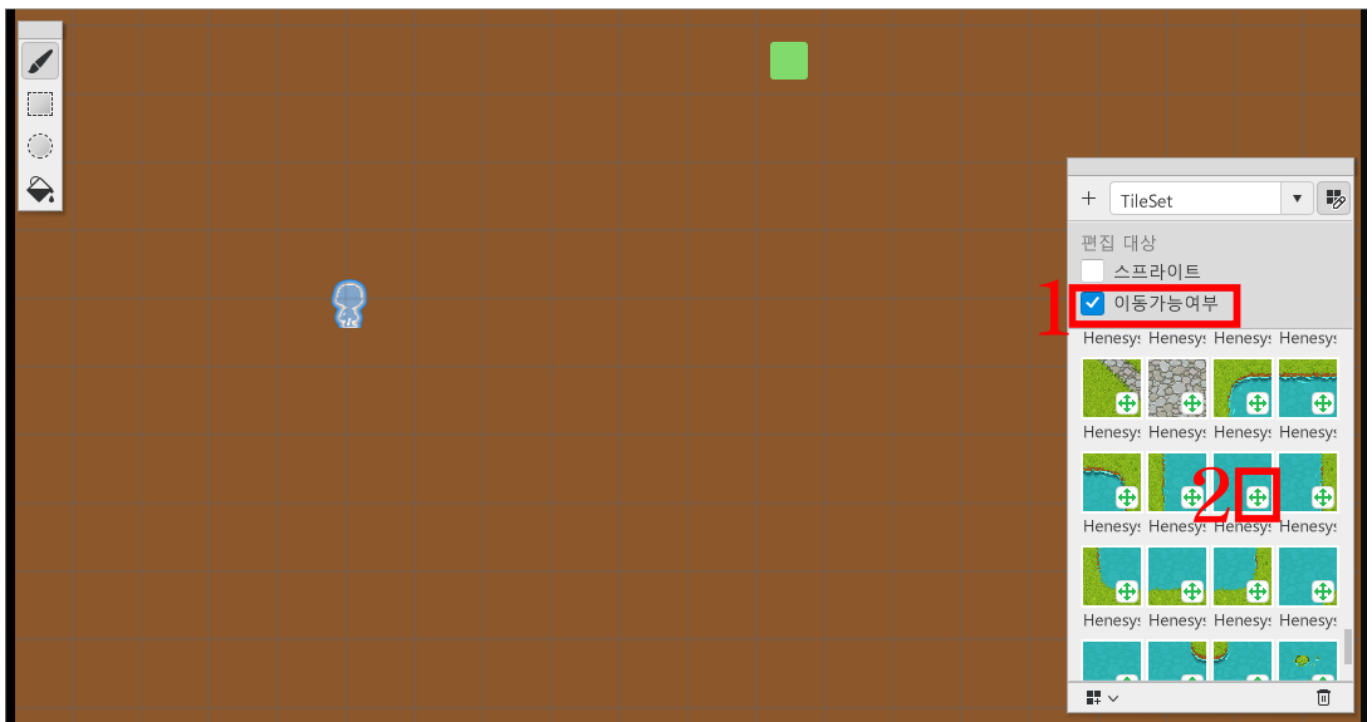


- 정상적으로 타일셋 제작이 마무리되었다면, 타일셋 에디터 화면에 배치할 수 있는 타일들이 출력됩니다.

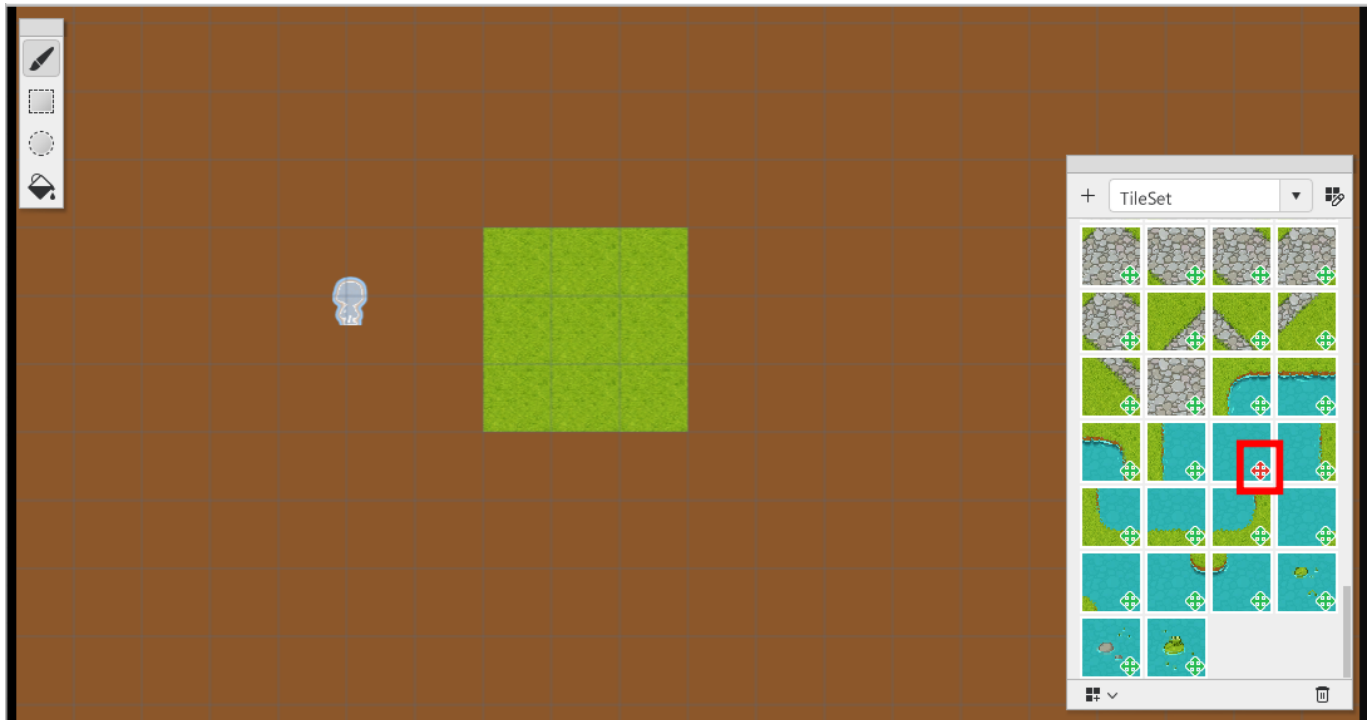
타일의 이동 가능 여부 변경하기



- 타일셋 에디터 화면의 우측에 있는 타일 편집 버튼을 클릭합니다.

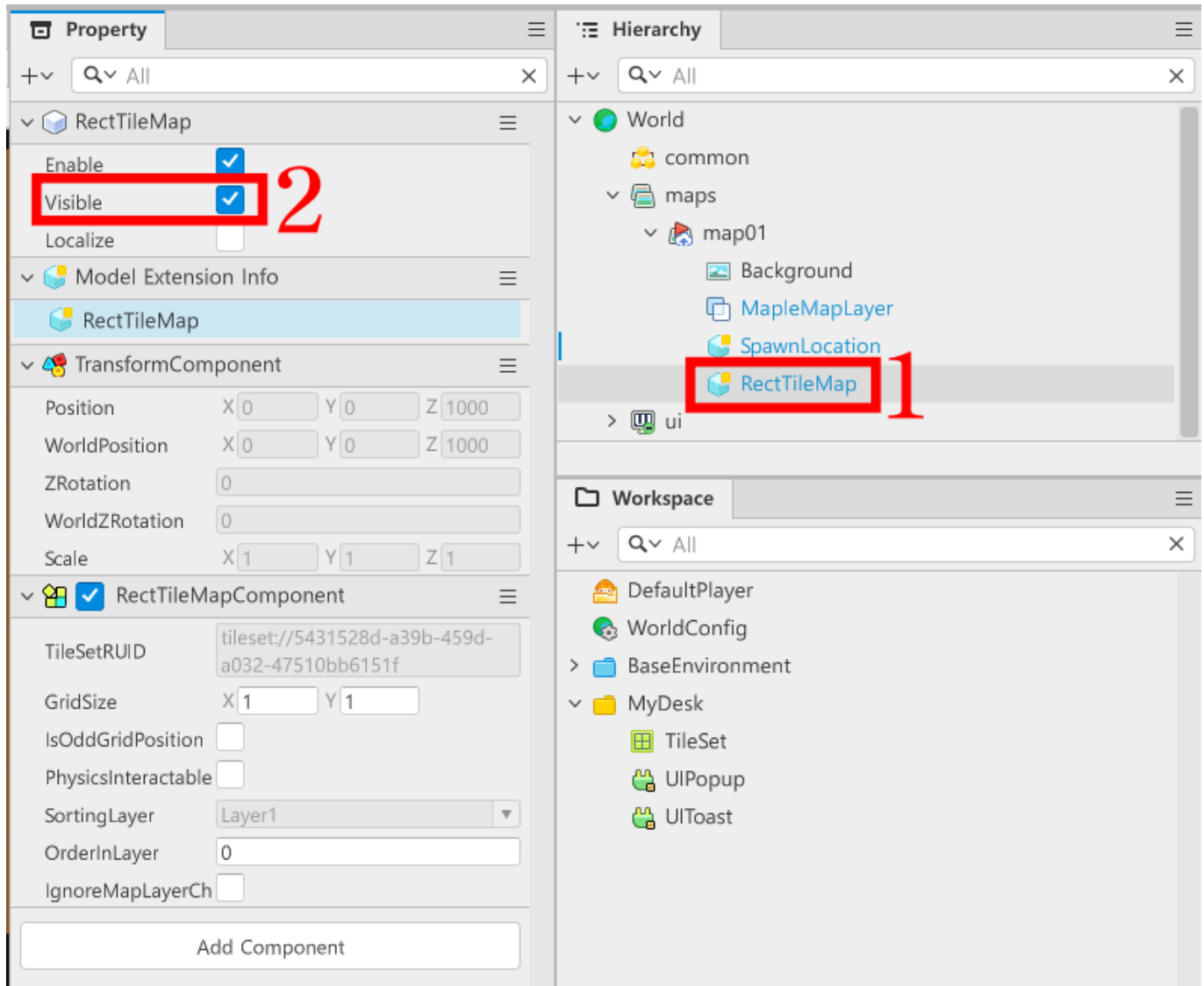


- 편집 대상에서 이동 가능 여부를 클릭합니다.
- 이동 불가능한 타일로 변경하려는 타일의 초록색 십자 U를 클릭하여 붉은색 십자 U로 변경합니다.

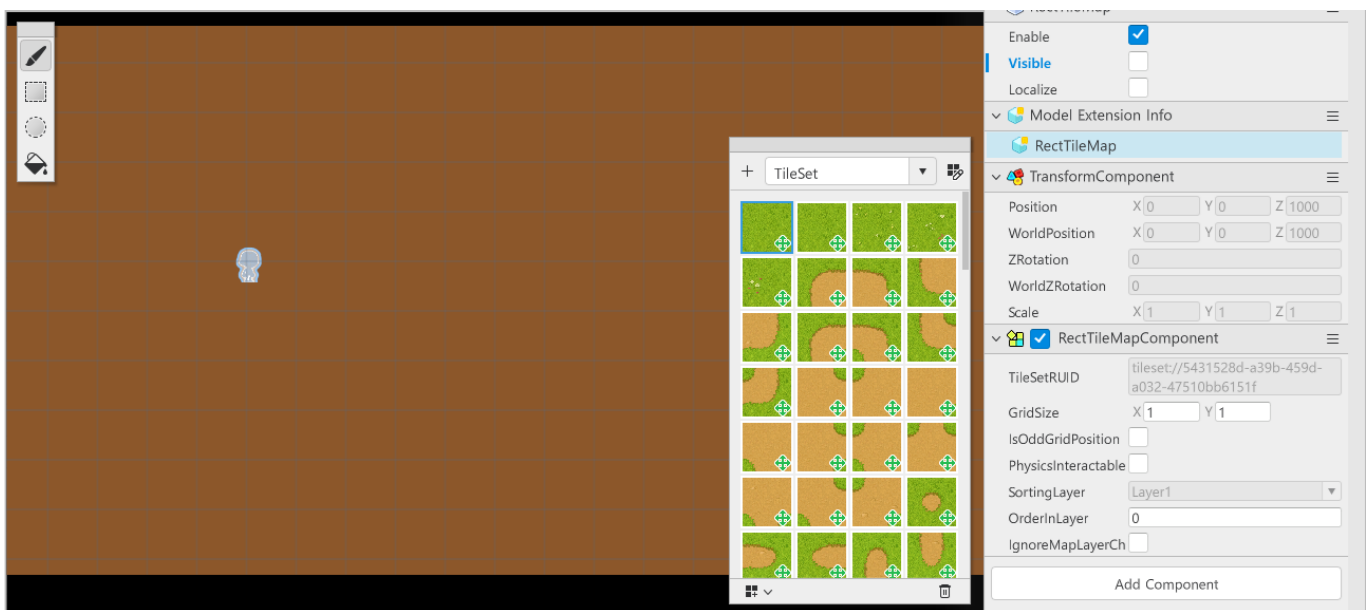


- 정상적으로 변경이 되었다면, 타일 배치 상태에서 붉은색 십자가로 UI가 변경됩니다.
- 해당 타일을 맵에 배치한 뒤, 테스트 플레이를 통해 정상적으로 캐릭터가 이동하지 못하는지 점검합니다.

배치한 타일 맵에서 가리기

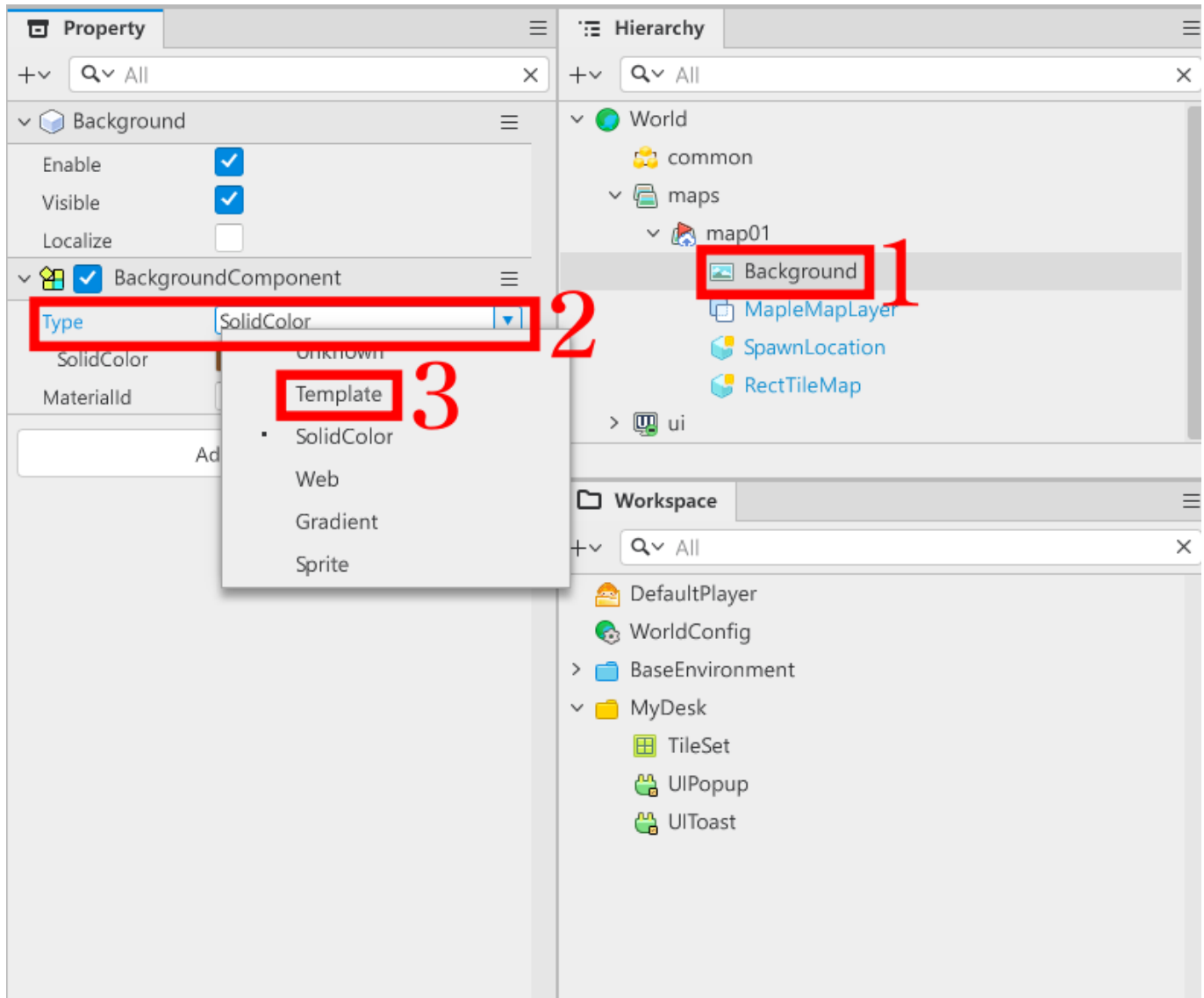


- Hierarchy 창에서 RectTileMap을 찾아 클릭합니다.
- Property 창에서 Visible을 클릭하여 체크 상태를 해제합니다.

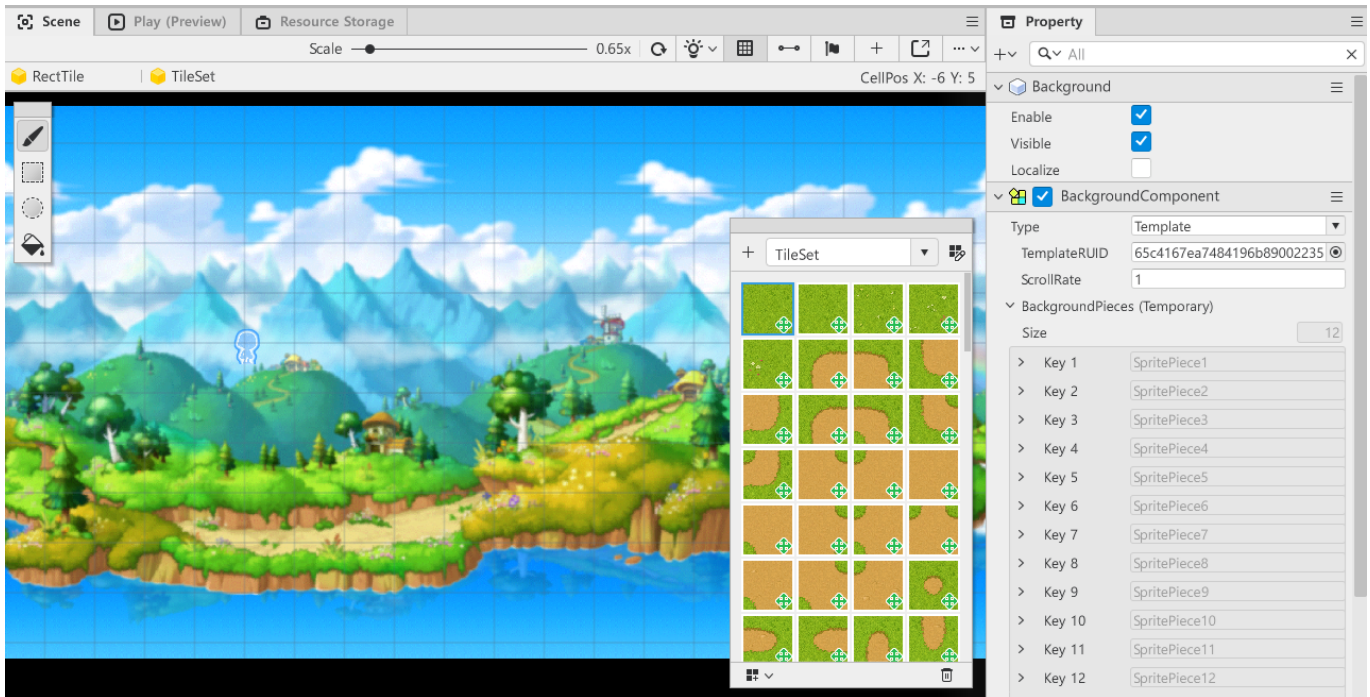


- Visible이 정상적으로 해제되었다면, Scene 화면에서 타일들이 숨김처리된 화면이 출력됩니다.

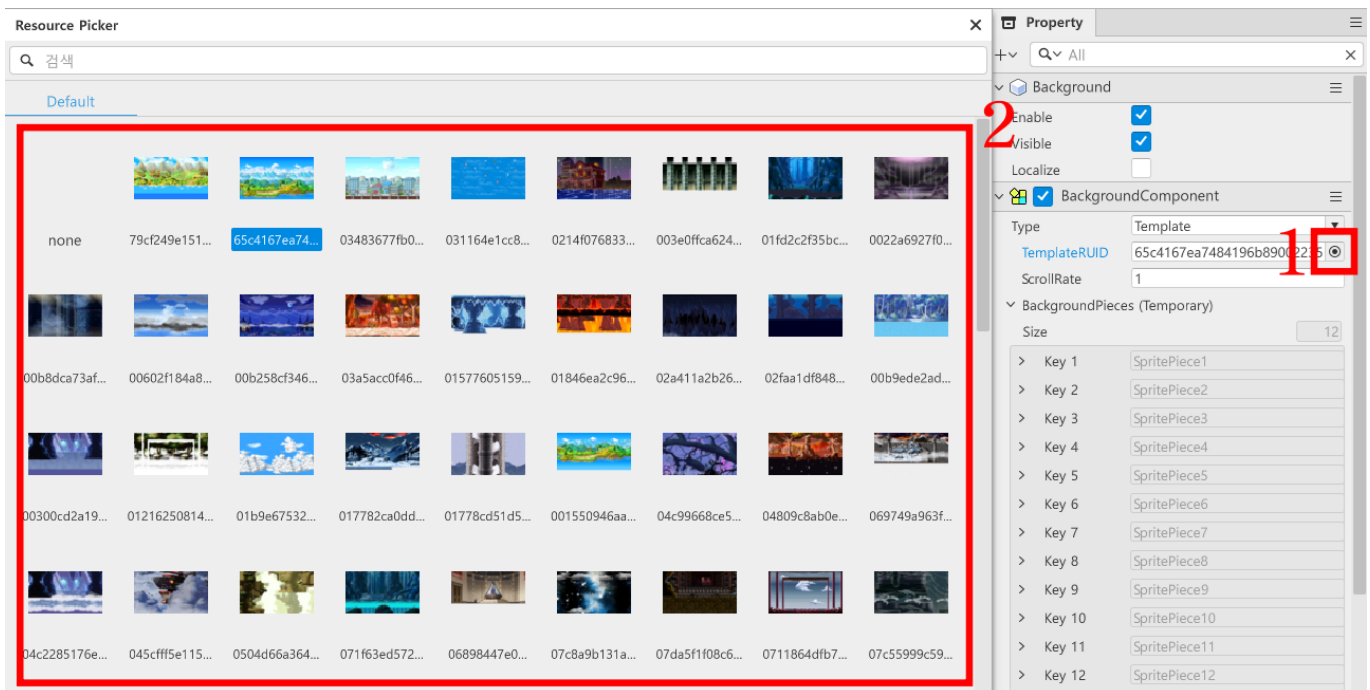
배경 변경하기



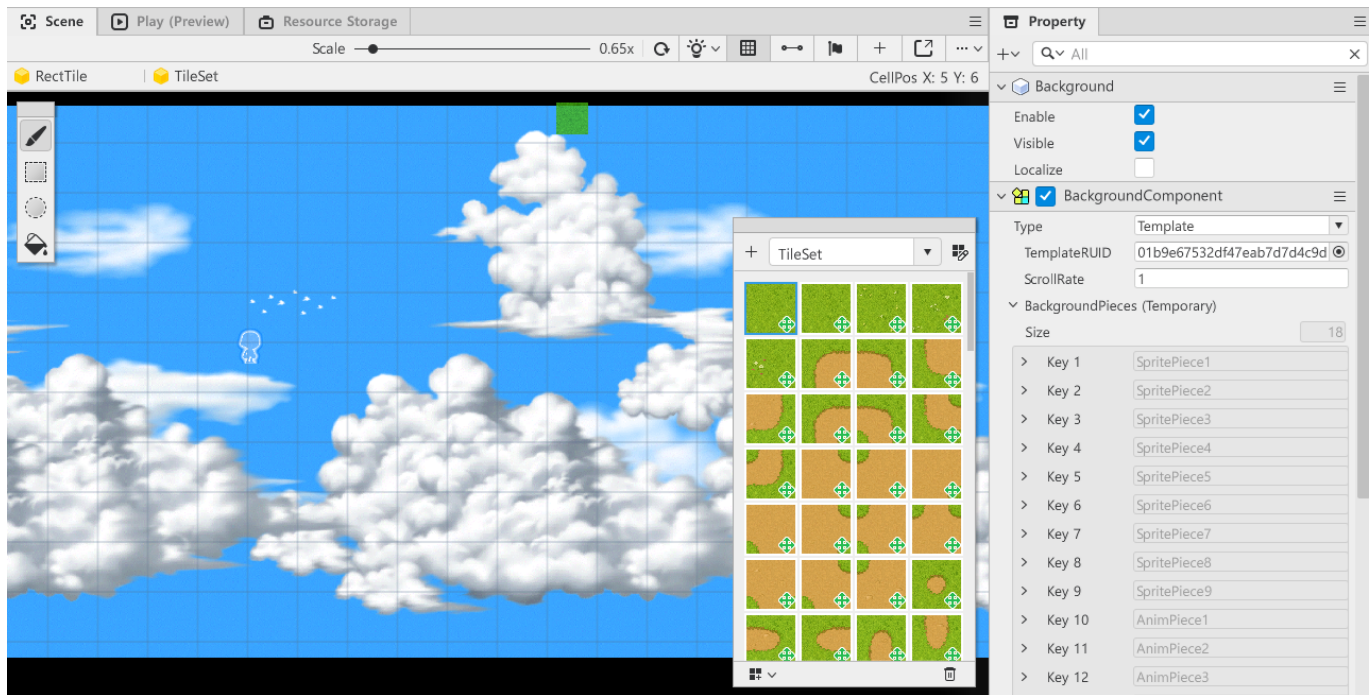
- Hierarchy 창에서 Background를 찾아 클릭합니다.
- Property 창에서 BackgroundComponent를 찾습니다.
- Type을 클릭합니다.
- SolidColor에서 Template로 변경합니다.



- 정상적으로 변경이 완료되었다면, Scene 화면에서 단일 색상의 배경이 아닌, 메이플스토리 월드가 제공 하는 배경으로 변경됩니다.



- 다른 배경을 사용하고자 할 경우, Property 창에서 BackgroundComponent를 찾습니다.
- TemplateRUID의 Resource Picker 버튼을 클릭합니다.
- 원하는 배경을 찾은 뒤 클릭하여 배경을 변경합니다.



- 정상적으로 변경이 완료되면, Scene 화면의 배경이 변경된 모습을 확인할 수 있습니다.

최소 구현 기준

- 실제로 사용하기 위한 슈팅 월드의 맵을 제작합니다.
 - TileSet을 활용하여 이동 가능한 구역과 그렇지 않은 구역을 구분해야 합니다.

응용 및 확장 방향

- 만약 메이플스토리 월드가 제공하는 리소스를 사용 할 경우, 스스로 리소스를 제작한 뒤 자신의 리소스를 활용한 TileSet을 제작합니다.
- 제작한 TileSet을 활용하여 메이플스토리 월드의 리소스가 아닌, 실제 사용 가능한 제작된 맵을 사용할 수 있습니다.
- CameraComponent를 학습하여 시점을 캐릭터에게 따라가지 않도록 고정하는 방법을 찾아서 적용합니다.
 - 메이플스토리 월드 공식 문서, 플레이어 카메라 제어 문서 [링크](#)

[💡 기초 학습]Logic 선언하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- Logic 이해하기: 00:23:57

학습 목표

- 메이플스토리 월드가 제공하는 기능인 Logic의 기능에 대하여 학습합니다.
- Manager 개념을 이해하고, 기능 단위의 Logic을 설계할 수 있습니다.
- Logic을 활용하여 게임의 시스템을 제어하는 Logic, 데이터를 관리하는 Logic 등을 제작할 수 있습니다.
- Logic의 장단점을 이해하고, 앞으로 추가적인 Logic을 제작할 때 해당 사항을 고려하여 설계할 수 있습니다.

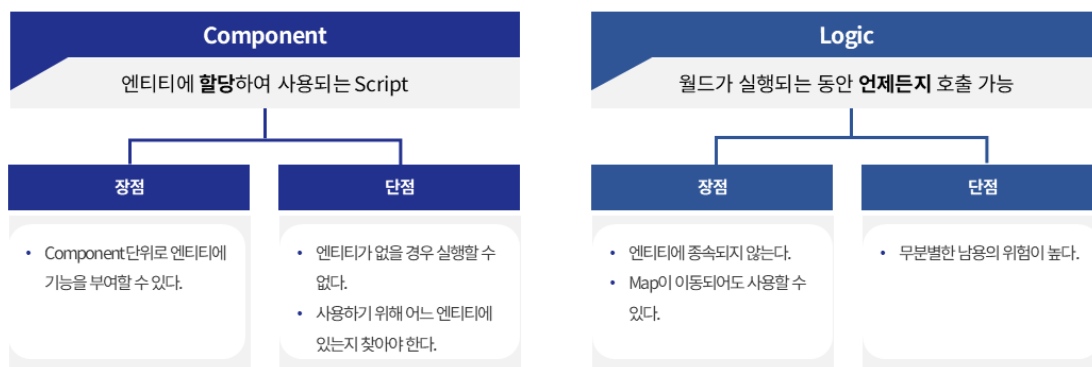
해당 영상 시청 후 수행해 볼 내용

- 빈 엔티티에 Component를 추가한 뒤, 제작한 Logic을 호출하여 Logic의 특성을 직접 체험합니다.
- Manager성격을 가진 Logic을 직접 제작하여, 어떠한 방식으로 해당 Manager를 관리할 지 생각하고 설계합니다.

Logic이란?

Key Point

엔티티의 종속 여부에 따른 Script 차이점



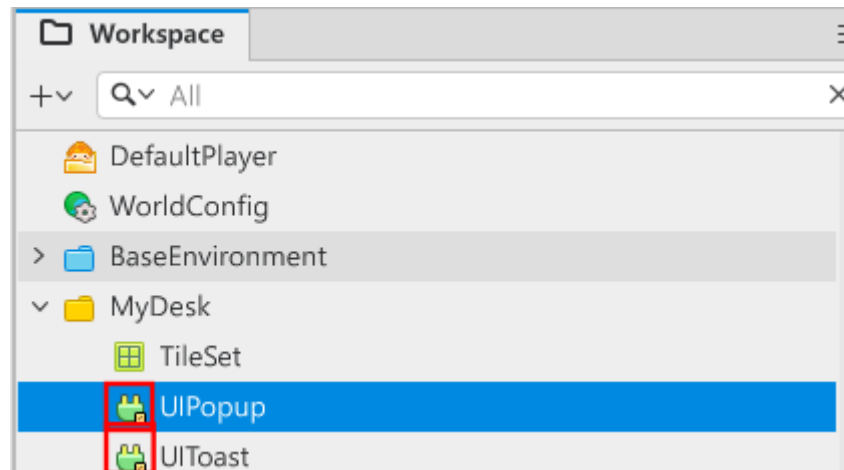
- 메이플스토리 월드가 제공하는 스크립트의 형태 중 하나입니다.
- Logic을 생성할 경우, Workspace 내에서 초록색 플러그 형태의 아이콘으로 표기됩니다.
- 또한 Component와 달리, 엔티티에 부여하지 않아도 스크립트에 접근할 수 있습니다

Manager 개념

- Logic은 특정한 기능이나 역할을 지정하고, 해당하는 기능을 수행하도록 담당자 형태로 제작할 수 있습니다.
- 하나의 기능이나 역할을 담당하는 담당자를 Manager라고 칭합니다
- 만약 특정 기능에 문제가 생겼을 경우, Manager의 내용만 수정하여, 기능을 빠르고 편하게 해결할 수 있습니다.
 - Case 1: 캐릭터가 데미지 계산을 완료하고 몬스터에게 전달하는 경우
 - 데미지 계산식이 변경될 때마다, 데미지 계산에 신규 요소가 추가될 때마다 계산식을 변경해야 합니다.

- 모든 캐릭터를 찾아서 입력된 코드를 수정하고 변경해야 합니다.
- Case 2: 데미지 계산을 Logic으로 제작한 Manager에게 요청하는 경우
 - Logic 내에 있는 데미지 계산식을 변경합니다.
 - 모든 캐릭터, 몬스터의 데미지 계산을 Logic이 담당하므로 더 이상 수정할 요소가 없습니다.
 - 추후 데미지 계산식에 문제가 생겨도, Manager를 수정하면 해결할 수 있습니다.

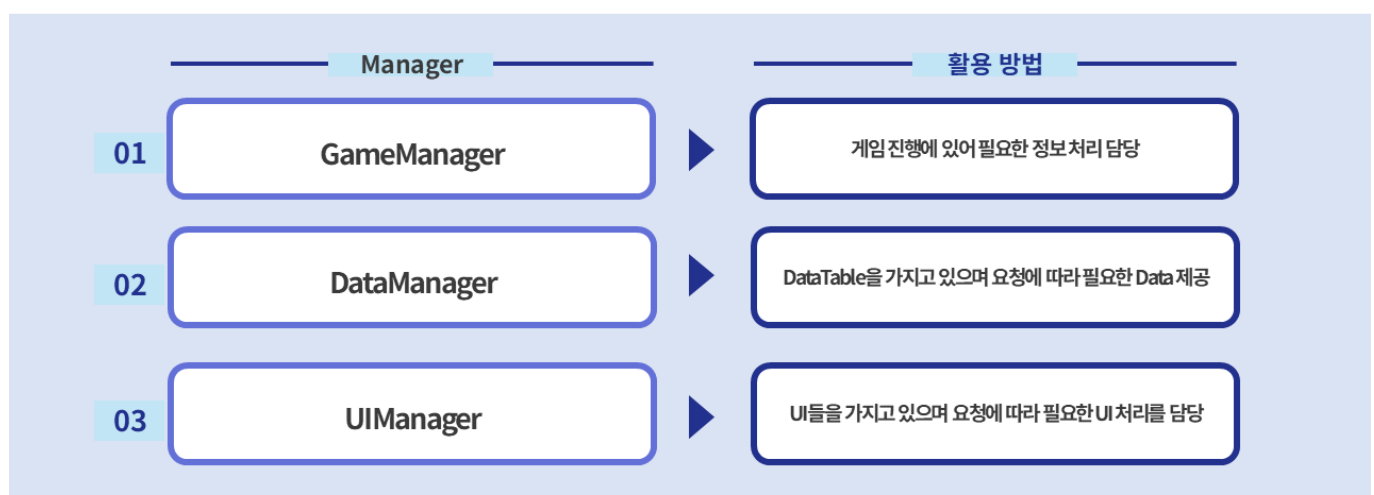
Logic 제작 방법



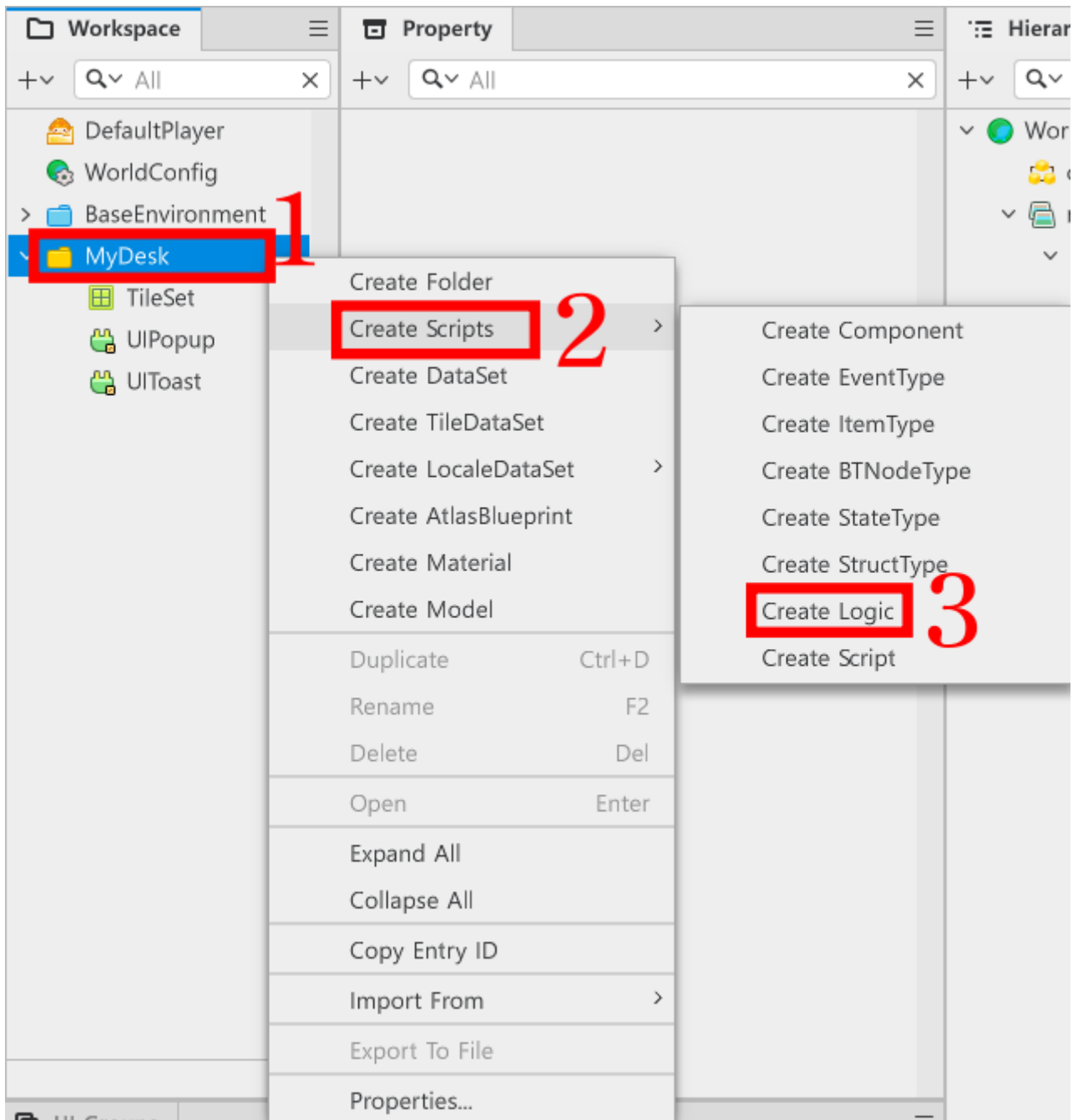
- Workspace에서 MyDesk를 우클릭합니다.
- 메뉴 화면에서 Create Scripts를 클릭합니다.
- Create Logic을 클릭하여 새로운 Logic을 생성합니다.

Key Point

Manager를 통해 특정 기능을 담당하도록 구성



- 생성한 Logic의 이름을 지정하려는 이름으로 변경합니다.



- 이름 변경까지 마쳤다면, Workspace 내에 Logic이 생성된 모습을 볼 수 있습니다.

샘플 월드내 Manager Logic의 사용 용도

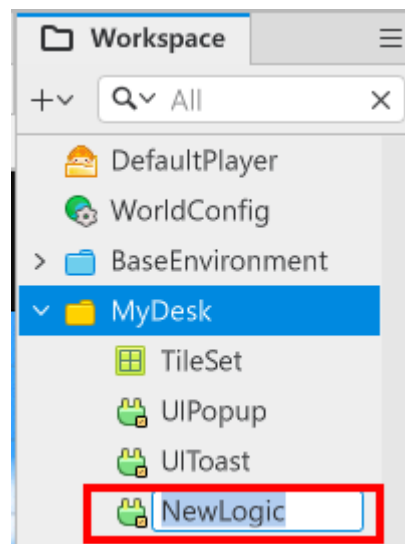
이름	사용 용도
GameLogic	게임의 진행에 따라 처리해야 하는 기능들을 관리하기 위해 제작된 Manager Logic입니다. 게임 클리어 여부, UI 호출, 캐릭터의 Teleport등 게임 진행에 있어 필요한 처리를 담당합니다.
DataTableLogic	DataSet을 관리하고 사용하기 위한 Logic입니다. 데이터가 필요할 때마다 필요한 DataSet을 일일이 로드하기엔 번거롭습니다. 게임이 시작되면 DataTableLogic이 모든 DataSet을 캐싱하며 필요할 때마다 캐싱된 데이터를 반환합니다.

이름	사용 용도
UIManageLogic	UIGroups들을 관리하기 위한 Manager Logic입니다.
컴포넌트 또는 GameLogic이 UI와 관련된 처리가 필요할 경우, 해당 Manager Logic에게 요청을 전달합니다.
UIManageLogic은 모든 UIGroups들의 컴포넌트를 가지고 있으므로, UI와 관련된 모든 요청을 전담합니다.

- 위와 같은 형태로 샘플 월드는 Logic을 생성하고, 사용하고 있습니다.
- 추가적인 기능 제작 또는 확장이 필요할 때, 여러분들만의 Logic을 선언하고 제작하여 사용할 수 있습니다.

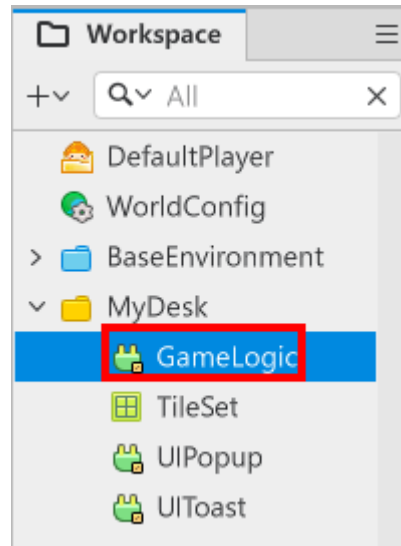
샘플 월드 내 Logic 사용 예시

GameLogic의 경우



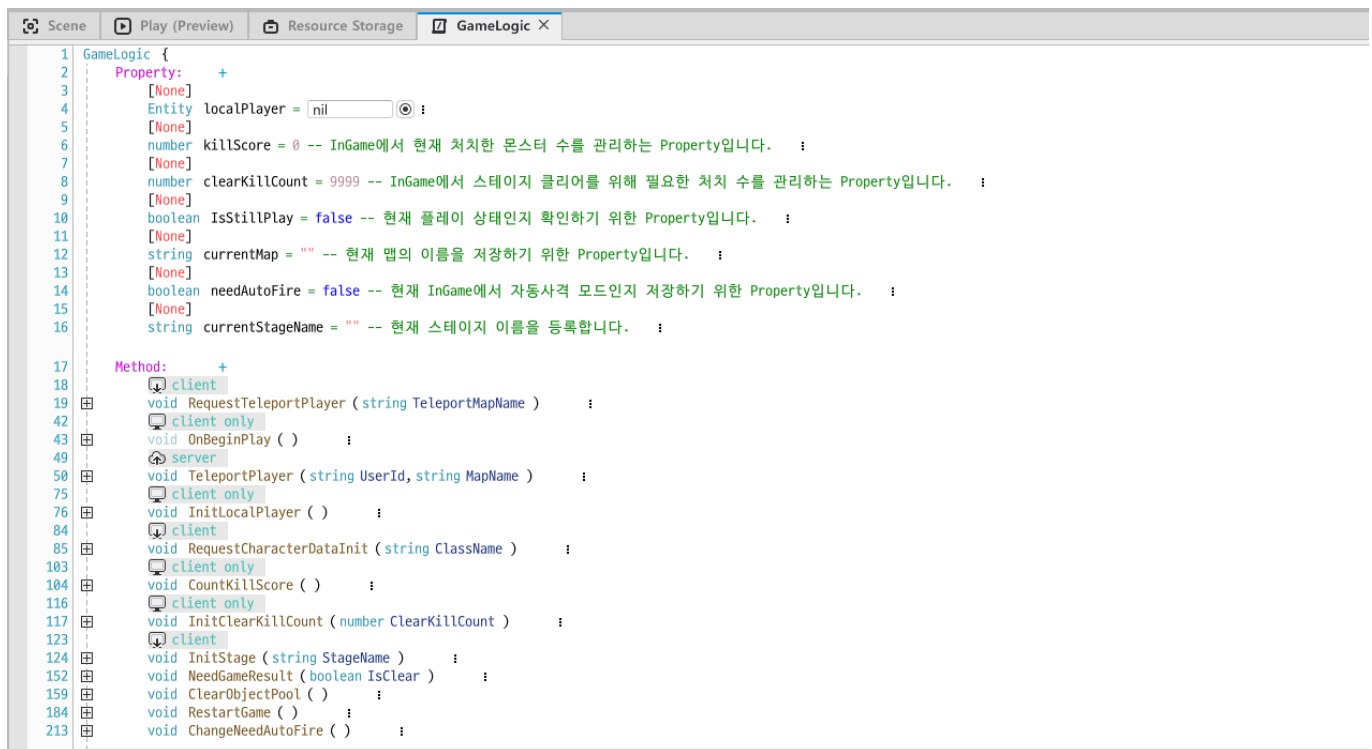
- 유저 엔티티가 있는 맵 위치를 변경하기 위한 기능을 처리하고 있습니다.
- 게임이 시작되면, 현재 스테이지에서 얼마나 적을 더 처리해야 하는지 계산하고, 처리된 수를 카운트합니다.
- 게임이 클리어 또는 게임오버될 경우, 해당하는 UI 연출을 호출하도록 판단합니다.
- 플레이어 캐릭터가 자동 공격을 필요로 하는지 선언하고 변경할 수 있습니다.

DataTableLogic의 경우



- 게임이 시작되면 모든 DataSet을 스스로 캐싱합니다.
- 요청에 따라 캐싱 된 DataSet을 가공된 형태로 다시 반환합니다.

UIManageLogic의 경우



- 게임 시작 시 UIGroups마다 스스로를 관리하는 Component들을 모두 등록합니다.
- 게임이 진행되는 동안 UI와 관련된 처리(UI 보여주기, UI 가리기, UI 업데이트 등)가 있을 때, 요청을 수신 하고 해당하는 UI의 작업을 지시합니다.

GameLogic 코드 작성하기

```
@Logic
script GameLogic extends Logic
```

```

property Entity localPlayer = nil

property number killScore = 0 -- InGame에서 현재 처치한 몬스터 수를 관리하는 Property
입니다.

property number clearKillCount = 9999 -- InGame에서 스테이지 클리어를 위해 필요한 처
치 수를 관리하는 Property입니다.

property boolean IsStillPlay = false -- 현재 플레이 상태인지 확인하기 위한 Property입
니다.

property string currentMap = "" -- 현재 맵의 이름을 저장하기 위한 Property입니다.

property boolean needAutoFire = false -- 현재 InGame에서 자동사격 모드인지 저장하기
위한 Property입니다.

property string currentStageName = "" -- 현재 스테이지 이름을 등록합니다.

@ExecSpace("Client")
method void RequestTeleportPlayer(string TeleportMapName)
log("GameManager RequestTeleportPlayer 시작")

-- 만약 현재 Player가 없을 경우, LocalPlayer를 찾습니다.
if not invalid(self.localPlayer) then
    self:InitLocalPlayer()
    -- 만약 LocalPlayer가 없을 경우, 코드 실행을 중단합니다.
    if not invalid(self.localPlayer) then return end
end

    -- UI가 현재 맵에 따라 변경될 수 있도록 합니다.
if TeleportMapName == "Title" then
    _UIManageLogic:ShowTitle()
elseif TeleportMapName == "InGame" then
    _UIManageLogic:ShowInGame()
end

self.currentMap = TeleportMapName
-- 메이플스토리 월드에서 플레이어 엔티티의 Name은 UserId와 같습니다.
self:TeleportPlayer(self.localPlayer.Name, TeleportMapName)
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 게임이 시작되면 로컬 플레이어를 찾아 등록합니다.
self:InitLocalPlayer()
end

@ExecSpace("Server")
method void TeleportPlayer(string UserId, string MapName)
-- 1. UserId를 통해 서버에서 다시 엔티티를 찾습니다.
log("TeleportPlayer진입 성공")
local playerEntity = _UserService:GetUserEntityByUserId(UserId)
-- 만약 playerEntity를 찾지 못했다면 에러 로그를 출력합니다.
if not invalid(playerEntity) then

```

```

        log("텔레포트 실패: 유저를 찾을 수 없습니다.")
        return
    end

    -- 이동하려는 경로를 지정합니다.
    local targetPath = ""
    if MapName == "Title" then
        targetPath = "/maps/Title"
    elseif MapName == "InGame" then
        targetPath = "/maps/InGame"
    end
    -- 이동 경로가 정상적으로 등록되어있다면, 이동 경로로 플레이어 캐릭터를 텔레포트시킵니다.
    if targetPath ~= "" then
        _TeleportService:TeleportToEntityPath(playerEntity, targetPath)
        log("텔레포트 실행 완료: " .. targetPath)
    end
end

@ExecSpace("ClientOnly")
method void InitLocalPlayer()
    -- 로컬 플레이어를 찾아 등록합니다.
    if not isValid(self.localPlayer) then
        self.localPlayer = _UserService.LocalPlayer
    end
end

@ExecSpace("ClientOnly")
method void RequestCharacterDataInit(string ClassName)
    -- 전달받은 CharacterDefaultData의 ID값을 기준으로, PlayerState를 초기화합니다.
    if ClassName ~= "" and ClassName ~= nil then
        -- DataTableManager에서 ClassName으로 해당하는 데이터 테이블을 찾습니다.
        local findPlayerData =
            _DataTableLogic:GetCharacterDefaultDataByCharacterID(ClassName)
        -- 찾았다면, 찾은 데이터로 PlayerState를 초기화합니다.
        if isValid(findPlayerData) and next(findPlayerData) ~= nil then
            log("CharacterDefaultData Init 요청 완료")
            self.localPlayer.PlayerState:InitPlayerData(findPlayerData)
        -- 만약 찾지 못했다면, log를 남깁니다.
        else
            log("비정상적인 CharacterDefaultData Init 실패!")
        end
    end
end

@ExecSpace("ClientOnly")
method void CountKillScore()
    -- 몬스터가 사망했을 경우, killScore를 1씩 상승시킵니다.
    self.killScore = self.killScore + 1
    -- 만약 killScore가 clearKillCount보다 높을 경우, 스테이지 클리어로 처리합니다.
    if self.killScore >= self.clearKillCount then
        self.IsStillPlay = false
        self:NeedGameResult(true)
        log("게임 클리어")
    end
end

```



```

end
end

@ExecSpace("ClientOnly")
method void InitClearKillCount(number ClearKillCount)
-- 스테이지 클리어 조건을 계산된 몬스터 수로 초기화합니다.
self.clearKillCount = ClearKillCount
end

@ExecSpace("ClientOnly")
method void InitStage(string StageName)
-- StageName값을 기반으로, 해당 맵의 MonsterSpawnManager의 경로를 지정합니다.
local spawnManagerPath = "/maps/"..StageName.."/MonsterSpawnManager"
-- 미리 지정한 경로에서 MonsterSpawnManager 엔티티를 찾습니다.
local stageSpawnManager = _EntityService:GetEntityByPath(spawnManagerPath)
-- 만약 MonsterSpawnManager를 찾는데 실패했을 경우, log를 남기고 return합니다.
if not isValid(stageSpawnManager) then
    log(StageName.."에서 MonsterSpawnManager 오브젝트를 찾을 수 없습니다!")
    return
end
-- ItemHelper가 현재 맵의 ItemSpawnManager를 등록하도록 메소드를 호출합니다.
_ItemHelper:InitData()
-- 현재 상태가 게임 진행중인 상태가 아닐 경우, 게임을 실행시키며 IsStillPlay의 값을
true로 변경합니다.
if not self.IsStillPlay then
    stageSpawnManager.MonsterSpawnManager:StartGame()
    self.IsStillPlay = true
end
-- 현재 몬스터 킬 카운트를 0으로 초기화 합니다.
self.killScore = 0
-- 모든 절차를 마쳤으므로, 전달받은 스테이지 명을 등록합니다.
self.currentStageName = StageName
-- 일시적으로 모든 BGM을 중단합니다.
_SoundService:StopBGM(true)
-- InGame에서 출력해야 하는 BGM을 출력합니다.
_SoundService:PlayBGM("d155c5573a3f46a7b2ffe38629441132", 1.0)
end

@ExecSpace("ClientOnly")
method void NeedGameResult(boolean IsClear)
-- 게임 클리어 결과를 전달하여 결과에 따른 UI 연출을 재생합니다.
self.IsStillPlay = false
_UIManageLogic:ShowResultUI(IsClear)
end

@ExecSpace("ClientOnly")
method void ClearObjectPool()
--만약 InGame이 아닌 다른 맵에서 해당 메소드를 호출 할 경우 아래의 코드를 무시합니다.
if not self.currentMap == "InGame" then
    return
end
-- InGame 맵 안에서 각각의 ObjectPool을 관리하는 Manager들의 경로를 지정합니다.
local missilePoolPath = "/maps/"..self.currentMap.."/ProjectileManager"
local monsterPoolPath = "/maps/"..self.currentMap.."/MonsterSpawnManager"

```

```

-- 위에서 지정한 경로를 기반으로 Manager들을 찾습니다.
local missilePoolEntity = _EntityService:GetEntityByPath(missilePoolPath)
local monsterPoolEntity = _EntityService:GetEntityByPath(monsterPoolPath)
-- 찾는데 성공했다면, 각 오브젝트 풀의 엔티티들을 비활성화 처리합니다.
if invalid(missilePoolEntity) then
    missilePoolEntity.ProjectileManager:ResetPool()
end
if invalid(monsterPoolEntity) then
    monsterPoolEntity.MonsterSpawnManager:ClearAllChild()
end

-- 헬퍼 클래스가 InGame맵이 아닌 곳에서 호출되어 에러를 일으키는 경우를 방지합니다.
_ItemHelper:ReleaseData()
end

@ExecSpace("ClientOnly")
method void RestartGame()
-- 로컬 플레이어를 찾습니다.
local findLocalPlayer = _UserService.LocalPlayer
-- 만약 로컬 플레이어를 찾을 수 없을 경우, log를 출력한 뒤 return합니다.
if not invalid(findLocalPlayer) then
    log("LocalPlayer를 찾을 수 없습니다!")
    return
end
-- 만약 로컬 플레이어 엔티티에 PlayerState 컴포넌트가 없을 경우, log를 출력한 뒤
return합니다.
if not invalid(findLocalPlayer.PlayerState) then
    log("LocalPlayer에게 PlayerState 컴포넌트를 찾을 수 없습니다!")
    return
end

--PlayerState를 받은 뒤 현재 캐릭터의 ID값을 찾습니다.
local currentPlayerCharacter = findLocalPlayer.PlayerState.characterID
-- 다시 시작을 위해 ObjectPool을 초기화합니다.
self:ClearObjectPool()
-- 재시작 하려는 스테이지의 데이터를 초기화합니다.
self:InitStage(self.currentStageName)
-- 캐릭터 정보를 최초 상태로 돌리기 위해 다시 데이터를 초기화합니다.
self:RequestCharacterDataInit(currentPlayerCharacter)
-- UI 상태를 인게임 시작 상태로 전환합니다.
_UIManageLogic:ShowInGame()
-- 게임이 다시 시작되었으므로, IsStillPlay의 상태를 true로 전환합니다.
self.IsStillPlay = true
end

@ExecSpace("ClientOnly")
method void ChangeNeedAutoFire()
-- 해당 함수가 호출 될 경우, 현재 needAutoFire의 값을 반대 값으로 변경합니다.
self.needAutoFire = not self.needAutoFire
end

end

```

DataTableLogic 코드 작성하기

```

@Logic
script DataTableLogic extends Logic

property table missileData = {} -- MissileData 데이터셋을 관리하기 위한 Property입니다.

property table characterDefaultData = {} -- CharacterDefaultData 데이터셋을 관리하기 위한 Property입니다.

property table characterSelectUIData = {} -- CharacterSelectUIData 데이터셋을 관리하기 위한 Property입니다.

property table monsterDefaultData = {} -- MonsterDefaultData 데이터셋을 관리하기 위한 Property입니다.

property table monsterSpawnData = {} -- MonsterSpawnData 데이터셋을 관리하기 위한 Property입니다.

property table skillData = {} -- SkillData 데이터셋을 관리하기 위한 Property입니다.

property table itemData = {} -- ItemData 데이터셋을 관리하기 위한 Property입니다.

property table itemDropData = {} -- ItemDropData 데이터셋을 관리하기 위한 Property입니다.

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- MissileData 데이터셋을 초기화합니다.
self:InitMissileData()
-- CharacterSelectUIData 데이터셋을 초기화합니다.
self:InitCharacterSelectUIData()
-- MonsterSpawnData 데이터셋을 초기화합니다.
self:InitMonsterSpawnData()
-- MonsterDefaultData 데이터셋을 초기화합니다.
self:InitMonsterDefaultData()
-- CharacterDefaultData 데이터셋을 초기화합니다.
self:InitCharacterDefaultData()
-- SkillData 데이터셋을 초기화합니다.
self:InitSkillData()
-- ItemData 데이터셋을 초기화합니다.
self:InitItemData()
-- ItemDropData 데이터셋을 초기화합니다.
self:InitItemDropData()
end

@ExecSpace("ClientOnly")
method void InitMissileData()

```

```

-- 미사일 데이터 테이블 파싱이 되어있는지 점검합니다.
if isValid(self.missileData) == false or next(self.missileData) == nil then
    -- 미사일 데이터 파싱이 필요하므로, MissileData 데이터 테이블을 찾습니다.
    local missileDefaultData = _DataService:GetTable("MissileData")
    if isValid(missileDefaultData) == nil then
        -- MissileData테이블을 찾을 수 없으므로 로그를 출력한 뒤 Return 합니다.
        log("MissileData 데이터 테이블을 찾을 수 없습니다!")
        return
    end
    -- 전체 Row 개수를 계산합니다.
    local rowCount = missileDefaultData:GetRowCount()

    -- Key가 될 MissileID를 우선 데이터 테이블 Row에서 1번부터 순서대로 대입합니다.
    for i = 1, rowCount do
        local missileID = missileDefaultData:GetCell(i, "MissileID")

        -- 유효한 ID인지 체크합니다.
        if missileID ~= nil and missileID ~= "" then

            -- 해당 행의 모든 정보를 하나의 table로 묶습니다.
            local rowData =
            {
                MissileID = missileID,
                AtkValue = tonumber(missileDefaultData:GetCell(i, "AtkValue")),
                MoveSpeed = tonumber(missileDefaultData:GetCell(i, "MoveSpeed")),
                LifeTime = tonumber(missileDefaultData:GetCell(i, "LifeTime")),
                HitEffectRUID = tostring(missileDefaultData:GetCell(i,
"HitEffectRUID"))),
                ModelRUID = tostring(missileDefaultData:GetCell(i, "ModelRUID"))
            }

            -- missileData Property에 등록합니다. (Key: MissileID, Value: 데이터)
            self.missileData[missileID] = rowData
        end
    end
    -- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
    log("MissileData 데이터 테이블 캐싱 완료!")
    return
end

-- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
log("MissileData 데이터 테이블이 이미 캐싱되어 있거나, MissileData 데이터 테이블을 찾
지 못했습니다.")
end

@ExecSpace("ClientOnly")
method void InitCharacterDefaultData()
    -- 플레이어 캐릭터 파싱이 완료되어 있는지 체크하기 위한 항목
    if isValid(self.characterDefaultData) == false or next(self.characterDefaultData)
== nil then
        local getTableData = _DataService:GetTable("CharacterDefaultData")
        --데이터 테이블 가져오기에 실패했을 경우 경고 문구 출력
        if getTableData == nil then
            log("CharacterDefaultData 데이터 테이블을 가져오는데 실패했습니다!")
            return
        end
    end
end

```

```

end
--파싱 시작
local rowCount = getTableData:GetRowCount() -- 전체 행 개수를 계산합니다.

for i = 1, rowCount do
    -- Key가 될 CharacterID를 우선 데이터 테이블 Row에서 하나를 대입합니다.
    local characterID = tostring(getTableData:GetCell(i, "CharacterID"))

    -- 유효한 ID인지 체크합니다.
    if characterID ~= nil and characterID ~= "" then

        -- 해당 행의 모든 정보를 테이블로 묶습니다.
        local rowData =
        {
            CharacterID = characterID,
            StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
            StartSkillCount = tonumber(getTableData:GetCell(i,
"StartSkillCount")),
            StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
            DefaultMissile = tostring(getTableData:GetCell(i,
"DefaultMissile")),
            DefaultSkill = tostring(getTableData:GetCell(i, "DefaultSkill")),
            AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval"))
        }

        -- 캐시 테이블에 저장 (Key: MissileID, Value: 데이터)
        self.characterDefaultData[characterID] = rowData
    end
end
-- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
log("CharacterDefaultData 데이터 테이블 캐싱 완료!")
return
end
-- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
log("CharacterDefaultData 데이터 테이블이 이미 캐싱되어 있거나, CharacterDefaultData
데이터 테이블을 찾지 못했습니다.")
end

@ExecSpace("ClientOnly")
method void InitCharacterSelectUIData()
    -- 캐릭터 선택창 정보 테이블이 캐싱되어있는지 점검합니다.
    if isValid(self.characterSelectUIData) == false or
next(self.characterSelectUIData) == nil then
        -- 캐릭터 선택창 정보 테이블이 등록되어 있지 않으므로 해당 부분을 캐싱하기 위해 데이
터 테이블을 찾습니다.
        local characterSelectData = _DataService:GetTable("CharacterSelectUIData")
        -- 캐릭터 선택창 정보 테이블을 찾지 못했을 경우 log로 에러를 출력합니다.
        if characterSelectData == nil then
            log("CharacterSelectUIData 데이터 테이블을 찾지 못했습니다!")
            return
        end
    end
    -- 정상적으로 찾은 경우 데이터 테이블의 RowCount를 계산합니다.

```

```

        local rowCount = characterSelectData:GetRowCount()
        -- Key가 될 CharacterID를 우선 데이터 테이블 Row에서 1번부터 순서대로 대입합니
다.
        for i = 1, rowCount do
            local characterID = characterSelectData:GetCell(i, "CharacterID")

            -- 유효한 ID인지 체크합니다.
            if characterID ~= nil and characterID ~= "" then

                -- 해당 행의 모든 정보를 하나의 table로 묶습니다.
                local rowData =
                {
                    CharacterID = characterID,
                    LocalizeNameID = tostring(characterSelectData:GetCell(i,
"LocalizeNameID")),
                    SpriteRUID = tostring(characterSelectData:GetCell(i,
"SpriteRUID"))
                }

                -- 캐시 테이블에 저장 (Key: MissileID, Value: 데이터)
                self.characterSelectUIData[characterID] = rowData
            end
        end
        -- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
        log("CharacterSelectUIData 데이터 테이블 캐싱 완료!")
        return
    end
    -- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
    log("CharacterSelectUIData 데이터 테이블이 이미 캐싱되어 있거나, MissileData 데이터
테이블을 찾지 못했습니다.")
end

@ExecSpace("ClientOnly")
method void InitMonsterDefaultData()
    -- 몬스터 데이터 파싱이 완료되어 있는지 체크합니다.
    if isValid(self.monsterDefaultData) == false or next(self.monsterDefaultData) ==
nil then
        local getTableData = _DataService:GetTable("MonsterDefaultData")
        --데이터 테이블 가져오기에 실패했을 경우 경고 문구를 출력합니다.
        if getTableData == nil then
            log("MonsterDefaultData 데이터 테이블을 가져오는데 실패했습니다!")
            return
        end
        -- 전체 행 개수를 계산합니다.
        local rowCount = getTableData:GetRowCount()

        for i = 1, rowCount do
            -- Key가 될 MissileID를 우선 데이터 테이블 Row에서 하나를 대입합니다.
            local monsterID = getTableData:GetCell(i, "MonsterID")

            -- 유효한 ID인지 체크합니다.
            if monsterID ~= nil and monsterID ~= "" then

                -- 해당 행의 모든 정보를 테이블로 묶습니다.

```

```

        local rowData =
        {
            monsterID = monsterID,
            StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
            StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
            DefaultMissile = getTableData:GetCell(i, "DefaultMissile"),
            AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval")),
            ModelRUID = tostring(getTableData:GetCell(i, "ModelRUID"))
        }

        -- 캐시 테이블에 저장 (Key: MissileID, Value: 데이터)
        self.monsterDefaultData[monsterID] = rowData
    end
end
-- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
log("MonsterDefaultData 데이터 테이블 캐싱 완료!")
return
end
-- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
log("MonsterDefaultData 데이터 테이블이 이미 캐싱되어 있거나, MonsterDefaultData 데이
터 테이블을 찾지 못했습니다.")
end

@ExecSpace("ClientOnly")
method void InitMonsterSpawnData()
-- 몬스터 스폰 데이터 파싱이 완료되어 있는지 체크합니다.
if isValid(self.monsterSpawnData) == false or next(self.monsterSpawnData) == nil
then
    local getTableData = _DataService:GetTable("MonsterSpawnData")
    --데이터 테이블 가져오기에 실패했을 경우 경고 문구 출력
    if getTableData == nil then
        log("MonsterSpawnData 데이터 테이블을 가져오는데 실패했습니다!")
        return
    end
    -- 전체 행 개수를 계산합니다.
    local rowCount = getTableData:GetRowCount()

    for i = 1, rowCount do
        -- Key가 될 MissileID를 우선 데이터 테이블 Row에서 하나를 대입합니다.
        local stageID = getTableData:GetCell(i, "StageID")

        -- 유효한 ID인지 체크합니다.
        if stageID ~= nil and stageID ~= "" then

            -- 해당 행의 모든 정보를 테이블로 묶습니다.
            local rowData =
            {
                StageID = getTableData:GetCell(i, "StageID"),
                MonsterID = tostring(getTableData:GetCell(i, "MonsterID")),
                SpawnPosX = tonumber(getTableData:GetCell(i, "SpawnPosX")),
                SpawnPosY = tonumber(getTableData:GetCell(i, "SpawnPosY")),
                EntranceTime = tonumber(getTableData:GetCell(i, "EntranceTime")),
            }

```

```

        ItemDropRate = tonumber(getTableData:GetCell(i, "ItemDropRate")),
        MustDropItem = tostring(getTableData:GetCell(i, "MustDropItem"))
    }
    -- key값에 등록할 stageID를 number형태로 전환합니다.
    stageID = tonumber(stageID)
    -- 캐시 테이블에 저장합니다. (Key: MissileID, Value: 데이터)
    if self.monsterSpawnData[stageID] == nil then
        self.monsterSpawnData[stageID] = {}
    end
    table.insert(self.monsterSpawnData[stageID], rowData)
end
end
-- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
log("MonsterSpawnData 데이터 테이블 캐싱 완료!")
return
end
-- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
log("MonsterSpawnData 데이터 테이블이 이미 캐싱되어 있거나, MonsterSpawnData 데이터
테이블을 찾지 못했습니다.")
end

@ExecSpace("ClientOnly")
method void InitSkillData()
-- 캐싱된 skillData가 없다면, 데이터 테이블 캐싱을 시도합니다.
if not invalid(self.skillData) or not next(self.skillData) then
    local findData = _DataService:GetTable("SkillData")
    --만약 SkillData를 찾지 못했다면, 로그를 남기고 코드를 나옵니다.
    if not invalid(findData) then
        log("SkillData 로드 실패했습니다!")
        return
    end
    -- 전체 행 개수를 계산합니다.
    local rowCount = findData:GetRowCount()

    for i = 1, rowCount do
        -- Key가 될 MissileID를 우선 데이터 테이블 Row에서 하나를 대입합니다.
        local skillID = findData:GetCell(i, "SkillID")

        -- 유효한 ID인지 체크합니다.
        if skillID ~= nil and skillID ~= "" then

            -- 해당 행의 모든 정보를 테이블로 묶습니다.
            local rowData =
            {
                SkillID = findData:GetCell(i, "SkillID"),
                SkillType = findData:GetCell(i, "SkillType"),
                IsPierce = findData:GetCell(i, "IsPierce"),
                AtkValue = tonumber(findData:GetCell(i, "AtkValue")),
                MoveSpeed = tonumber(findData:GetCell(i, "MoveSpeed")),
                LifeTime = tonumber(findData:GetCell(i, "LifeTime")),
                MaxHitCount = tonumber(findData:GetCell(i, "MaxHitCount")),
                HitInterval = tonumber(findData:GetCell(i, "HitInterval")),
                ModelRUID = findData:GetCell(i, "ModelRUID"),
                HitEffectRUID = findData:GetCell(i, "HitEffectRUID")
            }

```



```

    }

    -- 캐시 테이블에 저장합니다. (Key: MissileID, Value: 데이터)
    self.skillData[skillID] = rowData
end
end
-- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
log("SkillData 데이터 테이블 캐싱 완료!")
return
end
-- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
log("SkillData 데이터 테이블이 이미 캐싱되어 있거나, SkillData 데이터 테이블을 찾지 못했습니다.")
end

@ExecSpace("ClientOnly")
method void InitItemData()
-- ItemData 데이터 테이블 파싱이 되어있는지 점검합니다.
if not isvalid(self.itemData) or not next(self.itemData) then
-- ItemData 데이터 테이블 파싱이 필요하므로, ItemData 데이터 테이블을 찾습니다.
local itemDefaultData = _DataService:GetTable("ItemData")
if not isvalid(itemDefaultData) then
-- ItemData 데이터 테이블을 찾을 수 없으므로 로그를 출력한 뒤 Return 합니다.
log("ItemData 데이터 테이블을 찾을 수 없습니다!")
return
end
-- 전체 Row 개수를 계산합니다.
local rowCount = itemDefaultData:GetRowCount()

-- Key가 될 ItemID를 우선 데이터 테이블 Row에서 1번부터 순서대로 대입합니다.
for i = 1, rowCount do
    local itemID = itemDefaultData:GetCell(i, "ItemID")

    -- 유효한 ID인지 체크합니다.
    if itemID ~= nil and itemID ~= "" then

        -- 해당 행의 모든 정보를 하나의 table로 묶습니다.
        local rowData =
        {
            ItemID = itemID,
            ValueType = tonumber(itemDefaultData:GetCell(i, "ValueType")),
            TargetPlayerStateName = tostring(itemDefaultData:GetCell(i,
"TargetPlayerStateName")),
            NumberValue = tonumber(itemDefaultData:GetCell(i, "NumberValue")),
            StringValue = tostring(itemDefaultData:GetCell(i, "StringValue")),
            SpriteRUID = tostring(itemDefaultData:GetCell(i, "SpriteRUID")),
            GainEffect = tostring(itemDefaultData:GetCell(i, "GainEffect"))
        }

        -- 캐시 테이블에 저장합니다. (Key: ItemID, Value: 데이터)
        self.itemData[itemID] = rowData
    end
end
-- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.

```

```

        log("ItemData 데이터 테이블 캐싱 완료!")
        return
    end
    -- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
    log("ItemData 데이터 테이블이 이미 캐싱되어 있거나, ItemData 데이터 테이블을 찾지 못했습니다.")
    end

@ExecSpace("ClientOnly")
method void InitItemDropData()
    -- ItemDropData 데이터 테이블 파싱이 되어있는지 점검합니다.
    if not isvalid(self.itemDropData) or not next(self.itemDropData) then
        -- ItemDropData 데이터 테이블 파싱이 필요하므로, ItemDropData 데이터 테이블을 찾습니다.
        local itemDropDefaultData = _DataService:GetTable("ItemDropData")
        if not isvalid(itemDropDefaultData) then
            -- ItemDropData 데이터 테이블을 찾을 수 없으므로 로그를 출력한 뒤 Return 합니다.
            log("ItemDropData 데이터 테이블을 찾을 수 없습니다!")
            return
        end
        -- 전체 Row 개수를 계산합니다.
        local rowCount = itemDropDefaultData:GetRowCount()

        -- Key가 될 ItemID 우선 데이터 테이블 Row에서 1번부터 순서대로 대입합니다.
        for i = 1, rowCount do
            local itemID = itemDropDefaultData:GetCell(i, "ItemID")

            -- 유효한 ID인지 체크합니다.
            if itemID ~= nil and itemID ~= "" then

                -- 해당 행의 모든 정보를 하나의 table로 묶습니다.
                -- 주의: GetCell로 가져온 숫자는 문자열일 수 있으므로 수치값일 경우
                tonumber() 사용 권장
                local rowData =
                {
                    ItemID = itemID,
                    DropRate = tonumber(itemDropDefaultData:GetCell(i, "DropRate")),
                }

                -- 캐시 테이블에 저장합니다. (Key: ItemID, Value: 데이터)
                self.itemDropData[itemID] = rowData
            end
        end
        -- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
        log("ItemDropData 데이터 테이블 캐싱 완료!")
        return
    end
    -- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
    log("ItemDropData 데이터 테이블이 이미 캐싱되어 있거나, ItemDropData 데이터 테이블을 찾지 못했습니다.")
    end

@ExecSpace("ClientOnly")

```

```

method table GetMissileDataByMissileID(string MissileID)
-- missileData가 정상적으로 캐싱되어 있는지 확인합니다.
if isvalid(self.missileData) or next(self.missileData) ~= nil then
-- MissileID를 Key로 가지는 Value가 있다면, 해당 값을 반환합니다.
    if self.missileData[MissileID] then
        return self.missileData[MissileID]
    end
-- missileData가 캐싱되어 있지 않을 경우, 에러 로그를 출력합니다.
else
    log("미사일 데이터 파싱이 없는 상태입니다!")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetCharacterDefaultDataByCharacterID(string CharacterID)
-- characterDefaultData가 정상적으로 캐싱되어 있지 않을 경우, 캐싱을 시도합니다.
if self.characterDefaultData == nil then
    self:InitCharacterDefaultData()
end
-- 현재 CharacterDefaultData가 정상적으로 등록되어 있는 상태인지 점검합니다.
if isvalid(self.characterDefaultData) and next(self.characterDefaultData) ~= nil
then
-- CharacterID값을 가지는 데이터가 있을 경우, 해당 데이터를 반환합니다.
    if self.characterDefaultData[CharacterID] then
        return self.characterDefaultData[CharacterID]
    end
-- 해당하는 Key의 데이터가 없을 경우 에러 로그를 출력합니다.
else
    log("PlayerDefaultData 데이터가 없거나 존재하지 않는 "..CharacterID.."값입니다!")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetCharacterSelectUIDataTable()
-- 캐릭터 선택 UI 데이터 테이블을 통째로 반환합니다.
if isvalid(self.characterSelectUIData) and next(self.characterSelectUIData) ~= nil
then
    return self.characterSelectUIData
else
-- 반환할 값이 없을 경우, 에러 로그를 출력합니다.
    log("CharacterSelectUIData 데이터 테이블이 존재하지 않거나 캐싱이 정상적으로 이뤄지지 않았습니다!")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetMonsterSpawnDataTableByStageID(number StageID)
-- 전달받은 StageID가 1이상인 정상값이 아닌 경우, 로그를 남기고 불러오기를 종료합니다.
if StageID <= 0 then
    log("DataTableManager에서 SpawnData호출을 위해 전달된 값이 1 이상이 아닙니다!")
    return

```

```

end
-- 정상적으로 몬스터 스폰 데이터를 전달 가능한 경우, 로그와 함께 데이터를 반환합니다.
if self.monsterSpawnData[StageID] ~= nil then
    log("받은 스폰데이터 값: "..self.monsterSpawnData[StageID][1].MonsterID)
    return self.monsterSpawnData[StageID]
end
-- 스폰 데이터를 반환할 수 없는 경우 에러 로그를 출력합니다.
log(StageID.."인 StageID값의 데이터를 MonsterSpawnData 데이터 테이블에서 찾을 수 없습니다!")
return nil
end

@ExecSpace("ClientOnly")
method table GetMonsterDefaultDataByMonsterID(string MonsterID)
-- monsterDefaultData가 캐싱되어 있는지 확인합니다.
if invalid(self.monsterDefaultData) or next(self.monsterDefaultData) ~= nil then
    -- MonsterID값을 가지는 데이터를 반환할 수 있다면 데이터를 반환합니다.
    if self.monsterDefaultData[MonsterID] then
        return self.monsterDefaultData[MonsterID]
    end
-- 데이터를 반환할 수 없는 경우 에러 로그를 출력합니다.
else
    log("몬스터 데이터 파싱이 없는 상태입니다!")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetSkillDataBySkillID(string SkillID)
-- skillData가 캐싱되어 있지 않을 경우, 캐싱을 시도합니다.
if not invalid(self.skillData) or not next(self.skillData) then
    self:InitSkillData()
end
-- skillData를 사용할 수 있고, SkillID를 Key로 가지는 값이 있을 경우 값을 반환합니다.
if invalid(self.skillData[SkillID]) and next(self.skillData[SkillID]) then
    return self.skillData[SkillID]
end
-- 데이터를 반환할 수 없다면 에러 로그를 출력합니다.
log(SkillID.."이름의 SkillData를 찾을 수 없습니다!")
return nil
end

@ExecSpace("ClientOnly")
method table GetItemDataByItemID(string ItemID)
-- itemData가 캐싱되어 있지 않을 경우, 캐싱을 시도합니다.
if not invalid(self.itemData) or not next(self.itemData) then
    self:InitItemData()
end
-- itemData의 ItemID를 Key로 가지는 Value가 있을 경우, 값을 반환합니다.
if invalid(self.itemData[ItemID]) and next(self.itemData[ItemID]) then
    return self.itemData[ItemID]
end
-- ItemID의 값을 반환할 수 없다면 에러 로그를 출력합니다.
log(ItemID.."이름의 ItemData를 찾을 수 없습니다!")

```

```

return nil
end

@ExecSpace("ClientOnly")
method table GetItemDropData()
-- itemDropData가 캐싱되어 있지 않을 경우, 캐싱을 시도합니다.
if not isvalid(self.itemDropData) or not next(self.itemDropData) then
    self:InitItemDropData()
end
-- itemDropData를 반환할 수 있다면 itemDropData를 전부 반환합니다.
if isvalid(self.itemDropData) and next(self.itemDropData) then
    return self.itemDropData
end
-- 반환이 불가능할 경우 에러 로그와 함께 nil을 반환합니다.
log("ItemDropData 값이 DataTableLogic에 등록되어 있지 않습니다!")
return nil
end

end

```

UIManageLogic 코드 작성하기

```

@Logic
script UIManageLogic extends Logic

property TitleUIController titleUIController = nil

property SelectCharacterUIController selectCharacterUIController = nil

property ResultUIController resultUIController = nil

property DefaultUIController defaultUIController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 게임이 시작될 경우 TitleUIController를 찾아 등록합니다.
self:InitTitleUIController()
-- 게임이 시작될 경우 SelectCharacterUIController를 찾아 등록합니다.
self:InitSelectCharacterUIController()
-- 게임이 시작될 경우 ResultUIController를 찾아 등록합니다.
self:InitResultUIController()
-- 게임이 시작될 경우 DefaultUIController를 찾아 등록합니다.
self:InitDefaultUIController()
end

@ExecSpace("ClientOnly")
method void InitTitleUIController()
--TitleUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
if not isvalid(self.titleUIController) then

```

```

    local titleUIEntity = _EntityService:GetEntityByPath("/ui/TitleUI")
    --titleUIGroup을 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등록합니다.
    if invalid(titleUIEntity) then
        local component = titleUIEntity.TitleUIController
        -- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
        if invalid(component) then
            self.titleUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void InitSelectCharacterUIController()
--SelectCharacterUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
if not invalid(self.selectCharacterUIController) then
    local selectCharacterUIEntity =
_EntityService:GetEntityByPath("/ui/SelectCharacterUI")
    --SelectCharacterUIController을 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등
    록합니다.
    if invalid(selectCharacterUIEntity) then
        local component = selectCharacterUIEntity.SelectCharacterUIController
        -- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
        if invalid(component) then
            self.selectCharacterUIController = component
        end
    end
end
end
end

@ExecSpace("ClientOnly")
method void ShowTitle()
-- 타이틀 화면을 보여줍니다.
self.titleUIController:Show()
-- 이외의 UI들을 모두 가림처리합니다.
self.selectCharacterUIController:Hide()
self.resultUIController:Hide()
self.defaultUIController:Hide()
end

@ExecSpace("ClientOnly")
method void CallSelectCharacterUI()
-- 캐릭터 선택창을 보여줍니다.
self.selectCharacterUIController:Show()
-- 캐릭터 선택창 내 캐릭터 리스트를 초기화 시도합니다.
self.selectCharacterUIController:InitCharacterSelectFrame()
end

@ExecSpace("ClientOnly")
method void ShowInGame()
-- 게임 화면 UI인 defaultUI를 초기화합니다.
self.defaultUIController:Show()
-- 자동 공격 UI를 초기화하도록 요청합니다.
self.defaultUIController:UpdateAutoUI()

```

```

-- 이외의 UI들을 가림 처리합니다.
self.titleUIController:Hide()
self.selectCharacterUIController:Hide()
self.resultUIController:Hide()
end

@ExecSpace("ClientOnly")
method void InitResultUIController()
--ResultUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
if not isvalid(self.resultUIController) then
    local resultUIEntity = _EntityService:GetEntityByPath("/ui/ResultUI")
    --ResultUIController를 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등록합니다.
    if isvalid(resultUIEntity) then
        local component = resultUIEntity.ResultUIController
        -- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
        if isvalid(component) then
            self.resultUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void ShowResultUI(boolean IsClear)
-- 결과 UI를 출력합니다.
self.resultUIController:Show(IsClear)
if IsClear then
    --축하 효과음 출력
    _SoundService:PlaySound("5d57b617cfc242cca347925e209c8e05", 1.0)
else
    --실패 효과음 출력
    _SoundService:PlaySound("52dccfb8889b4181b9181780dc7e9040", 1.0)
end
end

@ExecSpace("ClientOnly")
method void InitDefaultUIController()
--defaultUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
if not isvalid(self.defaultUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/DefaultUI")
    --DefaultUIGroup을 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등록합니다.
    if isvalid(titleUIEntity) then
        local component = titleUIEntity.DefaultUIController
        -- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
        if isvalid(component) then
            self.defaultUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void UpdateLifeCountUI(number PlayerLife)
-- DefaultUI의 체력 UI를 PlayerLife 수에 맞춰 업데이트합니다.

```

```

self.defaultUIController:UpdateLifeUI(PlayerLife)
end

@ExecSpace("ClientOnly")
method void UpdateSkillCountUI(number LeftSkillCount)
-- LeftSkillCount의 수에 맞춰 DefaultUI의 스킬 UI를 업데이트합니다.
self.defaultUIController:UpdateSkillUI(LeftSkillCount)
end

end

```

Logic의 장점

- 어디에서나 접근이 편하다는 장점이 존재합니다.
- 컴포넌트와 같이 특정 엔티티를 찾고, 찾은 엔티티가 해당 컴포넌트를 보유하고 있는지 확인할 필요가 없습니다.
- Logic을 만들어 둔다면 해당 Logic을 확실하게 신뢰하고 사용할 수 있습니다.
- 또한 기능을 Logic이 담당하기 때문에 책임의 문제가 나올 경우, 해당 책임을 가진 Logic만 수정하여 문제를 해결할 수 있습니다.

Logic 사용시 주의사항

- Logic 하나가 모든 기능을 담당하는 갯클래스화가 일어날 수 있습니다.
 - EX: 데미지 계산을 담당하는 Logic이 데미지 계산, 데미지 값 처리, 사운드 출력, 히트 이펙트 출력 등을 담당
- 갯클래스화가 일어날 경우, 해당 Logic이 자신에게 필요한 정보 외에도 더 많은 정보를 알고 있어야 합니다.
- 이는 객체지향 설계의 이론을 벗어나는 방식이며, 코드의 줄을 크게 늘리는 원인이 됩니다.
- 늘어난 코드의 줄은 가독성을 저해하며, 디버깅 진행 시 문제의 원인을 빠르게 찾을 수 없다는 문제점이 존재합니다.

최소 구현 기준

- 앞으로 사용할 DataSet을 관리하고, 사용하기 위한 Logic을 제작합니다.
 - 샘플 월드 기준 DataTableLogic을 제작하여 DataSet을 관리합니다.
- 게임의 진행에 따른 처리를 담당할 수 있는 Logic을 제작합니다.
 - 샘플 월드 기준 GameLogic을 제작하여 게임의 상태에 따른 처리를 구현합니다.
- 모든 UI를 관리할 수 있는 Logic을 제작합니다.
 - 샘플 월드 기준 UIManageLogic을 제작하여 다수의 UI들을 관리할 수 있도록 기능을 구현합니다.

응용 및 확장 방향

- 데미지 값을 전달받으면 공격 계산식에 따라 최종 데미지를 계산해주는 CalcDamageLogic을 제작하여 데미지 계산 기능을 관리합니다.
- Logic을 활용하여 **변경 예정이 없는** 고정값을 관리하기 위한 ConfigLogic 제작하여 사용합니다.
 - EX: 목숨이 5개를 넘기지 않는 게임의 경우, 최대 체력을 고정하기 위한 MaxHP를 등록하고 사용할 수 있는 Logic을 제작하고 사용합니다.

[💡 기초 학습]PlayerCharacter 제작하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- PlayerCharacter 제작하기: 00:49:40

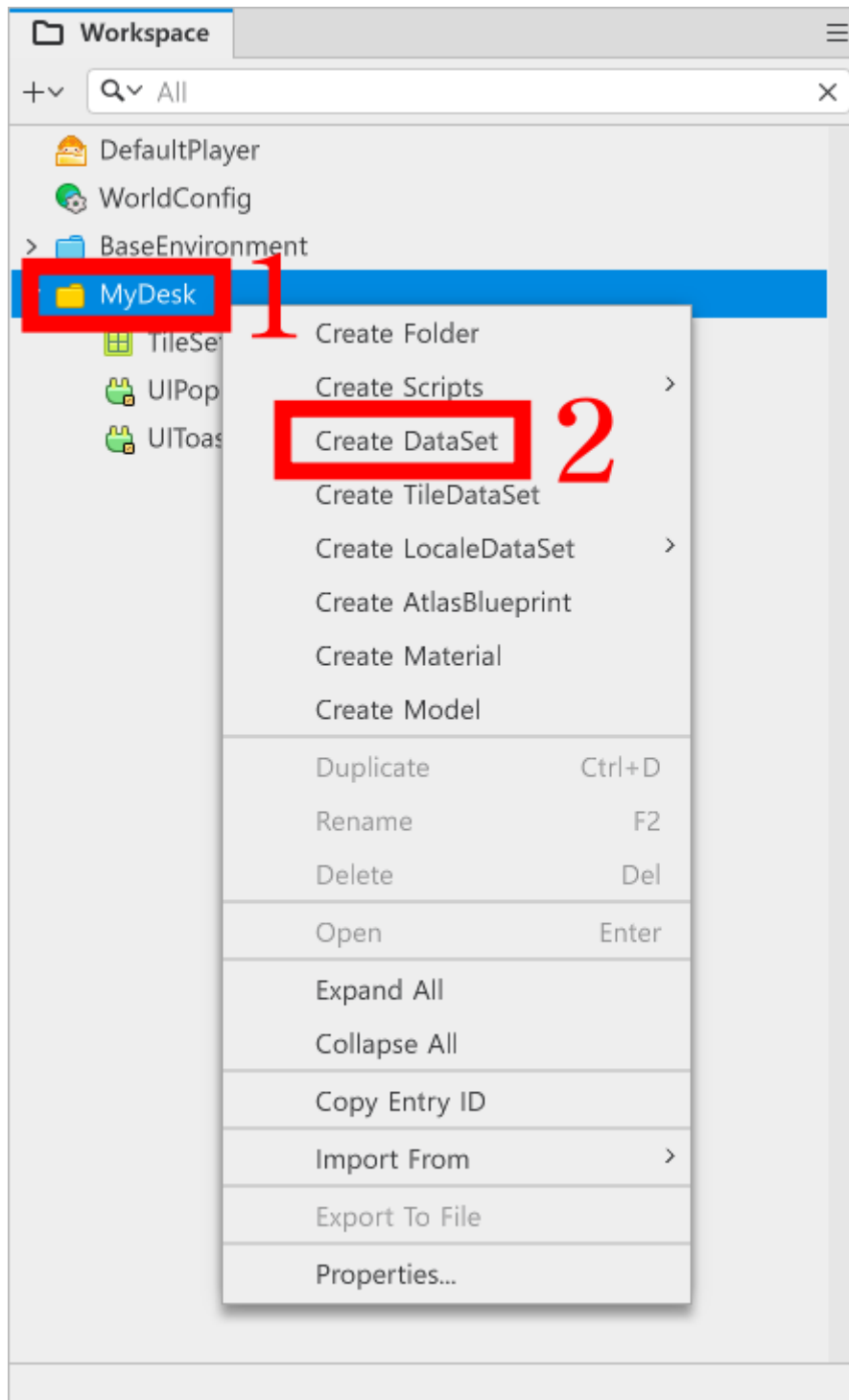
학습 목표

- 플레이어 캐릭터를 구성하기 위한 DataSet을 학습하고, DataSet을 수정할 수 있습니다.
- DataTableLogic을 활용하여 Manager를 직접 구현하고, 이를 사용할 수 있습니다.
- Component를 직접 제작하고 사용할 수 있습니다.
- DataSet을 활용하여 Component에 값을 초기화할 수 있습니다.

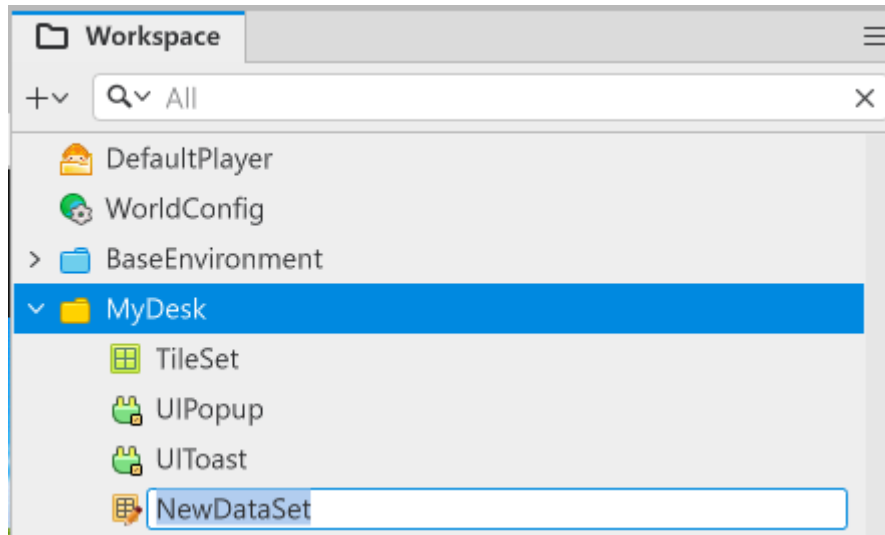
해당 영상 시청 후 수행해 볼 내용

- DataTableLogic을 구현하여 DataSet을 관리하는 Logic을 제작합니다.
- 사용자 정의 컴포넌트인 PlayerState 컴포넌트를 직접 제작하고, 이를 플레이어 캐릭터에 추가할 수 있습니다.
- TriggerComponent를 활용하여 플레이어에게 충돌 범위를 설정할 수 있습니다.

DataSet 제작하기



- Workspace에서 MyDesk를 우클릭합니다.
- Create DataSet을 눌러 새로운 DataSet을 제작합니다.



- 생성된 DataSet의 이름을 변경합니다.
- 샘플 월드에서는 **CharacterDefaultData**를 사용하고 있습니다.
- DataSet 생성을 마쳤다면, 생성한 DataSet을 더블클릭하여 편집 창으로 이동합니다.

Scene Play (Preview) Resource Storage CharacterDefaultData X							
텍스트 입력							
	1	2	3	4	5	6	7
ID	CharacterID	StartHP	StartSkillCount	StartMoveSpeed	DefaultMissile	DefaultSkill	AttackInterval
1	Pirate	3	2	5.0	Missile_Bullet	Skill_CannonBal	0.5

- 각 행, 열을 추가해야 할 경우 +버튼을 눌러 해당하는 행, 열을 추가할 수 있습니다.
- 1번째 행에는 사용하려는 데이터의 이름을 입력합니다.
- 2번째 행부터는 데이터의 이름에 충족하는 값들을 입력합니다.

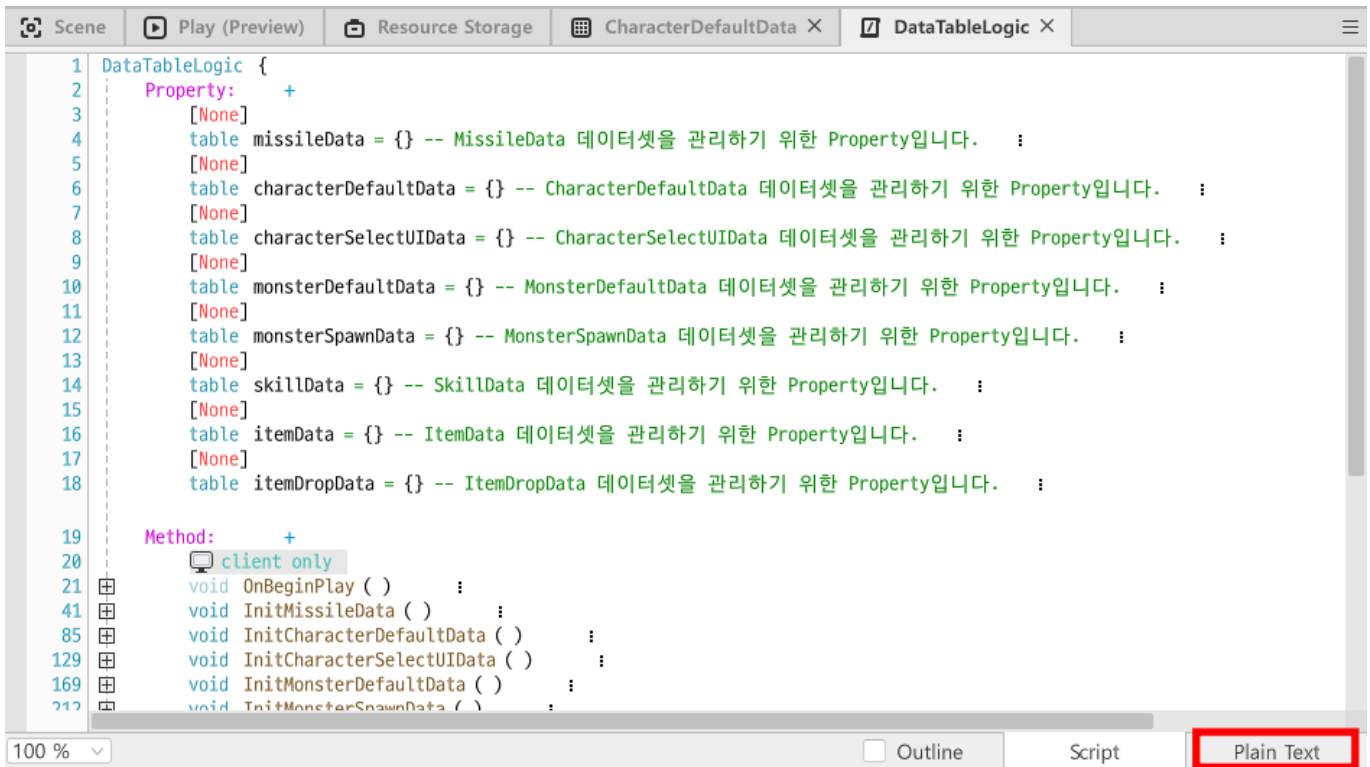
샘플 월드의 DataSet 구성

이름	설명
CharacterID	캐릭터를 구분하기 위한 ID 값입니다. 해당 값에 따라 캐릭터를 구성하는 정보들을 불러올 수 있습니다.
StartHP	게임이 시작될 때, 캐릭터의 시작 체력을 설정하기 위한 값입니다.
StartSkillCount	게임이 시작될 때, 캐릭터가 사용할 수 있는 필살기의 수를 설정하기 위한 값입니다.
StartMoveSpeed	게임이 시작될 때, 캐릭터의 이동 속도를 설정하기 위한 값입니다.
DefaultMissile	해당 캐릭터의 기본 투사체를 결정하기 위한 값입니다.
DefaultSkill	캐릭터의 필살기를 지정하기 위한 값입니다.

이름	설명
AttackInterval	캐릭터의 공격 딜레이를 결정하기 위한 값입니다. 값이 작을수록 연사 속도가 빨라집니다.

DataTableLogic에 DataSet 등록하기

- DataTableLogic에 다음과 같이 코드를 작성합니다.
- 코드의 예시는 PlainText를 기준으로 작성되어 있습니다.



- PlainText는 코드 에디터 창에서 우측 하단의 UI 버튼을 통해 변경할 수 있습니다.

```
property table characterDefaultData = {} -- CharacterDefaultData 데이터셋을 관리하기 위한 Property입니다.
```

```
@ExecSpace("ClientOnly")
```

```
method void OnBeginPlay()
```

```
-- CharacterDefaultData 데이터셋을 초기화합니다.
```

```
self:InitCharacterDefaultData()
```

```
end
```

```
method void InitCharacterDefaultData()
```

```
-- 플레이어 캐릭터 파싱이 완료되어 있는지 체크하기 위한 항목
```

```
if isValid(self.characterDefaultData) == false or next(self.characterDefaultData) == nil then
```

```
    local getTableData = _DataService:GetTable("CharacterDefaultData")
```

```
--데이터 테이블 가져오기에 실패했을 경우 경고 문구 출력
```

```
if getTableData == nil then
```

```
    log("CharacterDefaultData 데이터 테이블을 가져오는 데 실패했습니다!")
```

```
    return
```

```

end
--파싱 시작
local rowCount = getTableData:GetRowCount() -- 전체 행 개수를 계산합니다.

for i = 1, rowCount do
    -- Key가 될 CharacterID를 우선 데이터 테이블 Row에서 하나를 대입합니다.
    local characterID = tostring(getTableData:GetCell(i, "CharacterID"))

    -- 유효한 ID인지 체크합니다.
    if characterID ~= nil and characterID ~= "" then

        -- 해당 행의 모든 정보를 테이블로 묶습니다.
        local rowData =
        {
            CharacterID = characterID,
            StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
            StartSkillCount = tonumber(getTableData:GetCell(i,
"StartSkillCount")),
            StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
            DefaultMissile = tostring(getTableData:GetCell(i,
"DefaultMissile")),
            DefaultSkill = tostring(getTableData:GetCell(i, "DefaultSkill")),
            AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval"))
        }

        -- 캐시 테이블에 저장 (Key: MissileID, Value: 데이터)
        self.characterDefaultData[characterID] = rowData
    end
end
-- 데이터 테이블 캐싱이 완료되었음을 알리는 Log를 출력합니다.
log("CharacterDefaultData 데이터 테이블 캐싱 완료!")
return
end
-- 데이터 테이블 캐싱을 하지 않았으므로 이를 안내하는 로그를 출력합니다.
log("CharacterDefaultData 데이터 테이블이 이미 캐싱 되어 있거나, CharacterDefaultData
데이터 테이블을 찾지 못했습니다.")
end

method table GetCharacterDefaultDataByCharacterID(string CharacterID)
-- characterDefaultData가 정상적으로 캐싱 되어 있지 않을 경우, 캐싱을 시도합니다.
if self.characterDefaultData == nil then
    self:InitCharacterDefaultData()
end
-- 현재 CharacterDefaultData가 정상적으로 등록된 상태인지 점검합니다.
if invalid(self.characterDefaultData) and next(self.characterDefaultData) ~= nil
then
    -- CharacterID값을 가지는 데이터가 있을 때, 해당 데이터를 반환합니다.
    if self.characterDefaultData[CharacterID] then
        return self.characterDefaultData[CharacterID]
    end
    -- 해당하는 Key의 데이터가 없으면 에러 로그를 출력합니다.
else

```

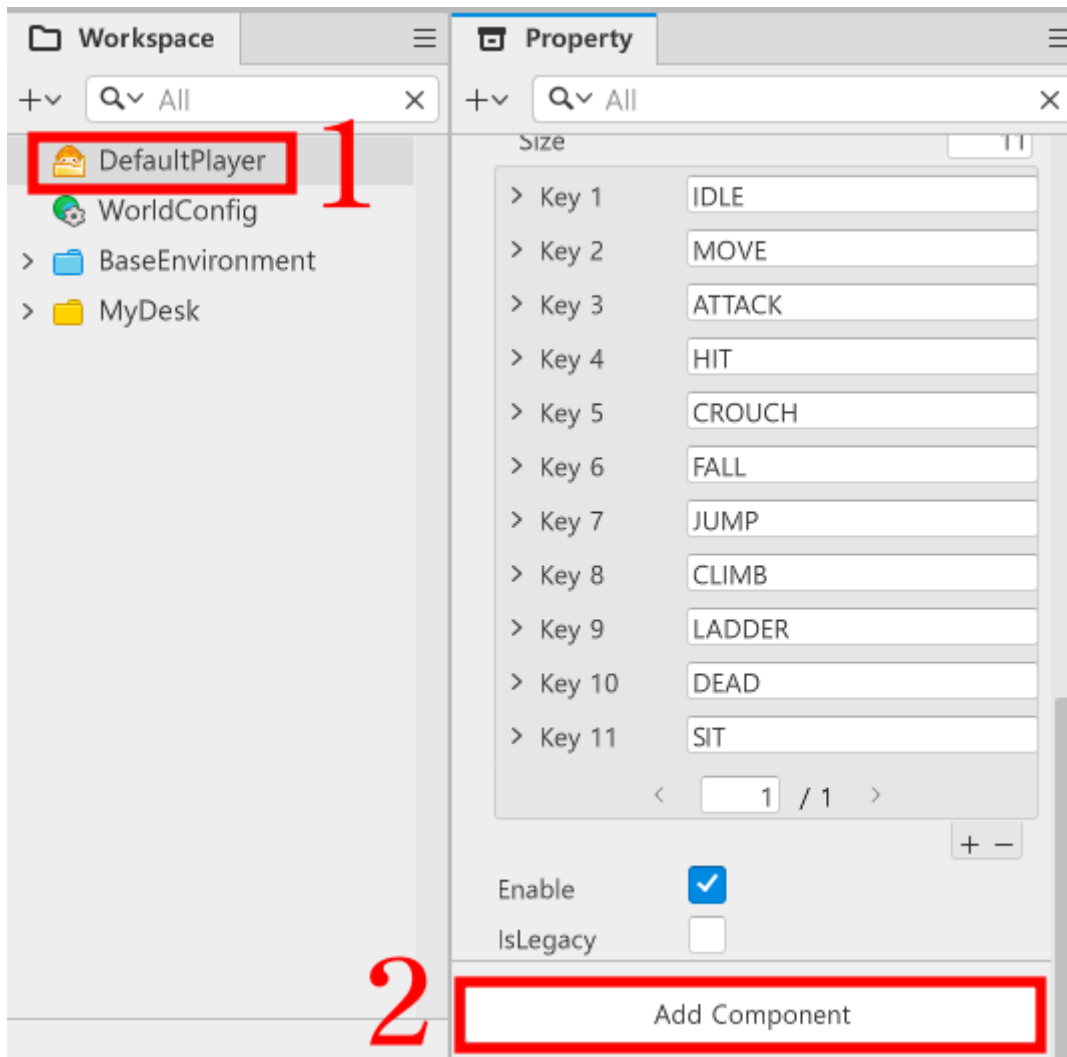
```

log("PlayerDefaultData 데이터가 없거나 존재하지 않는 "..CharacterID.."값입니다!")
return nil
end
end

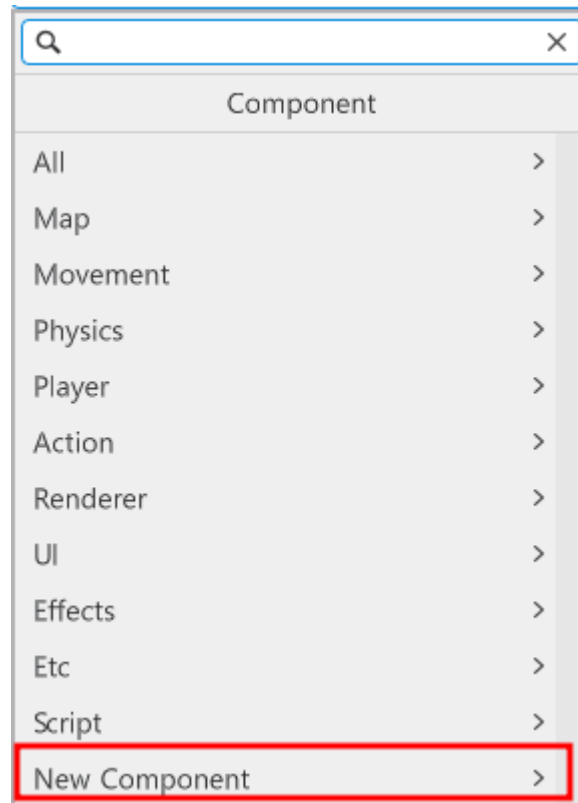
```

PlayerState Component 제작하기

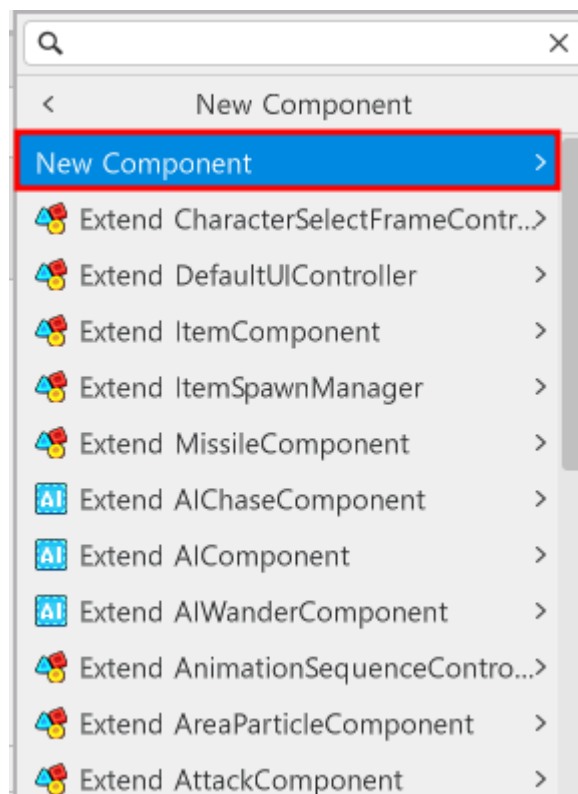
- DataSet에 플레이어 정보를 저장했다면, 저장된 정보를 사용해야 합니다.



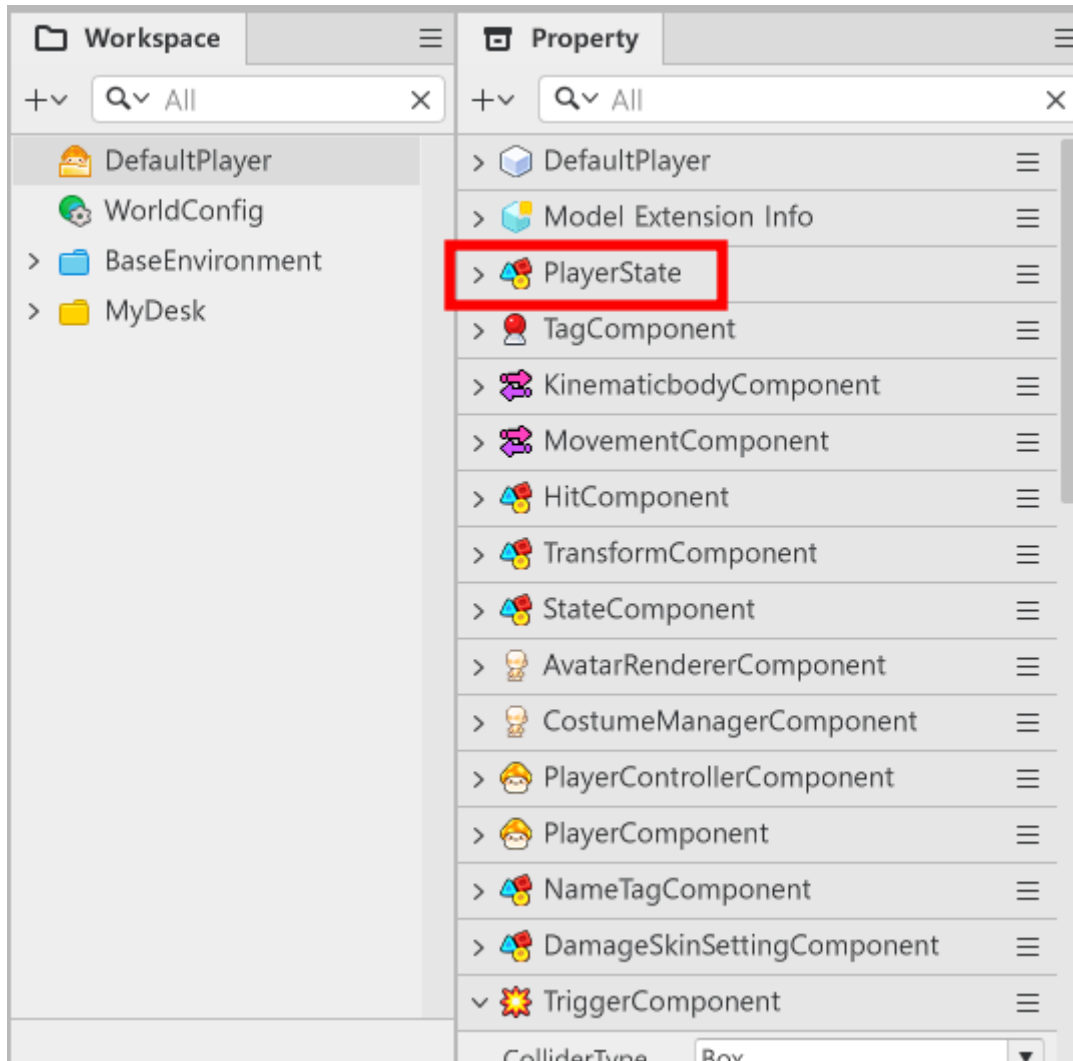
- Workspace에서 DefaultPlayer를 클릭합니다.
- Property 창을 끝까지 내린 뒤, Add Component를 클릭합니다.



- 추가된 메뉴에서 NewComponent를 클릭합니다.



- New Component를 클릭한 뒤, 생성하려는 Component의 이름을 입력하고, Create and Add를 클릭합니다.
- 샘플 월드에서는 해당 Component의 이름을 **PlayerState**로 설정했습니다.



- 정상적으로 등록되었다면, DefaultPlayer를 클릭했을 때, Property 창에서 입력한 Component가 등록된 모습을 확인할 수 있습니다.

PlayerState에 값 초기화하기

- PlayerState에 다음과 같이 코드를 입력합니다.
- 해당 코드는 Plain Text를 기준으로 작성되어 있습니다.

```
@Component
script PlayerState extends Component

property number currentHP = 0 -- 플레이어의 현재 체력을 관리하기 위한 Property입니다.

property string currentMissile = "" -- 플레이어의 현재 투사체의 ID를 저장하기 위한 Property입니다.

property string characterID = "" -- 현재 플레이 중인 캐릭터의 ID를 저장하기 위한 Property입니다.

property number attackInterval = 0.5 -- 플레이어의 공격 딜레이를 관리하기 위한 Property입니다.
```



```

property number attackTimer = 0 -- 플레이어의 공격 가능 여부를 계산하기 위한 Property
입니다.

property any projectilePoolManager = nil

property Vector3 firePos = Vector3(0.25,0.25,0) -- 플레이어의 공격 발사 위치를 조정
하기 위한 Property입니다.

property boolean isInvincibility = false -- 플레이어의 무적 상태를 구분하기 위한
Property입니다.

property number invincibilityDelta = 0 -- 무적 시간 계산을 위한 Property입니다.

property number invincibilityTime = 1.0 -- 무적 지속시간의 기준 Property입니다. 해당
값을 수정하여 무적 시간을 변경할 수 있습니다.

property boolean isAlive = false -- 플레이어의 생존 여부를 구분하는 Property입니다.

property string skillID = "" -- 플레이어의 사용 스킬 ID를 저장하기 위한 Property입니
다.

property number canUseSkillCount = 0 -- 현재 플레이어가 사용할 수 있는 스킬의 횟수를
관리하기 위한 Property입니다.

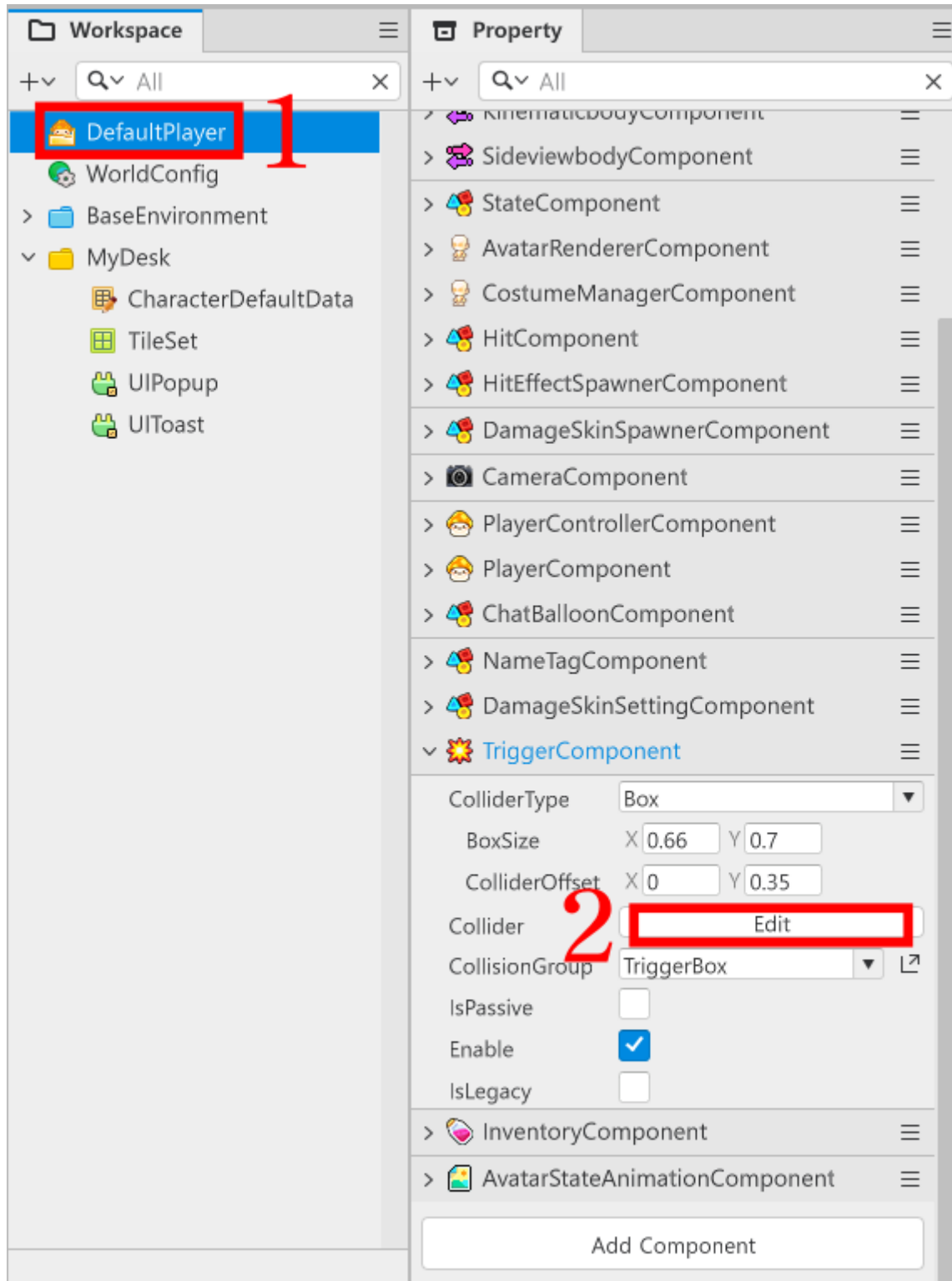
property number moveSpeed = 0 -- 플레이어의 이동속도를 등록하기 위한 Property입니다.

@ExecSpace("ClientOnly")
method void InitPlayerData(table PlayerDefaultData)
-- 전달받은 PlayerData가 정상적인 데이터가 아닐 경우, 에러 로그를 출력하고 코드 실행을
중단합니다.
if PlayerDefaultData == nil or next(PlayerDefaultData) == nil then
    log("PlayerDefaultData가 정상적이지 않습니다!")
    return
end
-- 플레이어의 현재 체력을 초기화합니다.
self.currentHP = PlayerDefaultData.StartHP
-- 플레이어의 사용 가능한 스킬 ID를 초기화합니다.
self.canUseSkillCount = PlayerDefaultData.StartSkillCount
-- 플레이어의 이동속도를 초기화합니다.
self.moveSpeed = PlayerDefaultData.StartMoveSpeed
-- 초기화한 플레이어 이동속도를 정상적으로 반영하기 위해 MovementComponent에 전달합니
다.
self.Entity.MovementComponent.InputSpeed = self.moveSpeed
-- 플레이어의 기본 투사체 ID를 초기화합니다.
self.currentMissile = PlayerDefaultData.DefaultMissile
-- 플레이어의 캐릭터 ID 값을 초기화합니다.
self.characterID = PlayerDefaultData.CharacterID
-- 플레이어의 공격 딜레이를 초기화합니다.
self.attackInterval = PlayerDefaultData.AttackInterval
-- 플레이어의 스킬 ID를 초기화합니다.
self.skillID = PlayerDefaultData.DefaultSkill
-- 플레이어의 시작 위치를 초기화합니다.
self.Entity.TransformComponent.Position = Vector3.zero

```

```
-- 플레이어 Entity의 사용 여부를 true로 변경합니다.  
self.Entity.Enable = true  
-- 플레이어 캐릭터의 Visible 여부를 true로 변경합니다.  
self.Entity.Visible = true  
end  
  
@ExecSpace("ClientOnly")  
method void OnBeginPlay()  
    local getData = _DataTableLogic:GetCharacterDefaultDataByCharacterID("Pirate")  
    self:InitPlayerData(getData)  
    self.isAlive = true  
end
```

TriggerComponent 수정하기



- Workspace에서 DefaultPlayer를 클릭합니다.
- Porperty 창에서 TriggerComponent를 찾은 뒤, Edit를 클릭합니다.



- 초록색 영역을 드래그 & 드랍하여 사이즈를 캐릭터의 이미지에 맞춰 조정합니다.

TriggerComponent란?

- TriggerComponent는 엔티티 간의 충돌을 감지하기 위한 Component입니다.
- 초록색 영역이 충돌 판정 영역입니다.
- 해당 영역이 교차하는 순간, 충돌했다고 인지합니다.
- 따라서 이미지의 리소스보다 크거나 작은 판정을 가질 경우, 플레이어는 불합리한 판정이라고 생각할 수 있습니다.
- 하지만 게임의 방식에 따라서는 고의적으로 판정을 늘려 관대한 처리를 유도하기도 합니다.
- 제작하고자 하는 게임의 성격에 따라 완벽하게 판정을 맞추지, 여유를 줄 것인지 고민하고 설계해야 합니다.

최소 구현 기준

- DataSet을 제작할 수 있습니다.
- 제작된 DataTableLogic을 통해 DataSet의 원하는 값을 불러올 수 있습니다.
- 불러온 DataSet의 값을 Component에 전달하고, 대입할 수 있습니다.
- TriggerComponent를 편집하여 충돌 범위를 설정할 수 있습니다.

응용 및 확장 방향

- 앞으로 제작될 플레이어와 관련된 기능을 Component단위로 분리할 수 있습니다.
 - EX: 공격을 담당한다면 PlayerAttackComponent를 제작하여 공격과 관련된 기능을 해당 Component에 구현합니다.
- 게임 제작에 있어 필요한 값들을 DataSet에 추가하고, 추가한 값을 불러올 수 있도록 DataTableLogic을 수정합니다.
 - 해당 방법을 학습하고 사용함으로 필요에 따라 DataSet을 확장 또는 추가할 수 있습니다.

[💡 기초 학습]Monster 제작하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- Monster 제작하기: 01:21:22

학습 목표

- 플레이어의 위협 요소인 적을 생성하여 게임에 긴장감을 조성할 수 있습니다.
- 적을 구성하기 위해 엔티티를 생성하고, Component를 추가하여 적을 구성할 수 있습니다.
- 메이플스토리 월드가 제공하는 Component를 활용하여, 적이 플레이어를 추적할 수 있도록 기능을 구현합니다.

해당 영상 시청 후 수행해 볼 내용

- 빈 엔티티를 생성한 뒤, 필요한 Component들을 추가하여 Model을 제작합니다.
- 필요에 따라 다양한 외형의 Monster를 제작합니다.
- 제작된 Monster의 값들을 DataSet에서 관리하여 서로 다른 능력치를 가지도록 제작합니다.

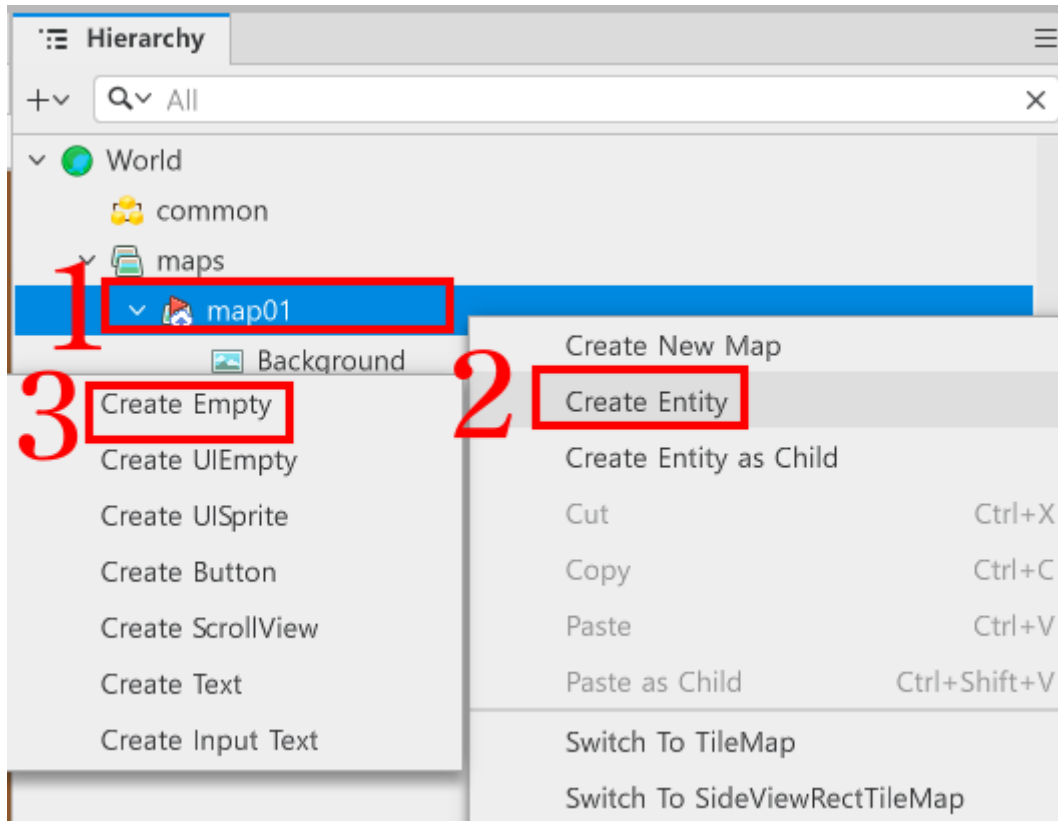
MonsterDataSet 구성하기

- Monster와 관련된 정보를 저장하기 위한 DataSet을 생성합니다.
- 샘플 월드에서는 해당 DataSet 이름을 **MonsterDefaultData**로 제작했습니다.
- 만약 DataSet 제작 방법을 모르신다면, 해당 문서의 PlayerCharacter 제작하기를 참고하여 제작합니다.

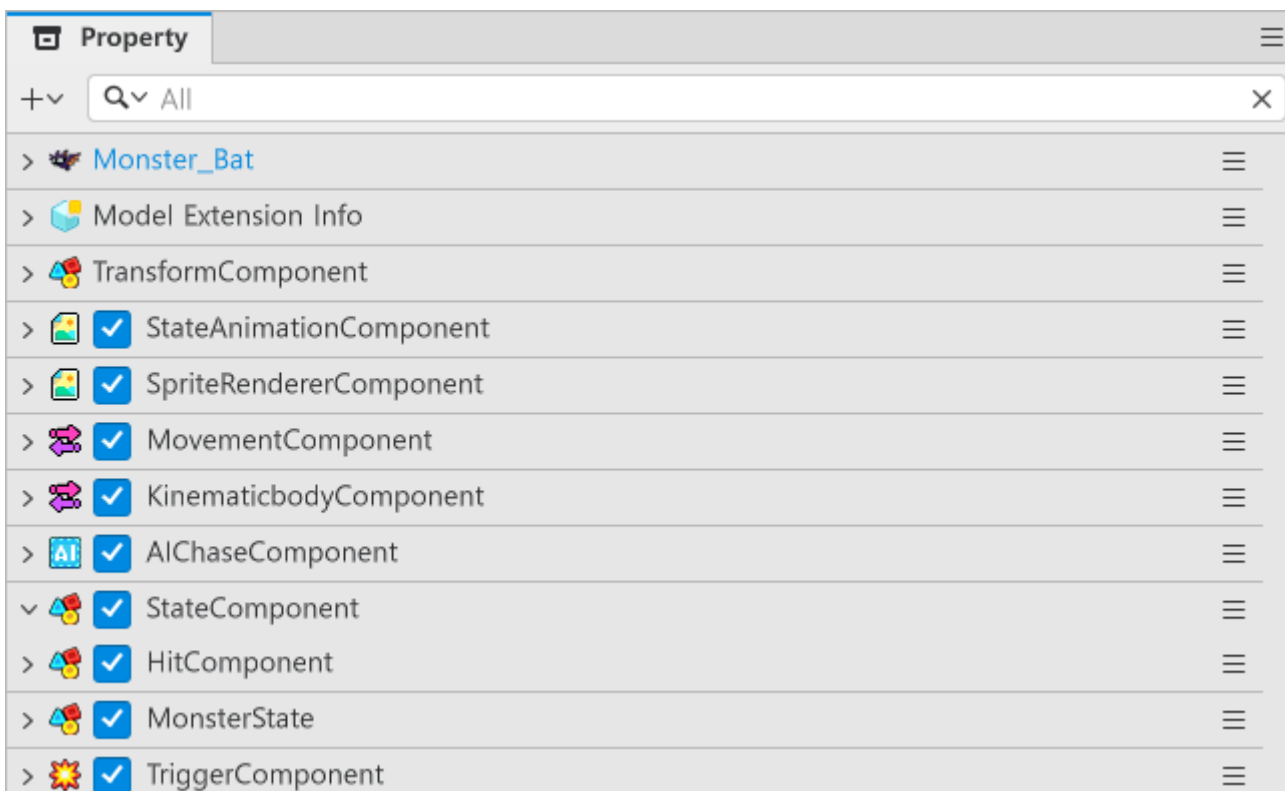
MonsterDefaultData 구성요소

이름	설명
MonsterID	Monster를 구분하기 위한 ID 값입니다. 해당 값을 기준으로 몬스터를 구분하고, 데이터를 불러올 수 있습니다.
StartHP	Monster가 게임에서 스폰되었을 때 시작 HP를 설정하기 위한 값입니다.
StartMoveSpeed	Monster가 스폰되었을 때, 기본 이동속도를 지정하기 위한 값입니다.
DefaultMissile	Monster의 기본 투사체를 지정하기 위한 값입니다.
AttackInterval	Monster의 공격 딜레이를 지정하기 위한 값입니다. 해당 값이 작아질수록 연사 속도가 빨라집니다.
ModelRUID	제작한 Monster Model의 RUID를 지정하기 위한 값입니다. 추후 MonsterSpawnManager에서 해당 값을 기준으로 Model을 복제합니다.

몬스터 엔티티 구성하기



- Hierarchy에서 map01을 우클릭합니다.
- Create Entity를 클릭합니다.
- Create Empty를 클릭하여 빈 엔티티를 map01의 자식 엔티티로 생성합니다.
- 생성한 엔티티의 이름을 변경할 수 있습니다.
- 샘플 월드에서는 예시를 위해 **Monster_Bat**이라는 이름으로 제작했습니다.

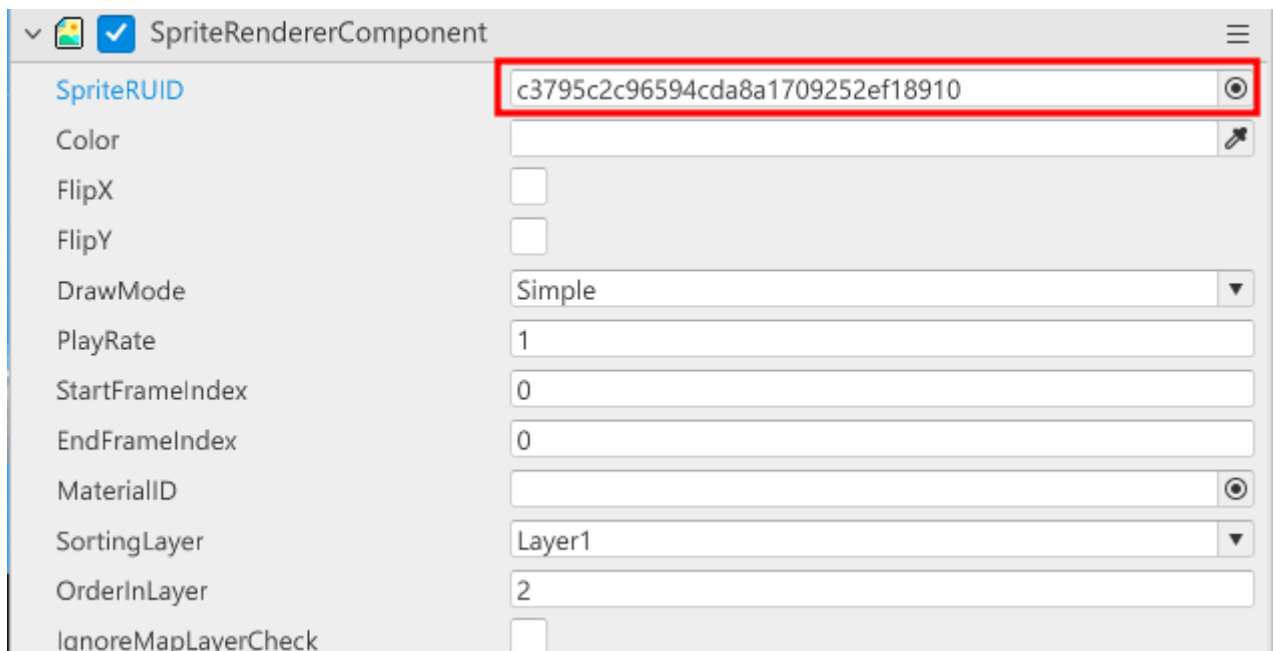


- 다음의 Component들을 제작한 엔티티에 추가합니다.
- **SpriteRendererComponent**: Monster의 이미지를 출력하기 위해 사용되는 Component입니다.

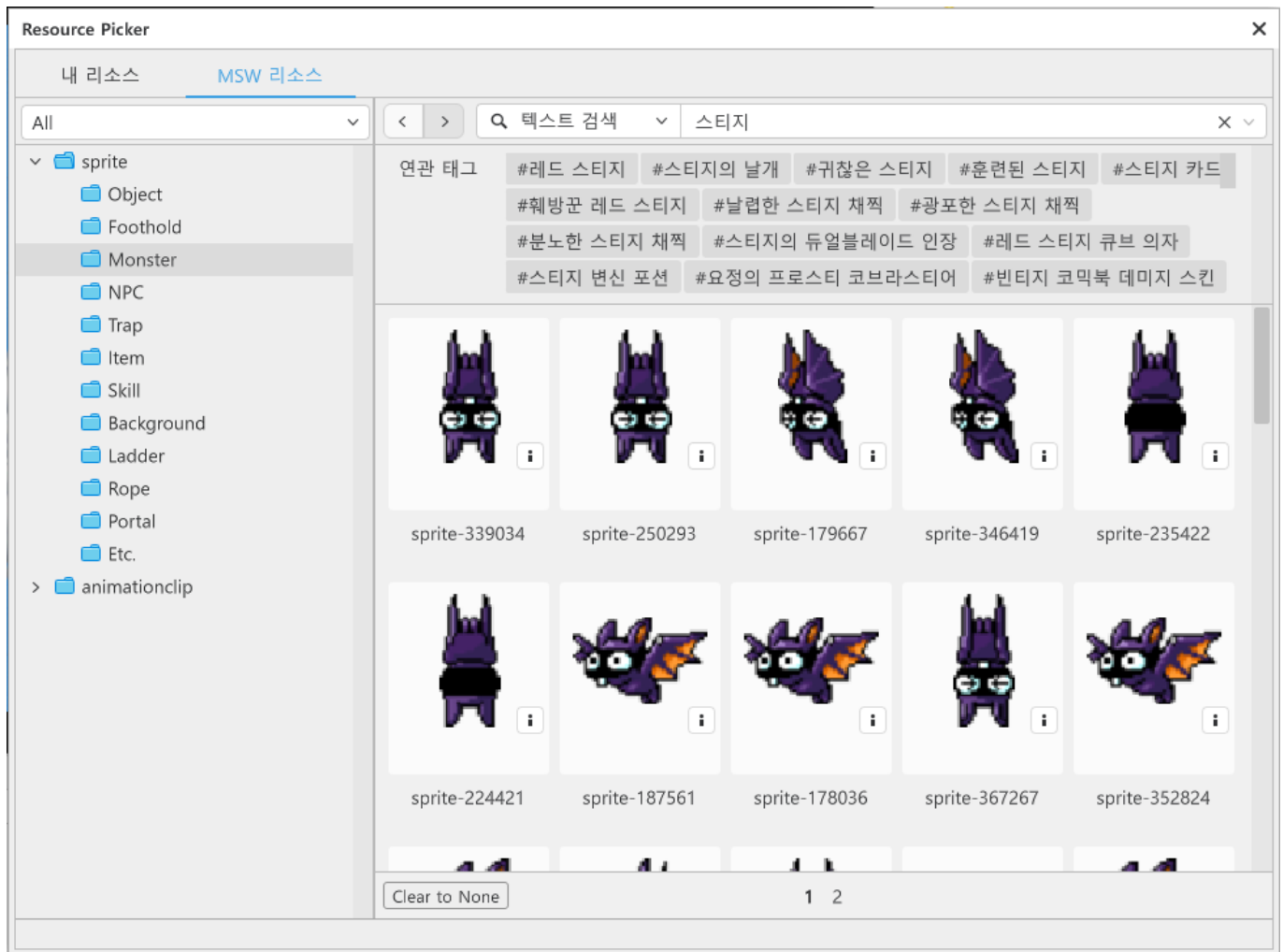
- **MovementComponent**: Monster가 이동하는 데 필요한 Component입니다. AIChaseComponent가 해당 Component를 활용하여 플레이어를 추적합니다.
- **KinematicBodyComponent**: RectTileMap에서 몬스터가 이동할 수 있도록 합니다. Monster는 이동 불가 영역이 없게 EnableTileCollision값을 false로 변경합니다.
- **AIChaseComponent**: 메이플스토리 월드가 제공하는 Component입니다. 해당 Component를 가진 엔티티는 플레이어를 추적할 수 있습니다.
- **HitComponent**: 투사체를 맞았을 때, 데미지 계산을 위한 Component입니다.
- **MonsterState**: 직접 제작해야 하는 Component입니다. 해당 Component는 PlayerState와 같이 Monster의 데이터를 담거나, 공격, 사망 처리와 같은 기능들을 담당하기 위한 Component입니다.
- **TriggerComponent**: Monster의 충돌 영역을 지정하기 위한 Component입니다. 해당 Component를 통해 몬스터의 충돌 영역을 지정하고, 투사체나 스킬의 충돌 여부를 검사할 수 있습니다.

몬스터 이미지 변경하기

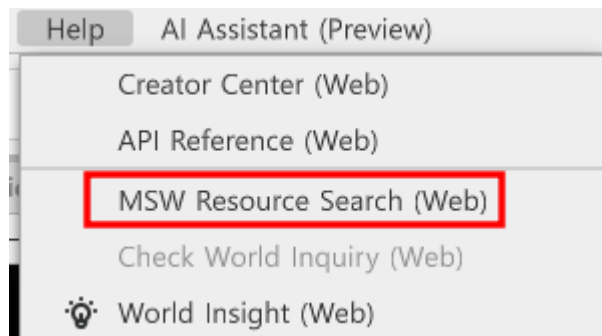
- 몬스터의 이미지를 변경하기 위해선 SpriteRendererComponent의 SpriteRUID를 변경하여 적용할 수 있습니다.



- SpriteRUID의 값을 변경하여 몬스터의 이미지를 변경할 수 있습니다.
- 우측의 Resource Picker를 활용하여 다른 이미지를 찾아서 사용할 수 있습니다.



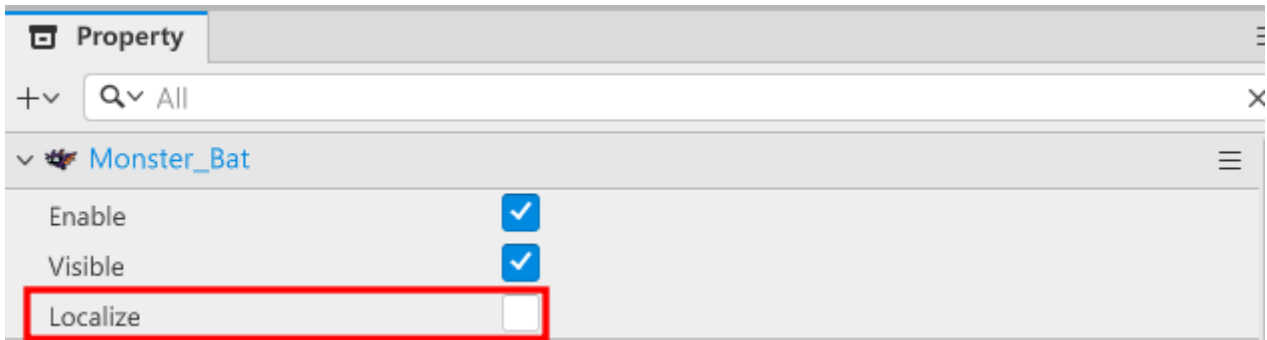
- Resource Picker에서 리소스를 검색하여 메이플스토리 월드가 제공하는 리소스를 사용할 수 있습니다.



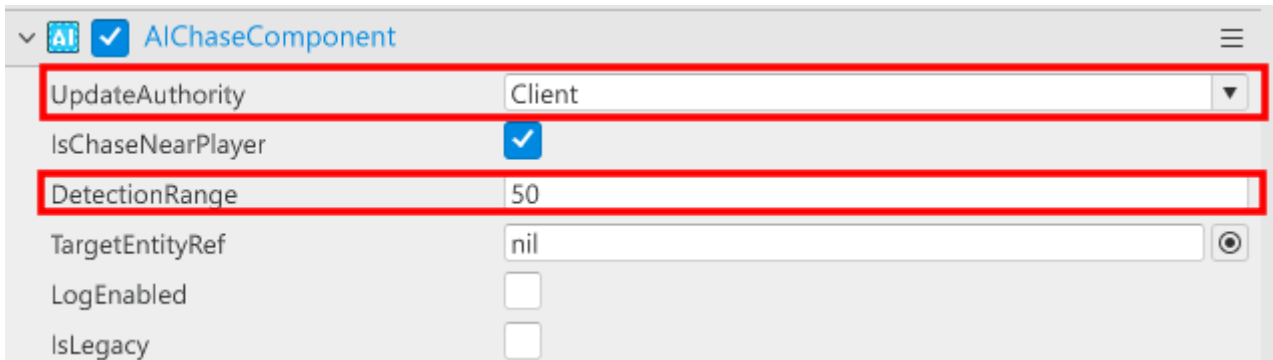
- 만약 Resource Picker를 활용하기 어렵다면, 에디터 상단의 Help를 통해 리소스를 찾을 수 있습니다.

몬스터가 플레이어를 추적할 수 있도록 구성하기

- 몬스터가 플레이어를 추적하게 만들기 위해서 다음의 요소들을 적용해야 합니다.



- Property에서 Localize를 활성화해야 합니다. - 샘플 월드는 클라이언트 구동을 기반으로 제작되었습니다. 따라서 Entity가 로컬에서 작동할 수 있도록 Localize를 체크해야 합니다.



- AIChaseComponent내의 UpdateAuthority를 Client로 변경해야 합니다.
 - 해당 값이 Server일 경우, 클라이언트 구동을 기반으로 한 샘플 월드에서는 정상적으로 플레이어를 추적하지 못할 수 있습니다.
- 필요에 따라 DetectionRange의 값을 늘려, 적이 플레이어를 인식하는 범위를 늘릴 수 있습니다.

MonsterState Component 구성하기

- Model에 추가한 MonsterState Component의 코드를 작성해야 합니다.
- 해당 코드를 통해 Monster는 MonsterDefaultData를 찾은 뒤 값을 받아올 수 있습니다.
- 다음과 같이 코드를 작성합니다.
- 해당 코드는 Plain Text를 기준으로 작성되어 있습니다.

```
@Component
script MonsterState extends Component

property number currentHP = 0 -- 몬스터의 현재 체력을 관리하기 위한 Property입니다.

property number attackInterval = 0 -- 몬스터의 공격 딜레이를 저장하기 위한 Property입니다.

property boolean isAlive = true -- 몬스터의 생존 여부를 체크하기 위한 Property입니다.

property string defaultMissile = "" -- 몬스터가 발사하는 투사체의 ID를 저장하기 위한 Property입니다.

property ProjectileManager projectileManager = nil

property Vector2 firePos = Vector2(0,0) -- 투사체가 발사되는 위치를 지정하기 위한
```

Property입니다.

property number deltaTimer = 0 -- 몬스터의 미사일 발사 딜레이를 계산하기 위한 Property입니다.

property number itemDropRate = 0 --몬스터 처치시 아이템이 드랍될 확률을 계산하기 위한 Property입니다.

property string mustDropItemValue = "" -- 몬스터가 반드시 아이템을 드랍해야 하는 경우, 아이템을 지정하기 위한 Property입니다.

@ExecSpace("Client")

method void InitData(table MonsterData, number ItemDropRate, string MustDropItem)
-- 전달받은 MonsterData가 nil인지 체크합니다.

if MonsterData == nil or next(MonsterData) == nil then
 log("전달받은 table MonsterData가 nil입니다!")
 return
end

-- 전달받은 MonsterData 내 필수 요소들이 정상적인 값을 가졌는지 검사합니다.

if MonsterData.StartHP == nil or MonsterData.DefaultMissile == nil or
MonsterData.DefaultMissile == "" then
 log("몬스터의 시작 체력, 미사일 타입이 설정되어 있지 않습니다!")
end

self:FindProjectileManager()

-- 필수 값들이 들어있다면 초기화 시작

-- 몬스터의 현재 체력을 시작할 때 최대 체력으로 초기화합니다.

self.currentHP = MonsterData.StartHP

-- 몬스터의 시작 투사체 ID를 초기화합니다.

self.defaultMissile = MonsterData.DefaultMissile

-- 몬스터의 공격 딜레이를 설정합니다. 만약 받아온 값이 nil일 경우, 1.0초로 초기화합니다.

self.attackInterval = MonsterData.AttackInterval or 1.0

-- 몬스터의 생존 상태를 true로 초기화합니다.

self.isAlive = true

-- 아이템의 드랍 확률을 초기화합니다.

self.itemDropRate = ItemDropRate

-- 필수로 드랍해야 하는 아이템이 있을 때, 해당 값을 초기화합니다.

self.mustDropItemValue = MustDropItem

-- 몬스터의 이동속도를 초기화합니다.

self.Entity.MovementComponent.InputSpeed = MonsterData.StartMoveSpeed

end

method void ShowMonster()

-- 몬스터의 Enable 상태를 true로 변경합니다.

self.Entity.Enable = true

end

@ExecSpace("ClientOnly")

method void OnUpdate(number delta)

-- 만약 게임의 상태가 Play 상태가 아닐 경우, 즉시 코드를 탈출합니다.

if not _GameLogic.IsStillPlay then
 self.Entity.MovementComponent.Enable = false

```

        return
    end
    -- 만약 현재 몬스터가 살아있고, 게임의 상태가 Play 상태일 경우 처리를 진행합니다.
    if self.isAlive and _GameLogic.IsStillPlay then
        -- 만약 몬스터의 MovementComponent가 비활성화 상태라면 활성화합니다.
        if self.Entity.MovementComponent.Enable == false then
            self.Entity.MovementComponent.Enable = true
        end
        -- deltaTime에 delta만큼 시간 흐름을 더합니다.
        self.deltaTime = self.deltaTime + delta
        -- 만약 공격 딜레이보다 deltaTime이 클 경우, 투사체를 발사하고 deltaTime을 0으로 초기화합니다.
        if self.deltaTime >= self.attackInterval then
            self:FireMissile()
            self.deltaTime = 0
        end
    end
end
end

@ExecSpace("Client")
method void FindProjectileManager()
    -- InGame 맵에 있는 MissileManager 엔티티를 찾습니다.
    local currentMap = self.Entity.CurrentMapName
    local poolEntity =
_EntityService:GetEntityByPath("/maps/"..currentMap.."/ProjectileManager")
    -- poolEntity를 정상적으로 찾았다면 missileManager를 poolEntity로 초기화합니다.
    if isvalid(poolEntity) then
        self.projectileManager = poolEntity.ProjectileManager
        self.isAlive = true
    else
        -- 만약 InGame에서 ProjectileManager를 찾지 못했다면, 에러 로그를 출력합니다.
        log("오류: InGame 맵에서 ProjectileManager를 찾을 수 없습니다.")
    end
end

@ExecSpace("Client")
method void FireMissile()
    -- projectileManager가 없으면 코드 실행을 중단합니다.
    if self.projectileManager == nil then return end

    -- projectileManager가 nil인지, 기본 투사체가 등록되어 있는지 체크합니다.
    if self.projectileManager == nil or self.defaultMissile == nil or
self.defaultMissile == "" then
        -- 만약 둘 중 하나라도 조건을 충족하지 못할 경우, 에러 로그를 남깁니다.
        log("플레이어 MissileData 초기화 실패!")
    end

    -- ProjectileManager에서 사용할 수 있는 defaultMissile 엔티티를 가져옵니다.
    local missileEntity = self.projectileManager:GetMissile(self.defaultMissile)

    -- projectileManager에서 전달받은 엔티티가 사용할 수 있을 때 코드를 실행합니다.
    if isvalid(missileEntity) then
        -- 전달받은 엔티티의 MissileComponent를 가져옵니다.
        local missileComp = missileEntity.MissileComponent

```

```

    if missileComp then
        -- 미사일의 발사 방향을 결정합니다.
        local fireDir = 1
        -- 기본 리소스가 왼쪽을 바라보고 있으므로 엔티티의 FlipX가 비활성화일 경우, 왼쪽을 향해 날아가도록 설정합니다.
        if self.Entity.SpriteRendererComponent.FlipX == false then
            fireDir = -1
        end
        -- 현재 몬스터의 위치를 가져옵니다.
        local monsterPos = self.Entity.TransformComponent.Position
        -- 투사체에 몬스터의 현재 위치를 전달합니다.
        local firePos = Vector3(monsterPos.x, monsterPos.y, 0)
        -- 투사체 발사를 명령합니다.
        missileComp:FireMissile(Vector2(fireDir, 0), false, firePos)
    end
end
end

@ExecSpace("Client")
method void SendCalcDropItem()
    -- 몬스터가 사망했을 경우, ItemHelper에 아이템 드롭 여부를 계산하도록 전달합니다.
    _ItemHelper:CalcDropItemData(self.itemDropRate, self.mustDropItemValue,
    self.Entity.TransformComponent.Position)
end

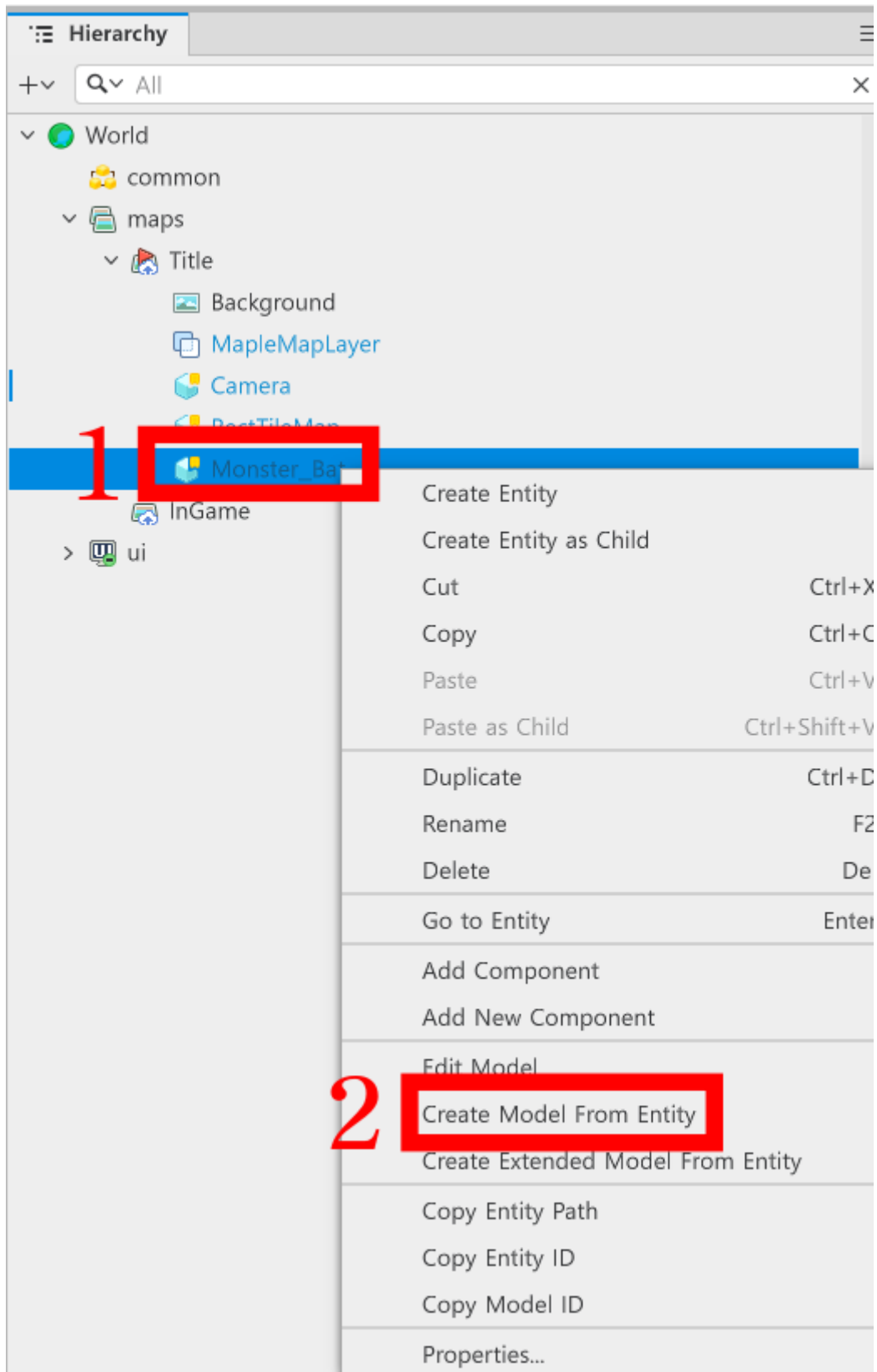
@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleHitEvent(HitEvent event)
    -- 몬스터의 상태가 죽은 상태라면 코드를 실행하지 않습니다.
    if not self.isAlive then
        return
    end
    -- 충돌된 미사일 엔티티를 가져옵니다.
    local attackerEntity = event.AttackerEntity
    -- 미사일이 전달한 데미지를 가져옵니다.
    local totalDamage = event.TotalDamage
    -- 히트 이펙트를 출력하기 위해 이펙트 출력 위치를 등록합니다.
    local effectSpawnPos = Vector3(attackerEntity.TransformComponent.Position.x,
    attackerEntity.TransformComponent.Position.y, 0)
    -- 데미지 계산을 진행합니다.
    self.currentHP = self.currentHP - totalDamage
    -- 맞았을 경우 히트 이펙트를 출력합니다.
    _EffectService:PlayEffect(event.Extra, self.Entity, effectSpawnPos, 0,
    Vector3(0.5, 0.5, 0.5))
    -- 피격 사운드를 재생합니다. 매개 변수의 첫 번째 값을 변경하여 다른 사운드를 재생할 수 있습니다.
    _SoundService:PlaySound("5873f227d7604a63997546a92824c135", 1.0)
    -- 체력이 0으로 되었을 경우, 사망 처리를 진행합니다.
    if self.currentHP <= 0 then
        -- 생존 상태를 false로 전환합니다.
        self.isAlive = false
        -- 몬스터가 사망했으므로 아이템 드롭 여부를 실행합니다.
        self:SendCalcDropItem()
        -- 사망시 출력할 이펙트를 출력합니다. 첫 번째 매개 변수의 값을 변경하여 사망시 출력

```

이펙트를 변경할 수 있습니다.

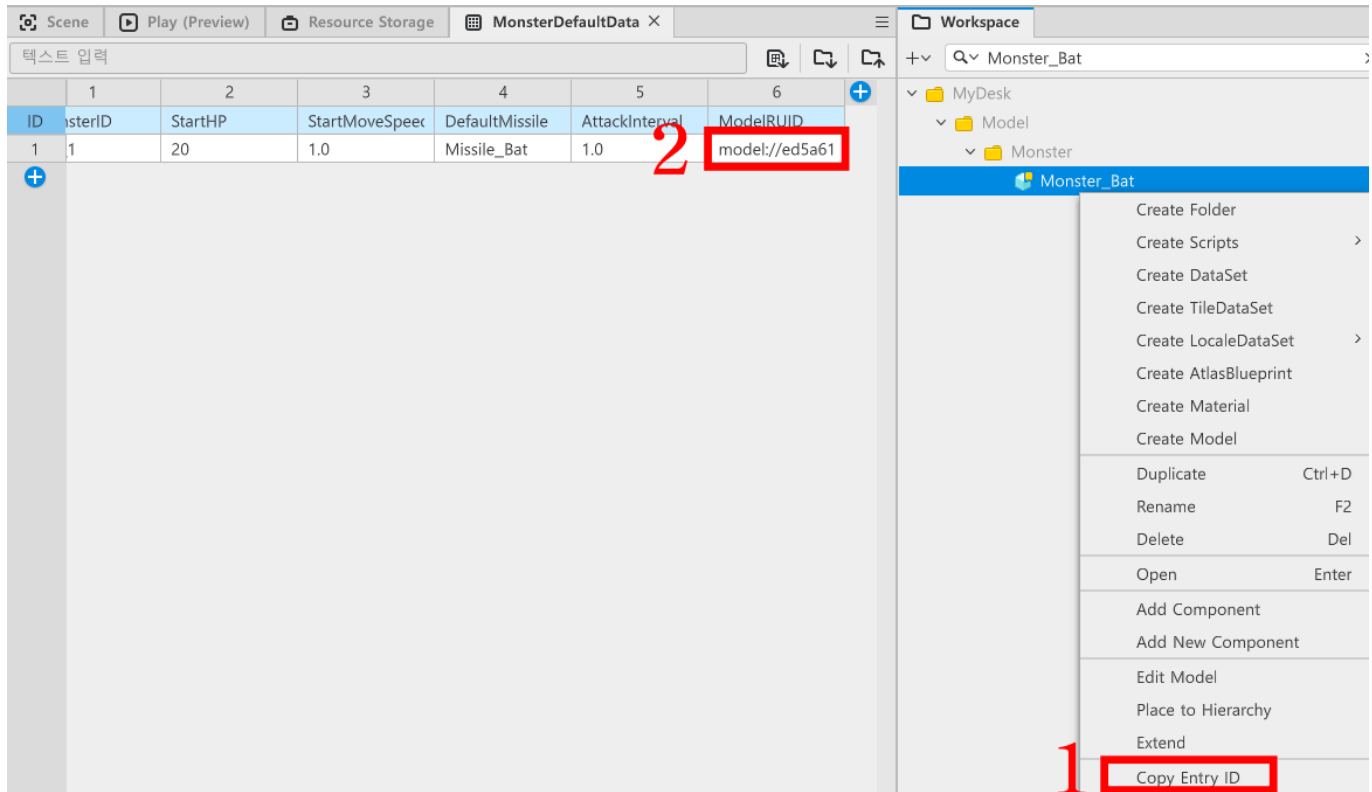
```
_EffectService:PlayEffect("f67f25d5f8e14d2c9962be0fa42808f7", self.Entity,  
effectSpawnPos, 0, Vector3(0.5, 0.5, 0.5))  
-- 몬스터 처치 여부를 GameLogic에 전달하여 조건 충족 여부를 검사하도록 합니다.  
_GameLogic:CountKillScore()  
-- 자기 자신을 파괴하여 엔티티를 소멸시킵니다.  
_EntityService:Destroy(self.Entity)  
  
end  
end  
  
end
```

제작한 엔티티를 Model화 하기



- 제작된 엔티티를 Model로 만든 뒤, WorkSpace에 추가해야 합니다.
- Model로 만드려는 엔티티를 마우스 우클릭한 뒤, **Create Model From Entity**를 클릭하여 Model을 제작합니다.

DataSet에 모델 ID 추가하기



- WorkSpace에 생성된 Model을 마우스 우클릭합니다.
- Copy Entry ID를 클릭하여 해당 모델의 ID값을 복사합니다.
- MonsterDefaultData의 ModelRUID에 해당 값을 추가합니다.

ModelRUID의 역할은 무엇입니까?

- ModelRUID의 역할은 WorkSpace에 있는 Model을 찾아서 해당 Model을 복제하기 위함입니다.
- 해당 부분은 심화 학습 항목인 아이템 구현하기의 MonsterSpawnManager를 제작할 때 사용됩니다.
- ModelRUID의 값으로 Model을 복제하는 방법을 학습하여 다양한 Monster를 소환하거나 필요에 따라 다른 Model을 복제할 수 있습니다.

최소 구현 기준

- Monster를 구성하는 Model이 제작되어야 합니다.
- 제작한 Monster Model이 플레이어를 추적해야 합니다.

응용 및 확장 방향

- 다양한 형태와 값을 가지는 Monster Model을 제작하여 Monster를 다양하게 준비합니다.
- AIChase를 사용하지 않고 Monster의 이동을 구현합니다.
- Monster마다 이동 방식을 구현하여 몬스터의 이동 패턴을 제작합니다.

[💡 기초 학습]기본 공격 제작하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 기본 공격 제작하기: 01:42:04

학습 목표

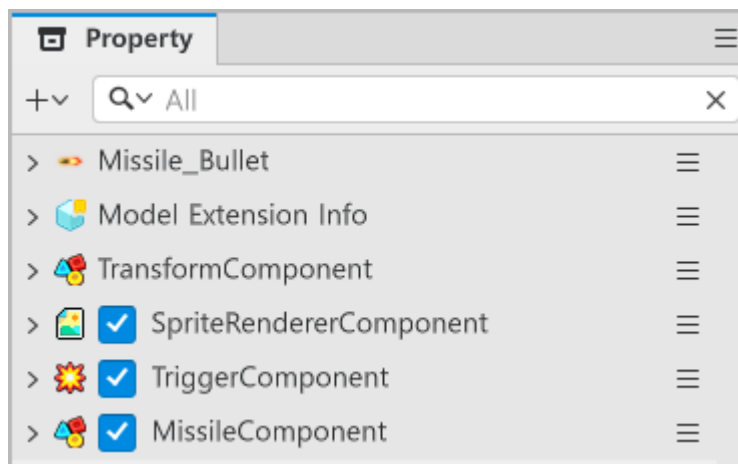
- TriggerComponent를 활용하여 충돌 그룹을 지정하는 Collision Group에 대해 학습합니다.
- Collision Group 간 충돌 관계를 설정하는 Matrix를 이해하고, 설정할 수 있습니다.

해당 영상 시청 후 수행해 볼 내용

- TriggerComponent의 Collision Group의 Matrix에 대해 학습합니다.
- 자신만의 Collision Group을 생성한 뒤, Matrix를 통해 관계를 형성하는 실습을 진행합니다.

투사체 엔티티 구성하기

- Hierarchy에서 빈 엔티티를 제작하고, 다음과 같이 Component를 구성합니다.



- SpriteRendererComponent:** 투사체의 이미지를 출력하기 위한 Component입니다. 원하는 투사체의 이미지를 지정하고 사용합니다.
- TriggerComponent:** 투사체의 충돌을 지정하기 위한 Component입니다.
- MissileComponent:** 투사체의 데이터와 행동을 제어하기 위해 제작한 Component입니다.

MissileComponent 스크립트 작성하기

- 다음과 같이 스크립트를 작성합니다.
- 해당 스크립트는 Plain Text를 기준으로 작성되어 있습니다.

```
@Component
script MissileComponent extends Component

property number moveSpeed = 0 -- 투사체의 속도를 저장하기 위한 Property입니다.

property number atkValue = 0 -- 투사체의 공격력을 저장하기 위한 Property입니다.

property number lifeTime = 0 -- 투사체의 유지 시간을 저장하기 위한 Property입니다.
```


property boolean isShotPlayer = false -- 투사체의 발사 주체를 구분하기 위한 Property입니다.

property string hitEffectRUID = "" -- 투사체가 명중했을 때 출력해야 하는 이펙트를 저장하기 위한 Property입니다.

property Vector2 fireDirection = Vector2(0,0) -- 투사체의 발사 방향을 저장하기 위한 Property입니다.

property number deltaTime = 0 -- 투사체가 생성된 시간을 계산하기 위한 Property입니다.

property string missileID = "" -- 현재 투사체의 ID를 저장하기 위한 Property입니다.

property any poolManager = nil

@ExecSpace("ClientOnly")

method void InitData(table MissileData)

-- 전달받은 MissileData가 사용 불가능한 값일 경우, 에러 로그를 출력하며 코드를 탈출합니다.

if MissileData == nil or next(MissileData) == nil then

log("MissileData가 비어 있습니다.")

return

end

-- 전달받은 데이터에서 투사체의 ID를 저장합니다.

self.missileID = MissileData.MissileID

-- 전달받은 데이터에서 투사체의 공격력을 저장합니다.

self.atkValue = MissileData.AtkValue

-- 전달받은 데이터에서 투사체의 이동속도를 저장합니다.

self.moveSpeed = MissileData.MoveSpeed

-- 전달받은 데이터에서 투사체의 유지 시간을 저장합니다.

self.lifeTime = MissileData.LifeTime

-- 전달받은 데이터에서 투사체가 충돌했을 경우 출력할 이펙트의 ID를 저장합니다.

self.hitEffectRUID = MissileData.HitEffectRUID

end

@ExecSpace("ClientOnly")

method void OnUpdate(number delta)

-- 만약 엔티티가 사용 불가능한 상태일 경우, 즉시 코드를 탈출합니다.

if self.Entity.Enable == false then

return

end

-- 만약 게임의 상태가 진행 중인 상태가 아닐 경우 즉시 코드를 탈출합니다.

if not _GameLogic.IsStillPlay then

return

end

-- 투사체의 현재 위치를 받아옵니다.

local transform = self.Entity.TransformComponent

-- 이동 속도, 시간만큼 이동해야 하는 거리를 계산합니다.

local moveVec = Vector3(self.fireDirection.x, self.fireDirection.y, 0) * self.moveSpeed * delta

-- 투사체의 현재 위치를 이동해야 하는 위치만큼 이동시킵니다.

```

transform.Position = transform.Position + moveVec

-- deltaTime를 delta 시간만큼 추가합니다.
self.deltaTime = self.deltaTime + delta
-- 만약 deltaTime가 lifeTime보다 클 경우, 투사체를 반환합니다.
if self.deltaTime >= self.lifeTime then
    self:ReturnSelf()
end
end

@ExecSpace("ClientOnly")
method void ReturnSelf()
    -- 투사체를 ProjectileManager에 반환합니다.
    if self.poolManager and invalid(self.poolManager) then
        self.poolManager:ReturnMissile(self.Entity)
    else
        -- 만약 poolManager가 nil인 상황인 경우, 투사체를 파괴합니다.
        _EntityService:Destroy(self.Entity)
    end
end

@ExecSpace("Client")
method void FireMissile(Vector2 FireDirection, boolean IsShotPlayer, Vector3
FireStartPos)
    -- deltaTime를 0으로 초기화합니다.
    self.deltaTime = 0
    -- 투사체의 발사 방향을 초기화합니다.
    self.fireDirection = FireDirection
    -- 투사체의 발사 주체를 초기화합니다. 만약 투사체의 발사 주체가 플레이어일 경우, true로
    저장됩니다.
    self.isShotPlayer = IsShotPlayer
    -- 투사체의 현재 위치를, 발사 위치로 변경합니다.
    self.Entity.TransformComponent.Position = FireStartPos
    -- 투사체의 사용 상태를 true로 전환합니다.
    self.Entity.Enable = true
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
    -- 충돌이 감지된 엔티티를 받아옵니다.
    local trigger = event.TriggerBodyEntity
    -- 충돌 가능 여부를 저장하기 위한 hitSuccess를 false로 초기화합니다.
    local hitSuccess = false
    -- 플레이어가 쏜 미사일이 몬스터와 충돌했을 경우 처리를 진행합니다.
    if invalid(trigger.MonsterState) and self.isShotPlayer then
        -- 충돌한 엔티티가 HitComponent를 가지고 있으며, 살아있는 상태일 경우, 데미지 계산을
        요청합니다.
        if trigger.HitComponent and trigger.MonsterState.isAlive then
            trigger.HitComponent:OnHit(self.Entity, self.atkValue, false,
self.hitEffectRUID, 1)
            hitSuccess = true
        end
    end
end

```

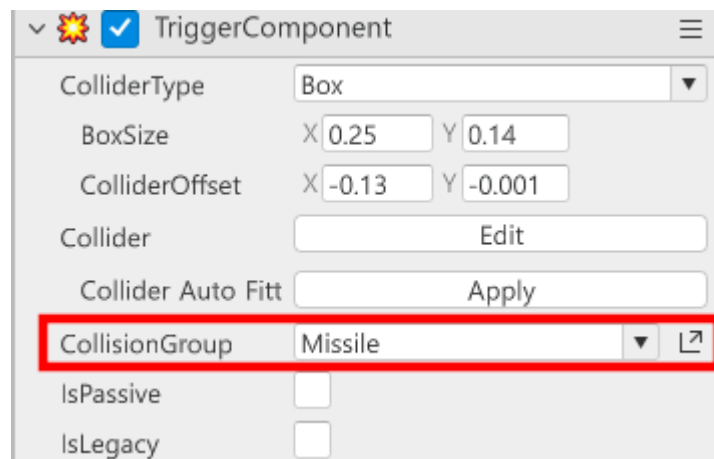
```
-- 몬스터가 쏜 미사일을 플레이어가 맞았을 경우 처리를 진행합니다.
elseif invalid(trigger.PlayerState) and self.isShotPlayer == false then
    -- 플레이어가 HitComponent를 가지고 있으며, 살아있는 상태인지, 현재 무적 상태인지
    -- 체크합니다.
    if trigger.HitComponent and trigger.PlayerState.isAlive and not
    trigger.PlayerState.isInvincibility then
        -- 데미지를 입을 수 있는 상태라면 OnHit를 호출하여 데미지 계산을 요청합니다.
        trigger.HitComponent:OnHit(self.Entity, self.atkValue, false,
        self.hitEffectRUID, 1)
        hitSuccess = true
    end
end

-- 만약 데미지 계산 처리가 이뤄졌다면 엔티티를 다시 비활성화합니다.
if hitSuccess then
    self:ReturnSelf()
end
end

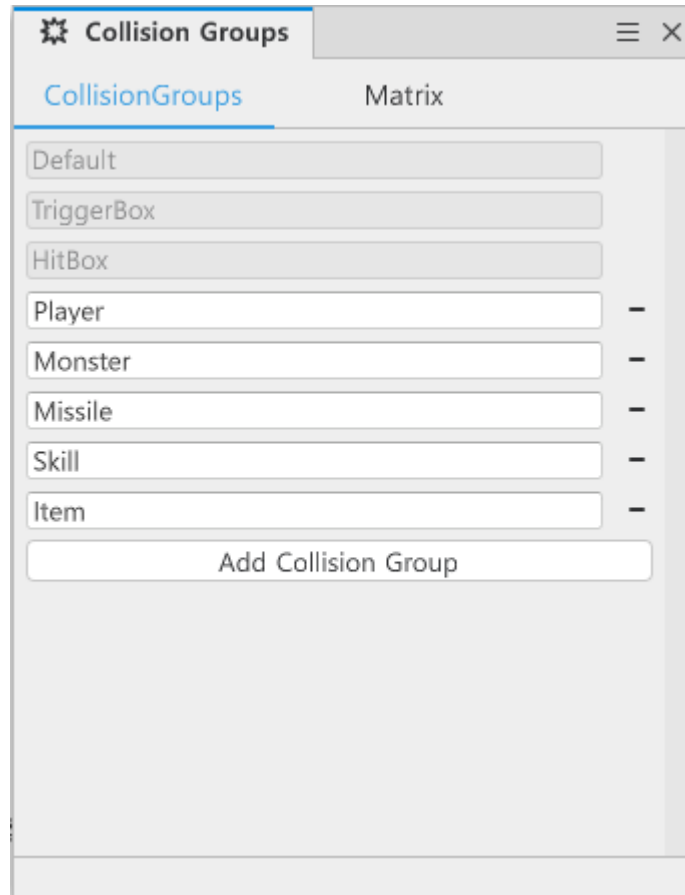
end
```

TriggerComponent 설정하기

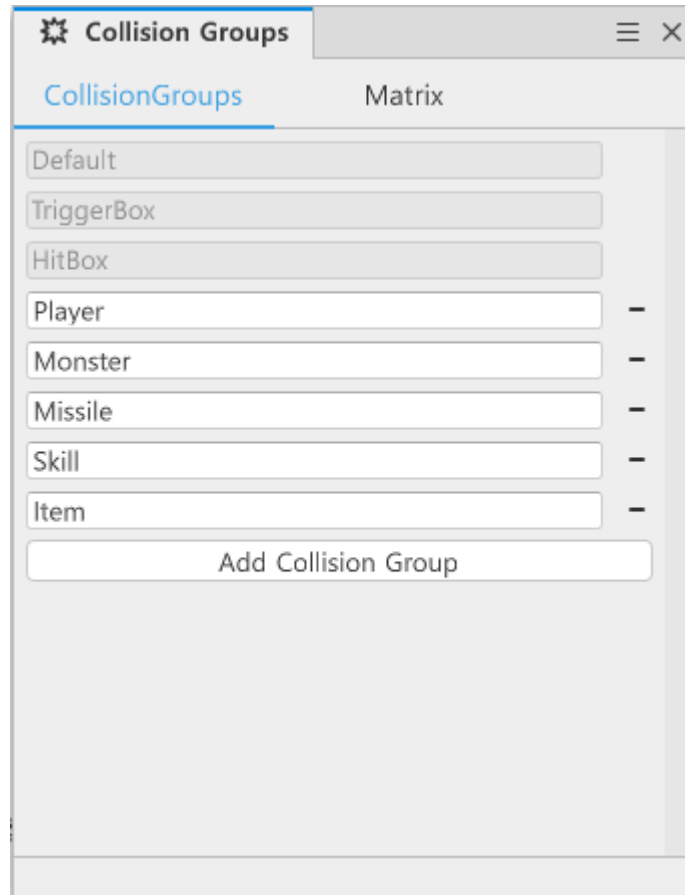
- TriggerComponent의 구성 요소를 변경하여, Collision Group과 Collision Group 간 충돌 관계를 변경할 수 있습니다.



- 제작한 엔티티의 TriggerComponent를 찾은 뒤, CollisionGroup의 우측에 있는 패널 열기 버튼을 클릭합니다.



- Collision Groups 창에서 Add Collision Group을 클릭하여, CollisionGroup을 추가합니다.
 - **Player**: 플레이어를 지정하기 위한 Collision Group입니다.
 - **Monster**: 적들을 지정하기 위한 Collision Group입니다.
 - **Missile**: 투사체를 지정하기 위한 Collision Group입니다.
 - **Skill**: 스킬을 지정하기 위한 Collision Group입니다.
 - **Item**: 플레이어에게 이로운 효과를 주기 위한 Collision Group입니다.
- 위의 설정을 마친 뒤, Matrix로 이동합니다.



- 위의 이미지와 같이 Matrix를 설정합니다.
- 해당 Matrix는 다음과 같은 충돌 관계를 형성합니다.
 - Player는 Monster, Missile, Item과 충돌을 체크합니다.
 - Monster는 Player, Missile, Skill과 충돌을 체크합니다.
 - Missile은 Player, Monster, Skill과 충돌을 체크합니다.
 - Skill은 Monster, Missile과 충돌을 체크합니다.
 - Item은 Player와 충돌을 체크합니다.

Matrix를 지정하는 이유

- 충돌 연산은 자주 일어나는 체크 요소 중 하나입니다.
- 불필요한 CollisionGroup을 추가하고, CollisionGroup들을 모두 충돌 체크한다면 불필요한 연산이 늘어나게 됩니다.
- 따라서 성능을 최적화하기 위해 불필요한 충돌 체크는 해제하고, 필요한 충돌만 체크하여 성능 향상을 노릴 수 있습니다.

최소 구현 기준

- 투사체 Model을 제작하여 최소 3개 이상의 사용 가능한 투사체가 존재해야 합니다.
- 투사체의 InitData를 통해 정상적으로 투사체 데이터가 입력되어야 합니다.
- InitData로 값이 입력된 투사체를 FireMissile로 발사 할 경우 정상적으로 투사체가 발사되어야 합니다.
- 투사체가 Monster와 충돌하여 투사체가 사라져야 합니다.

응용 및 확장 방향

- 투사체의 이동을 다양하게 추가하여 패턴을 제작합니다.

- 다양한 투사체를 통해 신규 캐릭터를 생성할 수 있습니다.
- 적마다 발사하는 투사체를 다르게 하여 패턴을 제작할 수 있습니다.

[💡 기초 학습]오브젝트 풀 생성하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 오브젝트 풀 제작하기: 02:02:03

학습 목표

- 오브젝트 풀의 원리에 대하여 학습합니다.
- 오브젝트 풀을 사용하여 투사체를 관리하고 사용할 수 있습니다.
- 오브젝트 풀을 통한 메모리 사용량을 관리하여 최적화 기법을 학습합니다.

해당 영상 시청 후 수행해 볼 내용

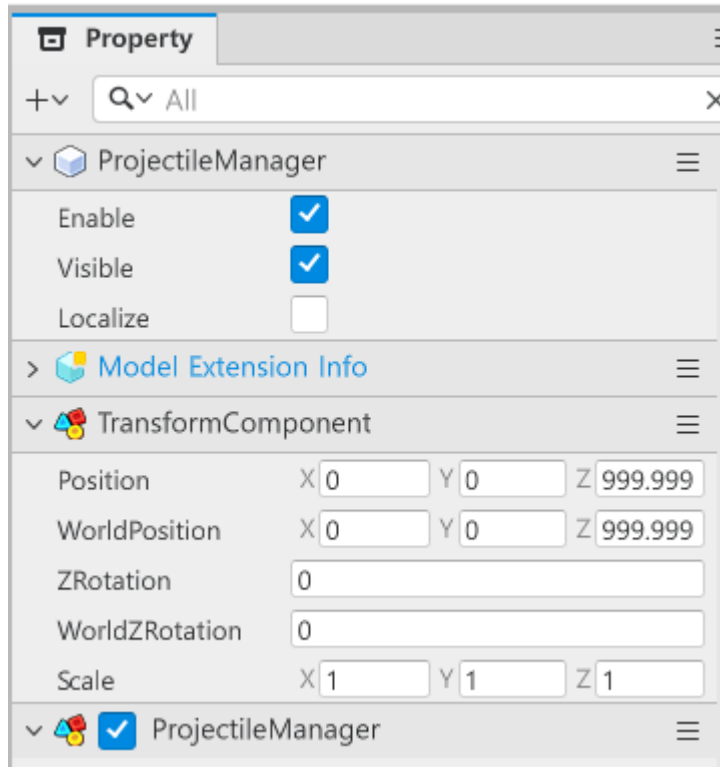
- 오브젝트 풀의 작동 원리와 실제 실행 결과를 직접 확인합니다.
- 해당 방식을 통해 어떤 요소들을 오브젝트 풀로 활용할 수 있을지 생각합니다.
- 생각한 요소를 오브젝트 풀 방식으로 관리할 수 있도록 구조를 설계합니다.

오브젝트 풀이란?

- 게임을 진행하는 데 있어 수많은 엔티티가 존재합니다.
- 이러한 엔티티 중, 몬스터와 같은 유닛은 사망하여 소멸하기도 합니다.
- 하지만 유닛의 수가 많을 경우 모든 유닛을 생성하고 소멸시키는 행위는 컴퓨터의 연산에 부하를 줍니다.
- 하지만 미리 엔티티를 생성하고, 소멸시키는 대신 활성화만 해제한다면 컴퓨터의 부하를 줄일 수 있습니다.
- 스위치를 켜듯 On, Off만 설정하면 되기 때문입니다.
- 오브젝트 풀은 미리 엔티티들을 만들어 놓고, 사용할 수 있는 엔티티를 전달하여 불필요한 생성과 소멸 업무를 최소화합니다.

오브젝트 풀 제작하기

- 오브젝트 풀을 제작하기 위해서는 Map 안에 엔티티를 생성해야 합니다.
- 샘플월드 기준 InGame맵 안에 ProjectileManager가 투사체를 관리하기 위한 오브젝트 풀로 사용되고 있습니다.
- 오브젝트 풀은 다음과 같은 Component로 이뤄져 있습니다.



- **TransformComponent:** 기본적으로 제공되는 Component입니다. 하지만 맵 내에 올바른 위치에 생성되기 위해 Position 값을 **0**으로 설정해야 합니다.
- **ProjectileManager:** 오브젝트 풀을 제작하기 위해 추가한 Component입니다. 해당 Component는 직접 제작해야 하므로 New Component를 통해 제작해야 합니다.

ProjectileManager 컴포넌트 설정하기

- 다음과 같이 코드를 작성해야 합니다.
- 해당 코드는 Plain Text 환경으로 작성되었습니다.

```
@Component
script ProjectileManager extends Component

property table projectilePool = {} -- 투사체 엔티티를 담는 table입니다.

@ExecSpace("ClientOnly")
method Entity GetMissile(string MissileKey)
-- 요구받은 MissileKey를 Key값으로 가지는 요소가 없으면, 초기화를 시도합니다.
if self.projectilePool[MissileKey] == nil then
    self.projectilePool[MissileKey] = {}
end

-- projectilePool내에 MissileKey를 Key로 가지는 요소를 Pool에 등록합니다.
local pool = self.projectilePool[MissileKey]
-- 사용할 수 있는 엔티티를 등록하기 위한 entity를 nil로 등록합니다.
local entity = nil

-- pool의 요소가 존재할 경우, 탐색을 시작합니다.
if #pool > 0 then
    -- pool 내에 있는 요소를 탐색합니다.
    for i = 1, #pool do
```

```

        -- 만약 i번째 엔티티가 존재하며, 현재 사용 상태가 false일 경우, 해당 엔티티를
        반환합니다.
        if isvalid(pool[i]) and not pool[i].Enable then
            entity = pool[i]
            break
        end
    end
end

-- 만약 현재까지 entity가 nil일 경우, 사용할 수 있는 엔티티가 존재하지 않으므로 생성을
시도합니다.
if entity == nil then
    -- MissieKey 값을 기준으로 DataTableLogic에 투사체 데이터를 요청합니다.
    local missileData = _DataTableLogic:GetMissileDataByMissileID(MissileKey)
    -- SpawnService를 통해 투사체를 생성합니다.
    entity = _SpawnService:SpawnByModelId(missileData.ModelRUID,
missileData.MissileID, Vector3.zero, self.Entity)
    -- 생성한 투사체에 투사체 데이터를 등록합니다.
    entity.MissileComponent:InitData(missileData)
    -- 투사체의 poolManager에 자기 자신을 등록합니다.
    entity.MissileComponent.poolManager = self
    -- 투사체의 사용 상태를 true로 변경합니다.
    entity.Enable = true
    -- projectilePool에 생성한 엔티티를 등록합니다.
    table.insert(self.projectilePool[MissileKey], entity)
else
    -- 사용할 수 있는 엔티티가 존재하므로, 찾은 엔티티의 사용 상태를 true로 변경합니다.
    entity.Enable = true
end
-- 찾은 혹은 생성한 엔티티를 반환합니다.
return entity
end

@ExecSpace("ClientOnly")
method void ReturnMissile(Entity missile)
    -- 만약 전달받은 엔티티가 사용 불가능할 경우, 코드를 즉시 탈출합니다.
    if not isvalid(missile) then return end
    -- 전달받은 엔티티의 사용 상태를 false로 반환합니다.
    missile.Enable = false
end

@ExecSpace("Client")
method void ResetPool()
    -- 자기 자신의 엔티티를 기준으로 자식 엔티티를 찾습니다.
    local childList = self.Entity.Children
    -- 만약 자식 엔티티가 없으면, return 합니다.
    if childList == nil then return end
    -- 찾은 자식 엔티티 리스트를 순회하며 엔티티의 Enable을 false로 바꿉니다.
    for i, child in pairs(childList) do
        if isvalid(child) then
            child.Enable = false
        end
    end
end
end

```



```

@ExecSpace("Client")
method Entity GetProjectileSkill(string SkillKey)
-- projectilePool에서 SkillKey를 가진 Value가 없으면, 초기화를 시도합니다.
if self.projectilePool[SkillKey] == nil then
    self.projectilePool[SkillKey] = {}
end

-- SkillKey를 Key로 가진 요소들을 pool에 등록합니다.
local pool = self.projectilePool[SkillKey]
-- 반환하기 위한 entity를 nil로 등록합니다.
local entity = nil

-- pool의 요소가 존재할 경우, 탐색을 시작합니다.
if #pool > 0 then
    -- pool 내에 있는 요소를 탐색합니다.
    for i = 1, #pool do
        -- 만약 i번째 엔티티가 존재하며, 현재 사용 상태가 false일 경우, 해당 엔티티를
        반환합니다.
        if invalid(pool[i]) and not pool[i].Enable then
            entity = pool[i]
            break
        end
    end
end

-- 만약 풀에 사용할 수 있는 스킬 투사체 엔티티가 없으면, 새로 엔티티를 생성합니다.
if entity == nil then
    -- SkillKey를 기준으로 DataTableLogic에서 데이터를 요청합니다.
    local skillData = _DataTableLogic:GetSkillDataBySkillID(SkillKey)
    -- SpawnService를 통해 투사체 스킬 모델을 생성 요청합니다.
    entity = _SpawnService:SpawnByModelId(skillData.ModelRUID, skillData.SkillID,
    Vector3.zero, self.Entity)
    -- 투사체 스킬의 데이터를 입력 요청합니다.
    entity.ProjectileSkillComponent:InitData(skillData)
    -- projectilePool에 생성한 엔티티를 등록합니다.
    table.insert(self.projectilePool[SkillKey], entity)
end

-- 반환하려는 엔티티의 사용 상태를 true로 변경합니다.
entity.Enable = true
-- 찾은 또는 생성한 엔티티를 반환합니다.
return entity
end

end

```

- 기존에 작성한 PlayerState와 MonsterState는 ProjectileManager를 참조하고 있습니다.
- PlayerState는 ConnectMissilePool로, MonsterState는 FindProjectileManager로 찾고 있습니다.
- 이를 통해 Player와 Monster는 오브젝트 풀을 통해 투사체를 발사할 수 있습니다.

최소 구현 기준

- 오브젝트 풀 기능을 활용하여 투사체 엔티티가 반복적으로 생성, 소멸하지 않도록 기능을 구현합니다.
- 투사체 생성 요청을 플레이어 또는 Monster가 자체적으로 요청하지 않습니다.
- 모든 투사체는 오브젝트 풀에서 생성 또는 사용 가능한 엔티티를 반환하는 형태로 구현합니다.

응용 및 확장 방향

- 오브젝트 풀의 원칙인 **미리 생성**하여 사용할 수 있도록 코드를 수정합니다.
- 수정된 코드를 기반으로 투사체들이 정상적으로 사용 가능한지 확인합니다.
- 오브젝트 풀이 **필요한 엔티티만** 미리 생성할 수 있도록 기능을 구현합니다.

[📖 심화 학습]스킬 구현하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 스킬 제작하기: **02:11:47**

학습 목표

- TriggerEvent의 종류를 학습합니다.
- 각 이벤트의 발생 조건을 이해하고, 필요한 기능을 제작할 수 있습니다.
- TriggerEvent 중 Stay의 경우 어떠한 위험성을 가졌는지 이해하고, 해당 Event를 활용해야 하는 상황을 설명할 수 있습니다.

해당 영상 시청 후 수행해 볼 내용

- 스킬 Model을 제작합니다.
- 영상에서 제공하는 투사체형 스킬을 다른 형태로 제작합니다.
- SkillData를 조절하여 스킬의 성능을 조절하고 변경합니다.
- 투사체형 스킬을 하나 더 제작한 뒤, 플레이어 캐릭터의 스킬을 변경합니다.

DataSet 제작하기

- 스킬 역시 데이터를 관리하기 위한 DataSet이 필요합니다.
- 샘플 월드에서는 다음과 같이 DataSet이 제작되어 있습니다.

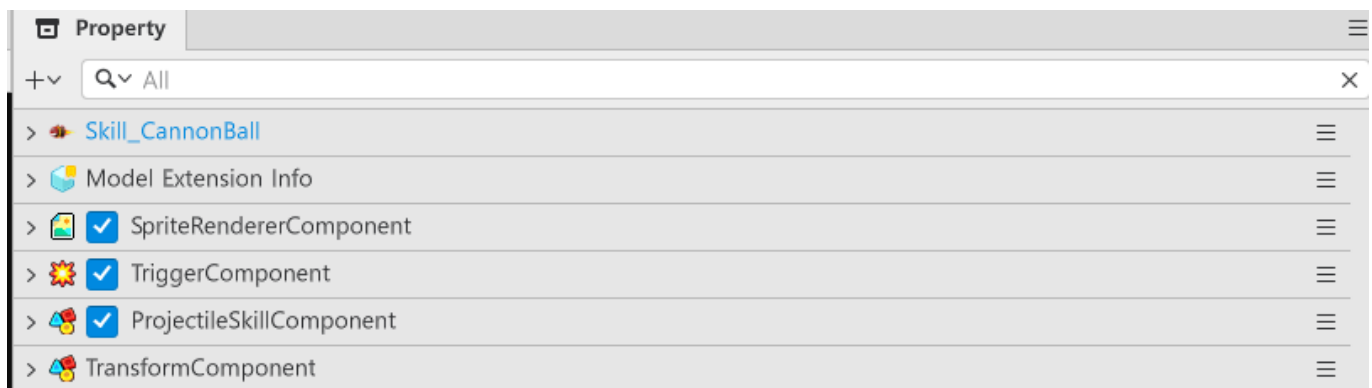
SkillData										
ID	SkillID	SkillType	IsPierce	AtkValue	MoveSpeed	LifeTime	MaxHitCount	HitInterval	ModelRUID	HitEffect
1	Skill_CannonBal	Projectile	true	20	1.0	5.0	5	0.2	model://a309b2	ee451f0e

이름	설명
----	----

이름	설명
SkillID	스킬을 구분하기 위한 ID 값입니다. 해당 값을 기준으로 플레이어의 스킬을 지정할 수 있습니다.
SkillType	스킬의 종류를 구별하기 위한 값입니다. 샘플 월드에서는 투사체형만 제작했지만, 추후 다른 형태의 스킬을 제작할 경우를 대비해 제작된 값입니다.
IsPierce	투사체 스킬의 경우 관통 여부를 결정하는 값입니다. 해당 값이 false일 경우, 투사체 스킬이 적 1개와 충돌해도 사라지도록 구성하기 위해 제작된 값입니다.
AtkValue	스킬의 공격력을 지정하기 위한 값입니다.
MoveSpeed	투사체 스킬의 경우 이동속도를 지정하기 위한 값입니다.
LifeTime	투사체 스킬의 경우 유지 시간을 지정하기 위한 값입니다.
MaxHitCount	다단 히트형 스킬의 경우, 횟수를 지정하기 위한 값입니다.
HitInterval	다단 히트형 스킬의 경우, 다음 히트까지 걸리는 시간을 지정하기 위한 값입니다.
ModelRUID	스킬 Model의 Entry ID를 등록하기 위한 값입니다.
HitEffectRUID	스킬에 피격되었을 때 출력해야 하는 이펙트의 RUID를 등록하는 값입니다.

스킬 엔티티 구성하기

- 스킬 Model 제작을 위해 엔티티를 구성해야 합니다.
- 다음과 같이 스킬 Entity에 Component를 구성합니다.



- SpriteRendererComponent:** 스킬의 이미지를 출력하기 위한 Component입니다.
- TriggerComponent:** 스킬의 충돌 여부를 검사하기 위한 Component입니다. 해당 Component의 CollisionGroup을 Skill로 설정해야 합니다.
- ProjectileSkillComponent:** 투사체 스킬의 데이터와 행동을 제어하기 위한 Component입니다. Add Component를 통해 제작해야 합니다.

스크립트 구성하기

- 다음과 같이 `ProjectileSkillComponent`를 구성해야 합니다.
- 아래의 코드는 Plain Text를 기준으로 작성되었습니다.

```
@Component
script ProjectileSkillComponent extends Component

property table collideTable = {} -- 스킬 투사체가 충돌 중인 엔티티를 관리하기 위한
table입니다.

property number atkValue = 0 -- 스킬 투사체의 공격력을 저장하기 위한 Property입니다.

property number hitInterval = 0 -- 투사체 스킬의 다단 히트 간격을 조절하기 위한
Property입니다.

property number maxHitCount = 0 -- 투사체 스킬의 최대 히트 간격을 조절하기 위한
Property입니다.

property boolean isPierce = false -- 투사체 스킬의 관통 가능 여부를 결정하기 위한
Property입니다.

property number lifeTime = 0 -- 투사체 스킬의 유지 시간을 결정하기 위한 Property입니
다.

property number moveSpeed = 0 -- 투사체 스킬의 이동 속도를 결정하기 위한 Property입니
다.

property string hitEffectRUID = "" -- 투사체 스킬에 맞았을 경우, 출력시킬 이펙트 ID를
저장하기 위한 Property입니다.

property number deltaTimer = 0 -- 투사체 스킬의 유지 시간 계산을 위한 Property입니다.

property Vector2 fireDirection = Vector2(0,0) -- 투사체가 진행되어야 하는 방향을 저장
하기 위한 Property입니다.

@ExecSpace("Client")
method void InitData(table SkillData)
--전달받은 스킬 데이터가 유효한지 체크합니다.
if not invalid(SkillData) or next(SkillData) == nil then
    log("전달받은 SkillData가 nil입니다!")
    return
end
-- 스킬 데이터 초기화를 진행합니다.
--투사체 스킬의 수명을 체크하는 deltaTimer를 초기화합니다.
self.deltaTimer = 0
-- 충돌한 몬스터들을 담은 테이블을 초기화합니다.
self.collideTable = {}
-- 스킬의 공격력 값을 초기화합니다.
self.atkValue = SkillData.AtkValue
-- 스킬의 공격 딜레이 값을 초기화합니다.
self.hitInterval = SkillData.HitInterval
```

```

-- 스킬의 최대 공격 횟수를 초기화합니다.
self.maxHitCount = SkillData.MaxHitCount
-- 스킬의 지속시간을 초기화합니다.
self.lifeTime = SkillData.LifeTime
-- 스킬의 이동속도를 초기화합니다.
self.moveSpeed = SkillData.MoveSpeed
-- 스킬이 관통 가능한지 여부를 검사합니다.
self.isPierce = SkillData.IsPierce
-- 스킬을 맞았을 때 출력할 이펙트 RUID를 초기화합니다.
self.hitEffectRUID = SkillData.HitEffectRUID
-- 투사체 스킬의 사용 여부를 우선 비활성화합니다.
self.Entity.Enable = false
end

@ExecSpace("Client")
method table GetOrCreateCollideEntity(Entity CollideEntity)
-- CollideEntity가 사용할 수 있는 경우 다음의 코드를 실행합니다.
if invalid(CollideEntity) then
-- CollideEntity의 ID를 가져옵니다.
local entityID = CollideEntity.Id
-- collideTable에서 ID 값을 가진 엔티티가 없다면 새롭게 collideTable에 등록합니다.
if not invalid(self.collideTable[entityID]) or not
next(self.collideTable[entityID]) then
self.collideTable[entityID] = {
Target = CollideEntity,
HitCount = 0,
LastHitTime = nil
}
end
-- CollideEntity의 ID를 가진 엔티티를 반환합니다.
return self.collideTable[entityID]
end
end

@ExecSpace("Client")
method void ApplyDamage(table CheckData)
-- 이미 사망하여 엔티티가 없으면, 해당 ID의 Table의 Value를 nil로 변경합니다.
if not invalid(CheckData.Target) then
self.collideTable[CheckData.Target.Id] = nil
return
end

-- 타겟의 MonsterState 컴포넌트 여부를 체크한 뒤, 컴포넌트에 데미지 계산을 수행합니다.
if invalid(CheckData.Target.MonsterState) then
CheckData.Target.HitComponent.OnHit(self.Entity, self.atkValue, false,
self.hitEffectRUID, 1)
end

-- 마지막 히트 시간을 등록합니다.
CheckData.LastHitTime = _UtilLogic.ElapsedSeconds
-- 해당 엔티티가 입을 수 있는 데미지의 횟수를 1 증가시킵니다.
CheckData.HitCount = CheckData.HitCount + 1
end

```

```

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 만약 게임의 Play 상태가 false의 경우 다음의 코드 실행을 중단합니다.
if not _GameLogic.IsStillPlay then
    return
end
-- 투사체 스킬이 이동하도록 요청합니다.
self:Move(delta)
-- 투사체의 유지 시간을 계산하도록 요청합니다.
self:RunLifeTimer(delta)

end

@ExecSpace("Client")
method void RunLifeTimer(number DeltaTime)
-- 투사체 스킬의 deltaTimer를 DeltaTime만큼 증가시킵니다.
self.deltaTimer = self.deltaTimer + DeltaTime
-- 만약 deltaTimer가 lifeTime을 넘을 경우, 사용 여부를 비활성화합니다.
if self.deltaTimer >= self.lifeTime then
    self.Entity.Enable = false
end

end

@ExecSpace("Client")
method void Move(number DeltaTime)
-- 엔티티의 현재 위치를 가져옵니다.
local transform = self.Entity.TransformComponent
-- 투사체 스킬의 위치를 이동 속도와 DeltaTime을 추가하여 계산합니다.
local moveVec = Vector3(self.fireDirection.x, self.fireDirection.y, 0) *
self.moveSpeed * DeltaTime
-- 투사체 스킬의 위치를 변경합니다.
transform.Position = transform.Position + moveVec
end

@ExecSpace("Client")
method void FireSkill(Vector2 FireDirection, Vector3 FireStartPos)
-- 투사체 스킬의 유지 시간 계산을 위한 deltaTimer 값을 0으로 초기화합니다.
self.deltaTimer = 0
-- 투사체 스킬의 발사 방향을 초기화합니다.
self.fireDirection = FireDirection
-- 투사체 스킬의 발사 위치를 초기화합니다.
local transform = self.Entity.TransformComponent
if transform then
    transform.Position = FireStartPos
end
-- 투사체의 사용 여부를 true로 전환합니다.
self.Entity.Enable = true
end

@EventSender("Self")
handler HandleTriggerStayEvent(TriggerStayEvent event)
-- 충돌 중인 Entity가 정상적으로 존재하는지 확인합니다.
if invalid(event.TriggerBodyEntity) then

```

```

--충돌 중인 Entity가 투사체인 경우, 적 투사체는 삭제합니다.
if invalid(event.TriggerBodyEntity.MissileComponent) and not
event.TriggerBodyEntity.MissileComponent.isShotPlayer then
    event.TriggerBodyEntity.MissileComponent:ReturnSelf()
    return
end
--충돌 중인 엔티티가 적인 경우, 히트 카운트가 충족하는 한 데미지 계산을 실행합니다.
if invalid(event.TriggerBodyEntity.MonsterState) then
    -- collideTable에서 충돌 중인 엔티티와 동일한 엔티티를 반환합니다.
    local collideTable =
self:GetOrCreateCollideEntity(event.TriggerBodyEntity)
    -- 만약 반환된 엔티티가 사용 불가능할 경우 코드 실행을 중단합니다.
    if not invalid(collideTable) then
        return
    end
    -- 만약 HitCount가 최대치를 넘길 경우, 코드 실행을 중단합니다.
    if collideTable.HitCount >= self.maxHitCount then
        return
    end
    -- 현재 시간을 가져옵니다.
    local currentTime = _UtilLogic.ElapsedSeconds
    -- 데미지 계산이 한번도 이뤄지지 않으면 경우 즉시 데미지를 가합니다.
    if collideTable.LastHitTime == nil then
        self:ApplyDamage(collideTable)
    -- 데미지를 받은 시간이 다단 히트 시간보다 클 경우 데미지 계산을 시도합니다.
    elseif currentTime - collideTable.LastHitTime >= self.hitInterval then
        self:ApplyDamage(collideTable)
    end
end
end
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- event가 비정상적일 경우 에러 로그를 출력합니다.
if not invalid(event) then
    log("event가 정상적인 event가 아닙니다!")
    return
end
-- 충돌한 엔티티가 MissileComponent를 가지고 있으며, 발사 주체가 플레이어가 아닌 경우
충돌한 엔티티를 비활성화합니다.
if invalid(event.TriggerBodyEntity.MissileComponent) and not
event.TriggerBodyEntity.MissileComponent.isShotPlayer then
    event.TriggerBodyEntity.MissileComponent:ReturnSelf()
    return
end
-- 충돌한 엔티티가 MonsterState를 가지고 있을 때 다음의 코드를 실행합니다.
if invalid(event.TriggerBodyEntity.MonsterState) then
    -- collideTable에서 충돌한 엔티티와 동일한 엔티티가 있는지 검사합니다.
    local findList = self:GetOrCreateCollideEntity(event.TriggerBodyEntity)

    -- 충돌한 엔티티가 데미지를 받은 횟수가 최대치를 넘었을 경우 코드 실행을 중단합니다.
    if findList.HitCount >= self.maxHitCount then return end

```

```

        -- 즉시 충돌한 엔티티에게 데미지 처리를 요청합니다.
        self:ApplyDamage(findList)
    end
end

@EventSender("Self")
handler HandleTriggerLeaveEvent(TriggerLeaveEvent event)
    -- 충돌 범위를 벗어난 엔티티가 사용 불가능한 상태일 경우 코드 실행을 중단합니다.
    if not isValid(event) then
        return
    end
    -- 충돌 범위를 벗어난 엔티티가 collideTable 내에 있는지 찾습니다.
    local record = self.collideTable[event.TriggerBodyEntity.Id]
    -- 만약 collideTable내 있는 엔티티일 경우, 마지막 히트 시간을 초기화하여 다시 충돌 범위에 들어올 경우 즉시 데미지를 받도록 합니다.
    if record then
        record.LastHitTime = nil
    end
end

end

```

- 해당 작업까지 마쳤다면, 스킬 엔티티를 Model화 한 뒤, Model의 EntryID를 SkillData의 ModelRUID에 등록해야 합니다.

HandleTrigger 이벤트의 호출 타이밍

- **EnterEvent:** TriggerComponent를 가진 2개 이상 엔티티의 충돌 범위가 겹치는 순간에 호출됩니다.
- **StayEvent:** TriggerComponent를 가진 2개 이상 엔티티의 충돌 범위가 겹쳐져 있는 동안, 프레임마다 호출됩니다.
- **LeaveEvent:** TriggerComponent를 가진 2개 이상 엔티티의 충돌이 이뤄지던 중, 충돌 범위의 겹침이 해제되는 순간 호출됩니다.

최소 구현 기준

- 플레이어가 스킬 엔티티를 사용할 수 있습니다.
- 스킬 엔티티가 적의 투사체를 제거할 수 있습니다.
- 투사체 스킬이 다수의 적을 공격하며, 지정된 횟수만큼 적을 공격할 수 있습니다.

응용 및 확장 방향

- 투사체형 스킬 외에 다른 형태의 스킬을 제작할 수 있습니다.
- 투사체형 스킬이 직선 이동이 아닌, 다양한 움직임을 보이며 스킬이 발사되도록 제작할 수 있습니다.

[📖 심화 학습]아이템 구현하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 아이템 구현하기: 02:32:10

학습 목표

- Logic을 활용하여 특정 기능을 담당하는 Logic 활용법을 학습합니다.
- 추후 아이템 생성의 기능을 변경하고자 할 경우, Logic만 변경할 수 있도록 확장성을 고려한 설계를 학습합니다.

해당 영상 시청 후 수행해 볼 내용

- 아이템의 효과 수치를 변경하고 변경된 수치만큼 값이 적용되는지 확인합니다.
- 제작된 아이템 외에 다른 효과를 제공하는 Model을 제작합니다.

DataSet 제작하기

- 아이템 드랍과 관련된 기능을 제작하기 위해서는 크게 2가지의 DataSet이 필요합니다.
- ItemData, ItemDropData 2가지가 존재합니다.
 - ItemData**: 아이템의 효과나 성능을 결정하기 위한 DataSet입니다.
 - ItemDropData**: 아이템의 드랍 확률을 결정하기 위한 DataSet입니다.

ItemDropData DataSet 구성

	1	2	+
ID	ItemID	DropRate	
1	Item_LifeRecove	20	
2	Item_SkillCount	20	
3	Item_MoveSpee	20	
4	Item_AttackInte	20	
5	Item_ChangeMi	20	
+			

이름	설명
ItemID	어떤 아이템인지 구분하기 위한 ID 값입니다. ItemData의 ID 값을 사용합니다.
DropRate	아이템의 드랍 확률을 결정하기 위한 값입니다.

ItemData DataSet 구성

	1	2	3	4	5	6	7	+
ID	ItemID	ValueType	TargetPlayerStat	NumberValue	StringValue	SpriteRUID	GainEffect	
1	Item_LifeRecover	1	currentHP	1		a2cfdb8831124f	32317a7b70484	
2	Item_SkillCount	1	canUseSkillCour	1		04640b1d108d4	196d4ef7652f44	
3	Item_MoveSpeed	1	moveSpeed	0.05		e223bdc2a6cd4	6802f1a489f94b	
4	Item_AttackInterval	1	attackInterval	-0.05		fafd61650bf146	3db73bd6398e4	
5	Item_ChangeMissile	2	currentMissile		Missile_Missile	fd8586b0c4c94f	0915b72bf4374	
+								

이름	설명
ItemID	아이템을 구분하기 위한 ID 값입니다. 해당 값을 ItemDropData의 ItemID 값으로 사용합니다.
ValueType	아이템의 효과 값이 숫자 값인지, 텍스트형 값인지 구분하기 위한 요소입니다.
TargetPlayerStateName	PlayerState에서 어떤 Property에 영향을 줘야 하는지 설정하는 값입니다. PlayerState의 Property 명과 동일하게 작성해야 합니다.
NumberValue	숫자 형태의 값을 가지는 아이템 효과인 경우, 해당 값을 기준으로 효과를 적용합니다.
StringValue	텍스트 형태의 값을 가지는 아이템 효과인 경우, 해당 값을 기준으로 효과를 적용합니다. 샘플 월드에서는 기본 공격 변경 아이템 구현을 목적으로 제작되었습니다.
SpriteRUID	아이템의 이미지를 변경하기 위해 제작된 값입니다. 해당 값을 통해 아이템의 이미지를 변경합니다.
GainEffect	아이템을 습득했을 때, 아이템의 습득 효과를 지정하기 위한 값입니다.

ItemSpawnManager 생성하기

- 아이템을 실질적으로 생성하는 담당자인 Component입니다.
- 해당 요소는 엔티티 형태로 제작되어 있으며 샘플월드 기준 InGame에 제작되어 있습니다.
- 엔티티와 Component를 제작한 뒤, 아래의 코드를 ItemSpawnManager에 등록해야 합니다.
- 코드는 Plain Text를 기준으로 작성되었습니다.

```
@Component
script ItemSpawnManager extends Component

property string itemModelID = "model://0b6340f2-19d9-4a27-b18a-70feb94ce2ae" --
Workspace 내 아이템으로 사용하기 위한 Model ID입니다.

property table itemObjectPool = {} -- 사용할 수 있는 엔티티를 찾고 재사용하기 위한
table입니다.

@ExecSpace("Client")
method void CreateItem(string ItemKey, Vector3 ItemSpawnPosition)
```

```

-- itemObjectPool이 nil인지 확인합니다.
if self.itemObjectPool[ItemKey] == nil then
    -- 만약 itemObjectPool이 nil인 경우, {}으로 초기화합니다.
    self.itemObjectPool[ItemKey] = {}
end

-- pool 내에 전달받은 ItemKey 종류의 엔티티를 pool에 등록합니다.
local pool = self.itemObjectPool[ItemKey]
-- 사용할 수 있는 엔티티를 등록하기 위해 returnEntity를 nil로 설정합니다.
local returnEntity = nil

-- pool의 요소가 1개라도 있을 때, 탐색을 시작합니다.
if #pool > 0 then
    -- pool의 요소 개수만큼 탐색을 시작합니다.
    for i = 1, #pool do
        -- 만약 i번째 엔티티가 사용할 수 있는, i번째 엔티티를 반환합니다.
        if invalid(pool[i]) and not pool[i].Enable then
            returnEntity = pool[i]
            break
        end
    end
end

-- DataTableLogic에서 ItemKey 값을 기준으로 값을 찾습니다.
local itemData = _DataTableLogic.GetItemDataByItemID(ItemKey)
-- 만약 지금까지 returnEntity가 nil일 경우, 사용할 수 있는 엔티티가 없다고 판단합니다.
if returnEntity == nil then
    -- Model로 만들어 둔 아이템 엔티티를 SpawnService를 이용해 엔티티를 맵에 생성합니
    다.
    returnEntity = _SpawnService:SpawnByModelId(self.itemModelID, itemData.ItemID,
    Vector3.zero, self.Entity)
    -- 생성한 아이템이 다시 반환될 수 있도록 ItemComponent의 poolManager에 자신을 등록
    시킵니다.
    returnEntity.ItemComponent.poolManager = self
    -- 아이템이 사용 불가능한 상태일 경우를 대비하여, 엔티티의 Enable을 true로 변경합니
    다.
    returnEntity.Enable = true
    -- itemObjectPool에 생성한 요소를 ItemKey 값을 Key로, 엔티티를 Value로 추가합니
    다.
    table.insert(self.itemObjectPool[ItemKey], returnEntity)
else
    -- 만약 returnEntity가 존재한 경우, 사용할 수 있도록 Enable을 true로 변경합니다.
    returnEntity.Enable = true
end

-- 생성한 아이템 엔티티의 데이터를 초기화합니다.
returnEntity.ItemComponent:InitData(itemData, ItemSpawnPosition)
end

@ExecSpace("Client")
method void ReturnItemEntity(Entity ItemEntity)
    -- 만약 전달받은 ItemEntity가 사용 불가능한 엔티티인 경우 return을 통해 코드를 탈출합니
    다.
    if not invalid(ItemEntity) then return end
    -- ItemEntity가 사용할 수 있는 경우, Enable을 비활성화하여 다시 사용할 수 있도록 대기상

```

```

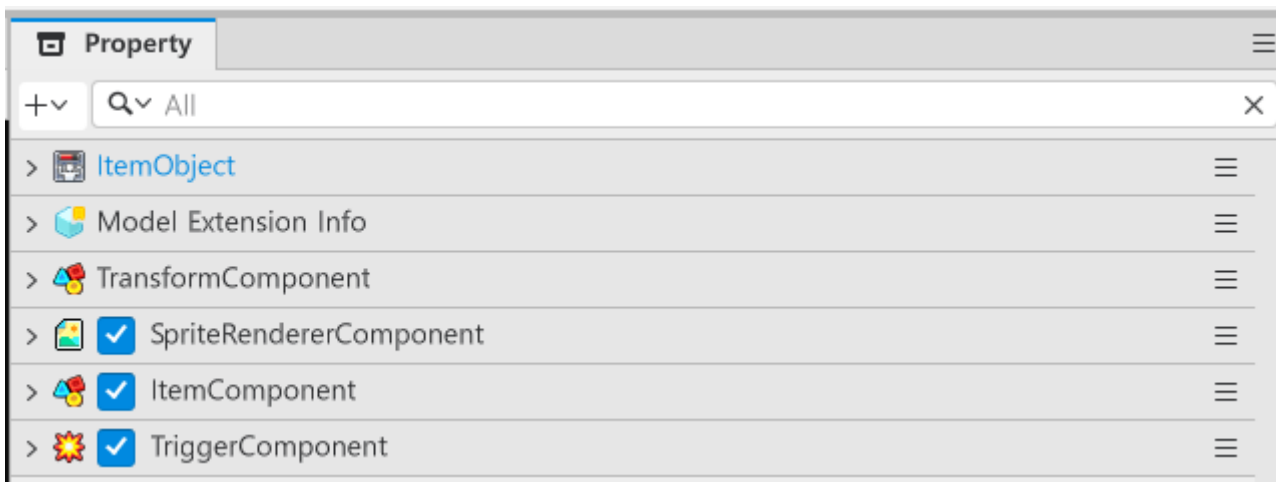
태로 전환합니다.
ItemEntity.Enable = false
end

end

```

아이템 엔티티 제작하기

- 아이템을 구성하는 엔티티를 제작하고, 해당 엔티티를 Model로 등록해야 합니다. 이때 Model의 EntryID 값은 ItemSpawnManager의 itemModelID의 값으로 사용됩니다.



- SpriteRendererComponent:** 아이템의 이미지를 출력하기 위한 Component입니다. 해당 Component의 ImageRUID값을 ItemData의 SpriteRUID값을 적용하여 아이템 아이콘을 변경합니다.
- TriggerComponent:** 아이템의 충돌을 감지하기 위한 Component입니다. 해당 Component를 추가한 뒤 Collision Group을 Item으로 변경해야 합니다.
- ItemComponent:** 아이템의 이동, 효과를 담기 위한 Component입니다. 해당 Component는 제작해야 하는 Component이므로 Add Component, New Component로 제작해야 합니다.

ItemComponent 스크립트 작성하기

- 아래의 코드를 ItemComponent에 작성해야 합니다.
- 코드는 Plain Text를 기준으로 작성되었습니다.

```

@Component
script ItemComponent extends Component

property SpriteRendererComponent imageSpriteRenderer = nil

property number itemID = 0 -- ItemData의 ID 값을 저장하기 위한 Property입니다.

property number valueType = 0 -- ItemData의 적용 타입을 구분하기 위한 Property입니다.

```

```

property string targetPlayerStateName = "" -- PlayerState 컴포넌트에서 적용해야 하는
Property의 이름을 담기 위한 Property입니다.

property number numberValue = 0 -- 숫자 형태의 값을 적용해야 할 경우, 숫자 값을 담기
위한 Property입니다.

property string stringValue = "" -- 텍스트 형태의 값을 적용해야 할 경우, 텍스트를 담
기 위한 Property입니다.

property number moveSpeed = 3.0 -- 아이템이 맵에서 이동하는 속도를 저장하기 위한
Property입니다.

property number lifeTime = 3.0 -- 아이템이 맵에서 잔존할 수 있는 시간을 담기 위한
Property입니다.

property string gainEffect = "" -- 아이템 획득 시 출력하려는 이펙트의 ID를 담기 위한
Property입니다.

property any poolManager = nil

property number deltaTime = 0 -- 아이템의 유지 시간을 계산하기 위한 Property입니다.

@ExecSpace("Client")
method void InitData(table ItemData, Vector3 SpawnPosition)
-- 아이템의 이미지 스프라이트를 관리하는 Property가 nil일 경우, 초기화를 시작합니다.
if not isvalid(self.imageSpriteRenderer) then
    -- 해당 컴포넌트가 붙은 엔티티에서 SpriteRendererComponent를 찾습니다.
    local findImageSpriteRenderer = self.Entity.SpriteRendererComponent
    if not isvalid(findImageSpriteRenderer) then
        -- 만약 SpriteRendererComponent를 찾지 못했다면, 찾지 못했다는 에러 로그를 남깁
        니다.
        log("아이템 모델에 SpriteRendererComponent가 존재하지 않습니다!")
        return
    end
    -- SpriteRenderer를 찾았다면, Property에 찾은 컴포넌트를 등록합니다.
    self.imageSpriteRenderer = findImageSpriteRenderer
end

-- 전달받은 ItemData가 nil일 경우 에러 로그를 반환하며 초기화를 시도하지 않습니다.
if not isvalid(ItemData) or not next(ItemData) then
    log("Init으로 전달받은 ItemData가 nil입니다!")
    return
end

-- deltaTime를 0으로 초기화하여, 유지 시간을 계산합니다.
self.deltaTime = 0
-- 전달받은 ItemData의 데이터를 Component의 Property에 등록합니다.
-- 아이템 아이콘 이미지 RUID를 등록합니다.
self.imageSpriteRenderer.SpriteRUID = ItemData.SpriteRUID
-- 아이템의 영향력을 결정하는 Value 값을 등록합니다.
self.valueType = ItemData.ValueType
-- PlayerState의 Property와 같은 이름을 가진 값을 등록합니다.
self.targetPlayerStateName = ItemData.TargetPlayerStateName
-- 숫자 형 값의 변동이 있는 아이템일 경우, 상승량을 등록합니다.

```

```

self.numberValue = ItemData.NumberValue
-- 기본 미사일과 같이 String 형태의 Key 값을 변경하는 아이템일 경우, String 값을 등록합
니다.
self.stringValue = ItemData.StringValue
-- 아이템 습득 시 출력될 이펙트 ID 값을 등록합니다.
self.gainEffect = ItemData.GainEffect
-- 몬스터가 죽은 위치를 받아, 해당 위치에서 스폰됩니다.
self.Entity.TransformComponent.Position = SpawnPosition
-- 만약 엔티티가 비활성화 상태일 경우, 활성화합니다.
if not self.Entity.Enable then
    self.Entity.Enable = true
end
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 만약 엔티티가 비활성화 상태일 경우 즉시 Return을 통해 아래 스크립트를 실행하지 않습니
다.
if not self.Entity.Enable then
    return
end

-- 게임의 플레이 상태가 플레이 중이 아닐 경우, Return을 통해 아래 스크립트를 실행하지 않
습니다.
if not _GameLogic.IsStillPlay then
    return
end

-- 만약 deltaTime가 아이템의 생존 시간을 넘길 경우, 아이템을 비활성화시킵니다.
if self.deltaTime > self.lifeTime then
    self:ReturnSelf()
    return
else
    -- 아직 deltaTime가 lifeTime을 넘기지 않았을 경우, 진행된 시간만큼 deltaTime를
상승시킵니다.
    self.deltaTime = self.deltaTime + delta
end

-- 아이템의 이동 함수를 프레임마다 호출합니다.
self:ItemMove(delta)
end

@ExecSpace("Client")
method void ItemMove(number DeltaTime)
-- 아이템의 위치를 관리하는 TransformComponent를 가져옵니다.
local transform = self.Entity.TransformComponent

-- 아이템이 이동해야 하는 거리를 계산합니다.
local moveVec = Vector3(-1, 0, 0) * self.moveSpeed * DeltaTime

-- 아이템의 현재 위치를 이동해야 하는 거리만큼 이동시킵니다.
transform.Position = transform.Position + moveVec
end

@ExecSpace("Client")

```

```

method void ReturnSelf()
    -- LifeTime이 끝났을 경우, 아이템의 Enable을 false로 변경합니다.
    if self.poolManager and invalid(self.poolManager) then
        self.poolManager:ReturnItemEntity(self.Entity)
    else
        -- 만약 poolManager가 없으면, 아이템 엔티티를 파괴합니다.
        _EntityService:Destroy(self.Entity)
    end
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
    -- 충돌한 엔티티가 PlayerState가 있을 때, 아이템 사용 처리를 진행합니다.
    if invalid(event.TriggerBodyEntity.PlayerState) then
        -- 충돌한 플레이어 엔티티의 AppliedItem 메소드를 실행합니다.
        event.TriggerBodyEntity.PlayerState:AppliedItem(self.valueType,
self.targetPlayerStateName, self.stringValue, self.numberValue, self.gainEffect)
        -- 해당 엔티티를 비활성화 처리합니다.
        self:ReturnSelf()
    end
end

end

```

ItemHelper 제작하기

- ItemHelper는 몬스터가 사망했을 경우, 아이템 생성을 요청하기 위한 Logic입니다.
- ItemHelper가 요청을 수신하면 확률 계산을 진행하고, 아이템을 드랍해야 한다면 아이템 드랍을 ItemSpawnManager에게 아이템 생성을 요청합니다.
- 샘플 월드에서는 ItemHelper라는 이름으로 Logic을 제작했습니다.
- 코드는 Plain Text를 기준으로 작성되었습니다.

```

@Logic
script ItemHelper extends Logic

property ItemSpawnManager itemSpawnManager = nil

method void CalcDropItemData(number ItemDropRate, string MustDropItemKey, Vector3
SpawnPosition)
    -- 아이템 드롭 확률이 0이므로, 확률 계산을 실행하지 않고 nil을 리턴합니다.
    if not invalid(ItemDropRate) or ItemDropRate <= 0.0 then
        return
    end

    -- 이외의 경우, 아이템 드롭 확률 여부를 결정합니다.
    if ItemDropRate >= 1.0 then
        -- 반드시 아이템을 드랍하는 상황이므로 추가적인 연산을 실행하지 않습니다.
    else
        -- ItemDropRate의 값에 맞춰 랜덤을 시행합니다.
        local randomValue = _UtilLogic:RandomDouble()
        if ItemDropRate < randomValue then

```

```

        -- 랜덤한 값이 Item이 드롭될 확률보다 높으므로, 드랍에 실패한 상황으로 간주하고
        nil을 반환합니다.
        return
    end
end

-- 만약 MustDropItemKey가 지정되어 있을 때, 해당 아이템을 반드시 반환합니다.
if not _UtilLogic:IsNilorEmptyString(MustDropItemKey) then
    -- MustDropItemKey를 기준으로 DataTableLogic에게 데이터를 요청합니다.
    local findData = _DataTableLogic:GetItemDataByItemID(MustDropItemKey)
    -- MustDropItemKey값을 가진 데이터가 있을 때 아이템을 생성합니다.
    if isValid(findData) and next(findData) then
        self:SendSpawnItem(findData.ItemID, SpawnPosition)
        return
    else
        -- 만약 없으면, 아이템을 찾을 수 없다는 에러 로그를 남깁니다.
        log("MustDropItemKey인 아이템을 찾을 수 없습니다!")
        return
    end
end

-- MustDropItemKey가 공란일 경우, 랜덤한 아이템을 제공하도록 랜덤을 실행합니다.
local getDropData = _DataTableLogic:GetItemDropData()
local totalValue = 0
-- 아이템 드랍 테이블의 모든 확률을 더한 뒤, 해당 확률에서 계산을 시도합니다.
for _, data in pairs(getDropData) do
    totalValue = totalValue + data.DropRate
end
-- 0에서 totalValue 사이의 값을 결정한 뒤, 1번부터 DropRate를 더해, 랜덤한 값보다 큰
타이밍의 아이템을 당첨으로 리턴합니다.
local randomValue = _UtilLogic:RandomIntegerRange(0, totalValue)
local resultValue = 0
for _, result in pairs(getDropData) do
    -- resultValue에 DropRate를 더합니다.
    resultValue = resultValue + result.DropRate
    -- resultValue의 값이 결정된 randomValue보다 클 경우, 당첨으로 처리합니다.
    if resultValue >= randomValue then
        -- DataTableLogic에서 ItemData를 ItemID 값으로 찾습니다.
        local findData = _DataTableLogic:GetItemDataByItemID(result.ItemID)
        -- 데이터를 정상적으로 가져왔다면, 찾은 데이터를 반환합니다.
        if isValid(findData) and next(findData) then
            self:SendSpawnItem(findData.ItemID, SpawnPosition)
            return
        else
            -- 만약 찾은 데이터가 nil 또는 사용할 수 없는 데이터일 경우, 로그를 남기며 nil
            을 반환합니다.
            log("ItemData에서 "..result.ItemID.."라는 이름의 값을 찾지 못했습니다!")
            return
        end
    end
end
end
end

method void SendSpawnItem(string ItemID, Vector3 SpawnPosition)

```



```

-- InGame에서 ItemSpawnManager를 찾지 못했다면 에러 로그를 출력합니다.
if not invalid(self.itemSpawnManager) then
    log("ItemHelper의 itemSpawnManager가 nil입니다!")
    return
end
-- InGame의 ItemSpawnManager에게 아이템을 생성하도록 요청합니다.
self.itemSpawnManager:CreateItem(ItemID, SpawnPosition)
end

method void InitData()
-- InGame의 ItemSpawnManager를 찾지 못했을 경우 InGame의 ItemSpawnManager를 찾습니
다.
if not invalid(self.itemSpawnManager) then
    local findEntity =
_EntityService:GetEntityByPath("/maps/InGame/ItemSpawnManager")
    -- 만약 ItemSpawnManager를 찾지 못했다면 에러 로그를 출력합니다.
    if not invalid(findEntity.ItemSpawnManager) then
        log("InGame맵에 ItemSpawnManager 엔티티, 또는 ItemSpawnManager 컴포넌트가 존
재하지 않습니다!")
        return
    end
    -- 찾은 ItemSpawnManager를 property에 등록합니다.
    self.itemSpawnManager = findEntity.ItemSpawnManager
end
end

method void ReleaseData()
-- Title 화면에서 ItemSpawnManager에 접근하여 사용하지 못하도록 해제합니다.
self.itemSpawnManager = nil
end

end

```

최소 구현 기준

- ItemHelper가 정상적으로 랜덤한 아이템을 출력할 수 있어야 합니다.
- ItemComponent가 정상적으로 ItemData의 DataSet 값을 받아오고, 적용되어야 합니다.

응용 및 확장 방향

- 제작된 아이템 외에도 다른 효과를 제공하는 아이템을 제작합니다.
- 필요에 따라 투사체 강화 기능과, 아이템 습득 효과를 추가하여 기본 공격 강화 시스템을 구현합니다.

[📖 심화 학습] MonsterSpawn 제작하기

영상 타임라인

- 해당 내용은 **아이템 구현하기**의 항목에서 함께 다루고 있습니다.
- 아이템 구현하기: **02:31:57:07**

학습 목표

- TimerService를 활용하여 일정한 시간 뒤, 원하는 행동을 실행할 수 있는 방법을 학습합니다.
- GameLogic을 활용하여, 게임오버와 스테이지 클리어를 체크할 수 있도록 기능을 구성합니다.

해당 영상 시청 후 수행해 볼 내용

- MonsterSpawnData의 값을 변경하여 Monster가 출력되는 시간을 조절합니다.
- Monster를 추가로 제작한 뒤, 해당 Monster가 스폰될 수 있도록 MonsterSpawnData를 수정합니다.
- ItemDropRate와 MustDropItem을 수정하여 각 Monster가 처치될 때 아이템을 드랍할 확률을 변경합니다.

DataSet 제작하기

- Monster를 스폰시키기 위해 DataSet을 제작해야 합니다.
- 샘플 월드에서는 **MonsterSpawnData**라는 이름으로 제작되었습니다.

	1	2	3	4	5	6	7	+
ID	StageID	MonsterID	SpawnPosX	SpawnPosY	EntranceTime	ItemDropRate	MustDropItem	
1	1	Bat_1	8.0	2.0	3.0	0.0		
2	1	Bat_1	8.0	-2.0	5.0	0.8		
3	1	Bat_1	6.0	0.0	8.0	1.0	Item_MoveSpee	
4	1	Bat_1	6.0	-5.0	8.0			
+								

이름	설명
StageID	MonsterSpawnManager가 현재 스테이지 번호에 맞춰, 스폰해야 하는 Monster의 종류를 구분하기 위한 ID 값입니다.
MonsterID	어떤 Monster를 스폰해야 하는지 찾기 위한 ID 값입니다. 해당 값을 기준으로 MonsterDefaultData에서 MonsterID를 기준으로 검색합니다.
SpawnPosX, SpawnPosY	Monster가 맵의 어느 위치에서 스폰되어야 하는지 지정하는 값입니다. 각각 Position의 X, Y의 값입니다.
EntranceTime	해당 Monster가 게임 시작 후 몇 초 뒤에 생성되어야 하는지 지정하는 값입니다.
ItemDropRate	Monster가 사망 시, 아이템을 드랍할 확률을 지정하기 위한 값입니다.
MustDropItem	Monster가 아이템을 드랍해야 할 경우, 특정 아이템을 고정적으로 드랍시키고 싶은 경우 사용하는 값입니다. 해당 값은 ItemData의 ItemID 값을 사용합니다.

MonsterSpawnManager제작하기

- MonsterSpawnManager를 제작하기 위해 InGame에 엔티티를 추가해야 합니다.
- 또한 MonsterSpawnManager Component를 구성해야 합니다.
- MonsterSpawnManager를 구성하는 코드는 아래와 같습니다.
- 해당 코드는 Plain Text를 기준으로 작성되었습니다.

```
@Component
script MonsterSpawnManager extends Component

property number stageID = 0 -- 스테이지를 구분하기 위한 Property입니다.

property number needKillCount = 0 -- 스테이지 시작 시, 처치 가능한 몬스터의 수를 계산
하고 저장하기 위한 Property입니다.

property table timerTable = {} -- 몬스터가 등장하는 시간을 저장하기 위한 table입니다.

@ExecSpace("Client")
method void StartGame()
-- 스테이지 클리어를 위한 몬스터 처치 수를 우선 0으로 초기화합니다.
self.needKillCount = 0
-- 스테이지의 몬스터 등장 시간을 담은 timerTable을 초기화합니다.
self.timerTable = {}
-- 만약 stageID가 0일 경우, 게임이 정상적으로 실행되도록 임의로 1로 설정합니다.
if self.stageID == 0 then
    self.stageID = 1
    -- 또한 현재 stageID가 0으로 등록되어 있음을 Log로 알립니다.
    log("StageID가 SpawnManager에 등록되지 않았습니다!")
    return
end
-- stageID에 맞춰, 몬스터 스폰 데이터를 DataTableLogic에서 받아옵니다.
local getData = _DataTableLogic:GetMonsterSpawnDataTableByStageID(self.stageID)
-- 만약 getData가 사용 불가능할 경우, log를 통해 에러를 출력합니다.
if not isValid(getData) or next(getData) == nil then
    log(self.stageID.."인 데이터 테이블의 값을 찾아오지 못했습니다!")
    return
end
-- 찾아온 getData의 개수를 세어 getData의 값을 하나씩 처리합니다.
for i = 1, #getData do
    -- getData의 i번째 데이터를 data에 등록합니다.
    local data = getData[i]
    -- data가 사용 가능할 경우 data의 값들을 통해 몬스터 스폰을 시도합니다.
    if data then
        --사용 가능한 몬스터 수가 있으므로 클리어를 위해 처치해야 하는 수를 1 증가시킵니
다.
        self.needKillCount = self.needKillCount + 1
        -- 몬스터가 생성되기 위한 타이머를 설정합니다.
        self.timerTable[i] = _TimerService:SetTimerOnce(function()
            -- 스폰해야 하는 몬스터의 데이터를 DataTableLogic에 요청합니다.
            local monsterData =
_DataTableLogic:GetMonsterDefaultDataByMonsterID(data.MonsterID)
```

```

-- 생성해야 하는 몬스터를 생성합니다.
local monster = _SpawnService:SpawnByModelId(
    -- 몬스터의 ID를 통해 Model을 찾아 해당 Model을 생성합니다.
    monsterData.ModelRUID,
    -- 엔티티의 이름을 MonsterID로 변경합니다.
    monsterData.MonsterID,
    -- 생성하려는 몬스터의 생성 위치를 설정합니다.
    Vector3(data.SpawnPosX, data.SpawnPosY, 0),
    -- 생성하려는 엔티티를 Hierarchy 상에서 자신의 자식으로 설정합니다.
    self.Entity
)
-- 정상적으로 monster가 생성되었다면, 값을 초기화합니다.
if monster then
    -- 몬스터의 위치를 MOnsterSpawnData의 위치로 다시 한번 초기화합니다.
    monster.TransformComponent.Position = Vector3(data.SpawnPosX,
data.SpawnPosY, 0)
    -- 몬스터의 데이터를 MonsterDefaultData에 맞춰 초기화합니다.
    monster.MonsterState:InitData(monsterData, data.ItemDropRate,
data.MustDropItem)
end
end,
-- MonsterSpawnData에 등록된 출현 시간에 맞춰 생성되도록 시간을 등록합니다.
data.EntranceTime)
end
end
-- 모든 몬스터 생성 예약이 완료되면, GameLogic에 클리어를 위한 몬스터 처치 수를 전달합
니다.
_GameLogic:InitClearKillCount(self.needKillCount)
end

@ExecSpace("Client")
method void ClearAllChild()
-- 자기 자신의 자식으로 있는 엔티티들을 모두 찾아 chidList에 등록합니다.
local childList = self.Entity.Children
-- 만약 childList가 nil일 경우, 코드를 즉시 종료합니다.
if childList == nil then return end
-- childList가 1개 이상의 개수를 가지고 있으므로, childList의 수만큼 탐색 및 처리를 시작
합니다.
for i, child in pairs(childList) do
    --만약 i번째 엔티티가 사용 가능한 경우, 요소인 child를 제거합니다.
    if isvalid(child) then
        _EntityService:Destroy(child)
    end
end
end
-- timerTable 내의 요소가 존재할 경우, timerTable의 요소를 탐색 시작합니다.
if self.timerTable ~= nil and next(self.timerTable) ~= nil then
    -- i번째에 있는 예약된 타이머를 해제합니다.
    for i = 1, #self.timerTable do
        _TimerService:ClearTimer(self.timerTable[i])
    end
end
end
end

end

```

최소 구현 기준

- MonsterSpawnData를 변경하여 Monster의 스폰과 관련된 부분을 조절할 수 있습니다.
- Monster를 추가하여 다양한 Monster가 스폰될 수 있어야합니다.
- ItemDropRate와 MustDropItem을 통해 각 몬스터마다 아이템 드랍 확률을 조절할 수 있습니다.

응용 및 확장 방향

- GameLogic에서 스테이지 진행도를 number로 관리하고, InGame이 시작될 때, MonsterSpawnManager의 StageID값을 변경합니다.
- MonsterSpawnData의 StageID값에 따라 등장하는 Monster가 변경되도록 기능을 구현합니다.

[💡 기초 학습]UI 구성하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- UI 구현하기: 03:00:20

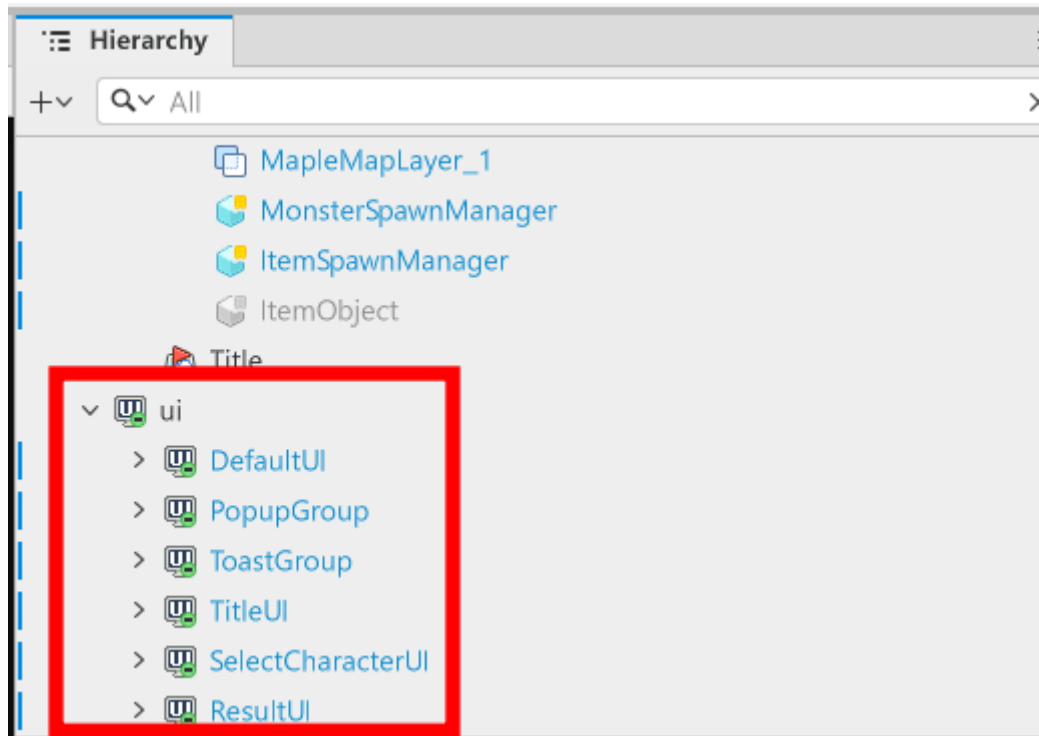
학습 목표

- 메이플스토리 월드에서 UI를 구성하는 방법에 대해 학습합니다.
- 로컬라이징 기능에 대해 학습하고, 로컬라이징 적용 방법을 학습할 수 있습니다.
- UIManageLogic의 역할을 이해하고, 활용하는 방법을 학습합니다.
- 샘플월드의 코드를 참고하여 제작하려는 게임에 맞는 UI를 제작하고, 기능을 부여할 수 있습니다.

해당 영상 시청 후 수행해 볼 내용

- 샘플 월드에 구현되어 있는 기능들을 직접 구현합니다.
- 구현 과정에서 제작하는 월드의 개성을 살릴 수 있도록 UI를 변경합니다.
- 새로운 UIGroup을 제작하고 관리하여 필요한 정보를 전달하도록 UI를 추가합니다.

UI Group의 이해

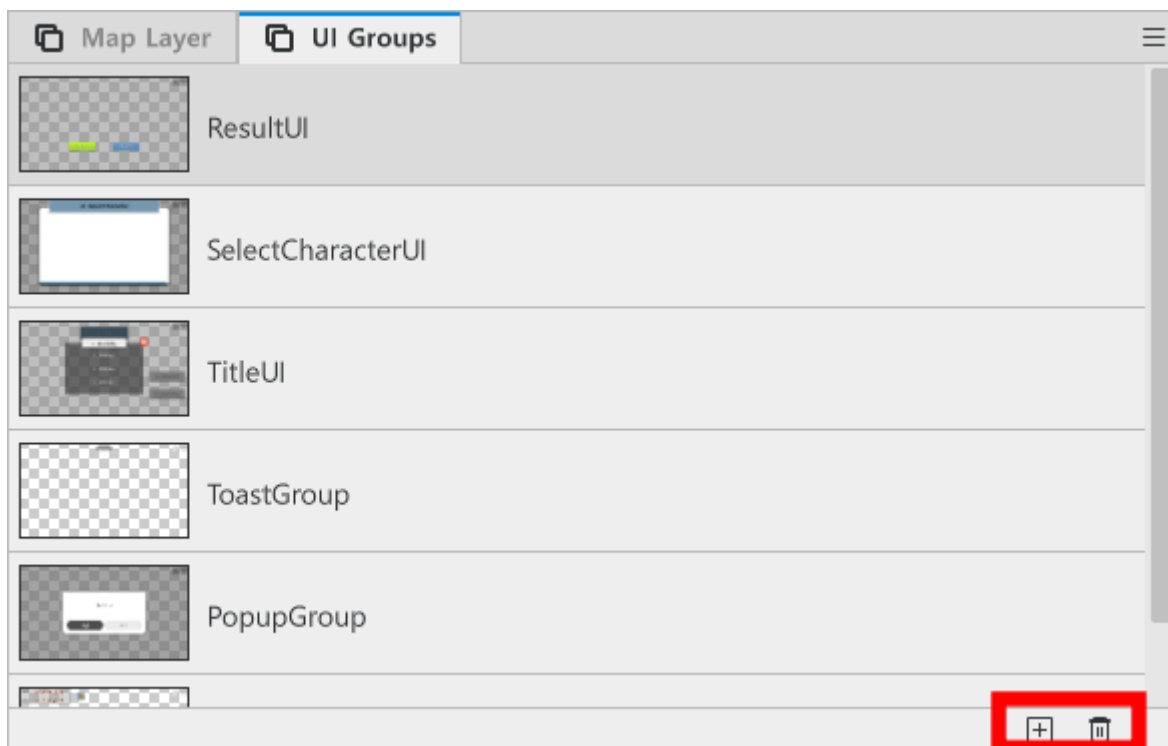


- Hierarchy 창의 아래를 보면, ui라는 이름으로 구성된 엔티티들을 확인할 수 있습니다.
- 해당 엔티티는 ui를 출력하기 위해 구성된 ui group입니다.
- ui group을 수정하여 출력해야 하는 ui를 변경하거나 제작할 수 있습니다.

UI Group 생성하기



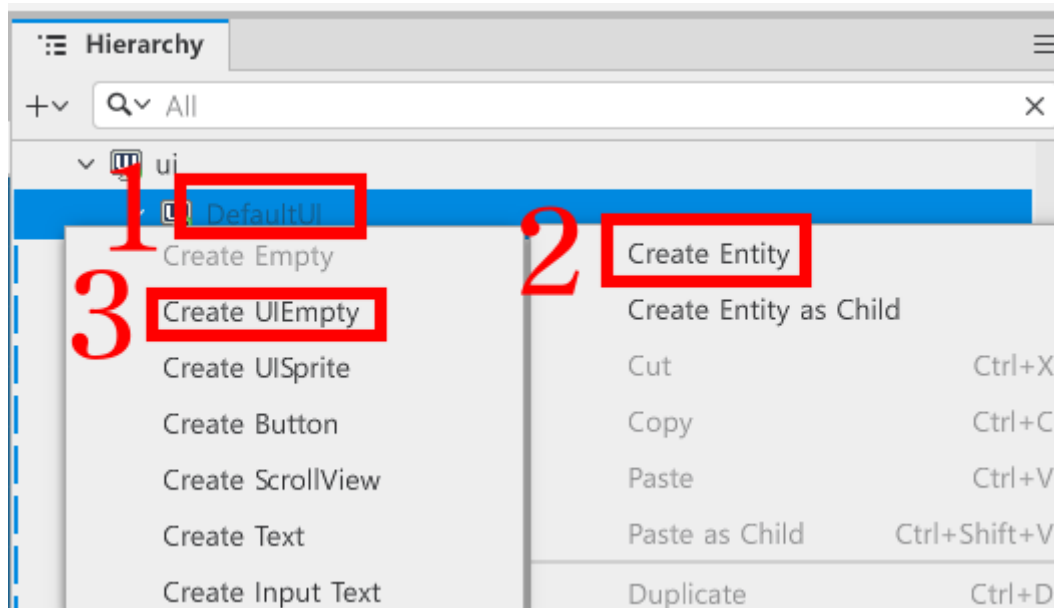
- 에디터 상단의 UI 버튼을 클릭하여 UI 편집상태로 변경합니다.



- 에디터 창 하단의 UI Groups에서 + 버튼을 클릭해 새로운 UI Groups를 추가할 수 있습니다.
- 삭제해야 하는 UI Groups가 있을 경우, 삭제하려는 UI Group을 클릭한 뒤, 휴지통 버튼을 클릭해 삭제할 수 있습니다.

빈 UI 엔티티 제작하기

- UI에서도 기존의 엔티티를 제작하듯이 엔티티 제작이 가능합니다.
- 차이점은 빈 엔티티를 생성할 때, Create Empty가 아닌 **Create UIEmpty**를 사용한다는 차이점이 있습니다.



- Hierarchy 창에서 엔티티를 추가하려는 ui group을 우클릭합니다.
- Create Entity를 클릭합니다.
- Create UIEmpty를 클릭하여 빈 UI 엔티티를 생성할 수 있습니다.

UI 엔티티에 Component 부여하기

- UI 엔티티 역시 일반 엔티티처럼 Component를 부여하여 추가적인 기능들을 수행할 수 있습니다.
- Component를 부여하는 방법은 일반 엔티티에 Add Component를 하는 방식과 동일합니다.
- 해당 단원에서는 자주 사용되는 Component에 대해 설명드리겠습니다.
 - **SpriteGUIRendererComponent**: UI에서 이미지를 출력할 때 사용되는 Component입니다. UI에서 이미지를 출력할 땐 SpriteRendererComponent보다 해당 Component를 더 권장드립니다.
 - **TextGUIRendererComponent**: UI에서 텍스트를 출력할 때 사용되는 Component입니다.
 - **ButtonComponent**: UI가 버튼의 역할을 수행해야 하는 경우 사용되는 Component입니다.

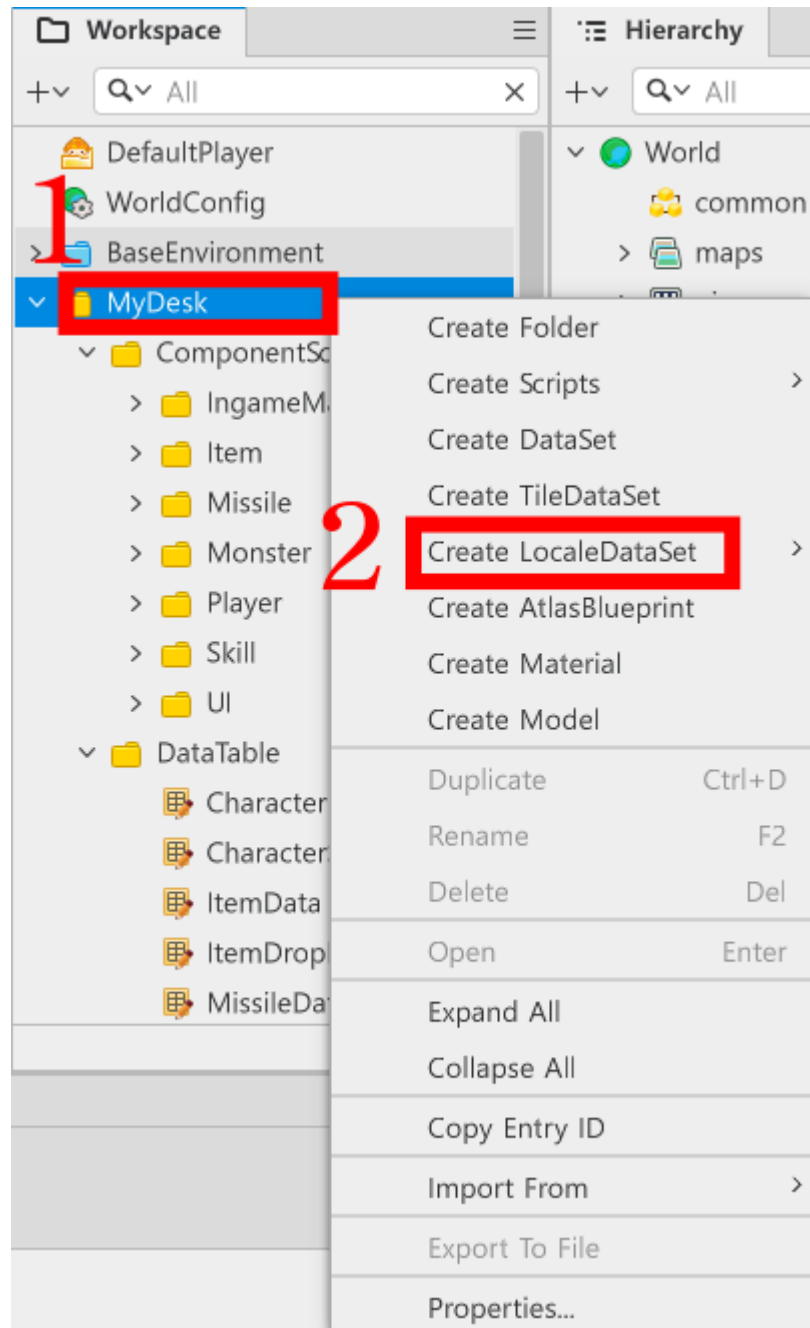
UIGroup에 Controller 부여하기

- Controller라는 개념은 Component를 제작하여 해당 UI를 관리하기 위한 개념입니다.

- 각 UIGroup마다 Controller Component를 부여하고, 해당 Component가 UI와 관련된 기능을 처리하도록 기능을 구성합니다.
- 각 Controller를 UIManageLogic에 등록하여, UIManageLogic을 호출하면 다른 UI에 UI와 관련된 기능을 요청할 수 있습니다.
- 샘플 월드에서는 다음과 같은 UIController들을 제작했습니다.
 - **DefaultUIController**: DefaultUI를 관리하기 위한 Component입니다. 주로 플레이어의 체력, 스킬 사용 횟수 업데이트를 담당하며 게임 시작 시 플레이어의 기본 공격, 스킬 사용 기능을 각 버튼에 등록하는 역할도 담당합니다.
 - **TitleUIController**: 타이틀 화면을 구성하기 위한 Component입니다. 해당 Component는 게임 시작 버튼과 게임 플레이 방법 UI를 출력하는 기능을 처리하기 위해 제작된 Component입니다.
 - **ResultUI**: GameLogic이 게임 결과를 UIManageLogic에 전달하면, UIManageLogic은 해당 Component에게 출력해야 하는 결과 UI를 요청합니다.

로컬라이징 DataSet 제작하기

- 로컬라이징 기능은 다른 국가에 서비스해야 하는 경우 필수적인 기능입니다.
- 메이플스토리 월드는 Locale DataSet을 활용하여 해당 기능을 제작할 수 있습니다.



- Workspace에서 MyDesk를 우클릭합니다.
- Create LocaleDataSet을 클릭합니다.
- Default를 클릭하여, Locale DataSet을 제작합니다.
- 샘플 월드에서는 StringTable로 제작되어 있습니다.

	1	2	3	4	5
ID	Key	Source	Note	ko	en
1	UI_Title	Shooting Game	타이틀에 사용	슈팅 게임 프레	Shooting Game
2	UI_StartGame	Start Game	타이틀 화면의	게임 시작	Start Game
3	UI_Pirate	Pirate	UI표기용 직업명	해적	Pirate
4	UI_SelectCharac	Select	캐릭터 선택 창	캐릭터 선택	Select Character
5	UI_Retry	Retry	결과창에서 다시	다시하기	Retry
6	UI_Exit	Exit	결과창의 나가기	나가기	Exit
7	UI_HowToPlay	How to play	게임 방법 버튼	플레이 방법	How to play
8	UI_InfoPlay1	Move	게임 방법 이동	이동 : WSAD	Move: WSAD
9	UI_InfoPlay2	Attack	게임 방법 공격	공격 : J	Attack : J
10	UI_InfoPlay3	Skill	게임 방법 스킬	필살기 : K	Skill : K

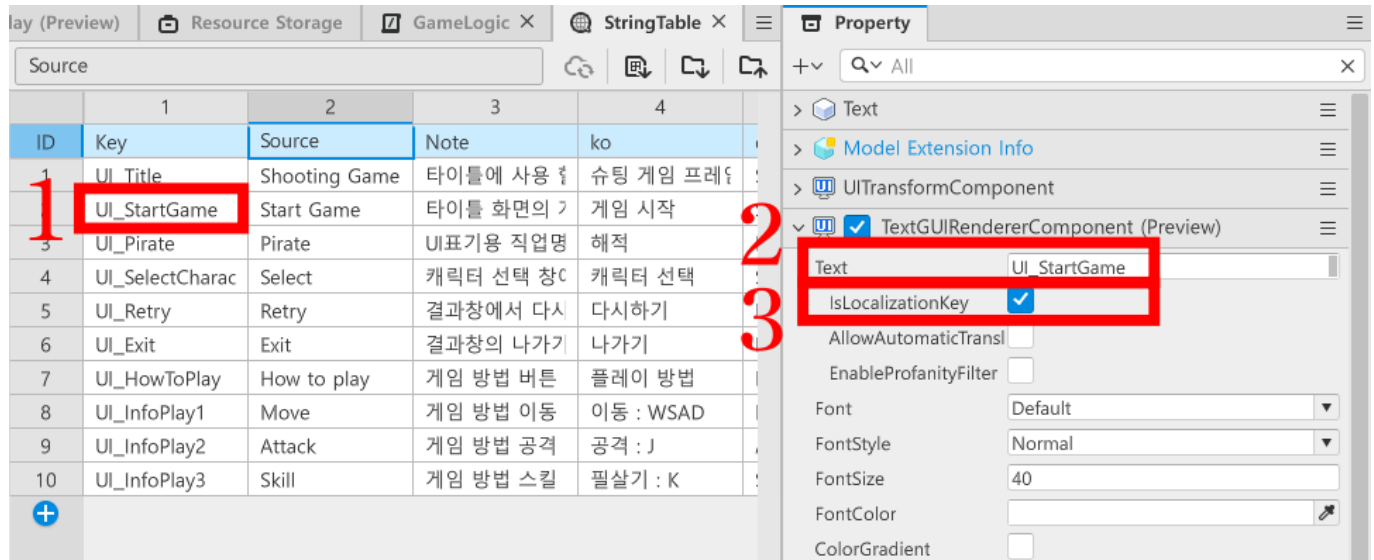
이름	설명
Key	메이플스토리 월드가 로컬라이징을 진행할 때, 로컬라이징을 해야하는 텍스트를 찾기 위한 ID 값입니다.
Source	번역의 원본 텍스트를 담기 위한 값입니다. 메이플스토리 월드가 제공하는 자동 번역 기능을 사용할 경우, 해당 값을 기준으로 번역 값을 찾습니다.
Note	해당 텍스트가 어떤 의미, 용도로 작성되었는지 기재하기 위한 칸입니다.
ko	한국어로 출력될 텍스트를 담기 위한 값입니다.
en	영어로 출력될 텍스트를 담기 위한 값입니다.

- 이 외에도 메이플스토리 월드가 공식적으로 지원하는 다양한 언어를 추가하여 제작할 수 있습니다.

로컬라이징 적용하기

- 로컬라이징 기능을 적용하는 방법은 크게 2가지가 있습니다.
- TextGUIRendererComponent의 값을 활용하는 방법과, 스크립트를 통해 사용하는 방법 2가지가 있습니다.

TextGUIRendererComponent를 활용하는 방법



- 제작한 Locale DataSet에 로컬라이징을 적용하려는 값을 입력합니다
- TextGUIRendererComponent의 Text 값으로 Locale DataSet의 Key를 입력합니다.
- TextGUIRendererComponent의 isLocalizationKey값을 체크 상태로 변경합니다.
- 해당 상태를 통해 자동으로 로컬라이징 기능이 적용되도록 만들 수 있습니다.

스크립트를 활용하는 방법

- 샘플월드에서는 빠르게 사용할 수 있도록 LocalizeLogic을 제작해 두었습니다.
- 아래는 제작된 LocalizeLogic의 코드입니다.
- 해당 코드는 Plain Text로 작성되었습니다.

```
@Logic
script LocalizeLogic extends Logic

@ExecSpace("ClientOnly")
method string GetLocalizeStringByKey(string StringKey)
-- 전달받은 StringKey 값을 기준으로 StringTable내 값을 찾아 반환합니다.
return _LocalizationService.GetText(StringKey)
end

end
```

- 매개 변수인 StringKey로 Locale DataSet의 Key 값을 전달 할 경우, 현재 게임이 실행 중인 국가의 언어에 맞춰 텍스트를 반환합니다.
 - EX: 플레이 국가가 미국과 같은 영어권일 경우, en의 값을 반환합니다.

UIManageLogic 활용하기

- 모든 UI의 설정이 마무리되었다면, 각각의 Controller를 관리하기 위해 UIManageLogic에 등록하고, 기능을 설정해야 합니다.

- 아래는 UIManageLogic의 코드입니다.
- 해당 코드는 Plain Text 환경에서 작성한 코드입니다.

```
@Logic
script UIManageLogic extends Logic

property TitleUIController titleUIController = nil

property SelectCharacterUIController selectCharacterUIController = nil

property ResultUIController resultUIController = nil

property DefaultUIController defaultUIController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 게임이 시작될 경우 TitleUIController를 찾아 등록합니다.
self:InitTitleUIController()
-- 게임이 시작될 경우 SelectCharacterUIController를 찾아 등록합니다.
self:InitSelectCharacterUIController()
-- 게임이 시작될 경우 ResultUIController를 찾아 등록합니다.
self:InitResultUIController()
-- 게임이 시작될 경우 DefaultUIController를 찾아 등록합니다.
self:InitDefaultUIController()
end

@ExecSpace("ClientOnly")
method void InitTitleUIController()
--TitleUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
if not isvalid(self.titleUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/TitleUI")
    --titleUIGroup을 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등록합니다.
    if isvalid(titleUIEntity) then
        local component = titleUIEntity.TitleUIController
        -- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
        if isvalid(component) then
            self.titleUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void InitSelectCharacterUIController()
--SelectCharacterUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
if not isvalid(self.selectCharacterUIController) then
    local selectCharacterUIEntity =
_EntityService:GetEntityByPath("/ui/SelectCharacterUI")
    --SelectCharacterUIController을 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등
    록합니다.
    if isvalid(selectCharacterUIEntity) then
        local component = selectCharacterUIEntity.SelectCharacterUIController
```

```

        -- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
        if invalid(component) then
            self.selectCharacterUIController = component
        end
    end
end
end

@ExecSpace("Client")
method void ShowTitle()
    -- 타이틀 화면을 보여줍니다.
    self.titleUIController:Show()
    -- 이외의 UI들을 모두 가림처리합니다.
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
    self.defaultUIController:Hide()
end

@ExecSpace("Client")
method void CallSelectCharacterUI()
    -- 캐릭터 선택창을 보여줍니다.
    self.selectCharacterUIController:Show()
    -- 캐릭터 선택창 내 캐릭터 리스트를 초기화 시도합니다.
    self.selectCharacterUIController:InitCharacterSelectFrame()
end

@ExecSpace("Client")
method void ShowInGame()
    -- 게임 화면 UI인 defaultUI를 초기화합니다.
    self.defaultUIController:Show()
    -- 자동 공격 UI를 초기화하도록 요청합니다.
    self.defaultUIController:UpdateAutoUI()
    -- 이외의 UI들을 가림 처리합니다.
    self.titleUIController:Hide()
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
end

@ExecSpace("Client")
method void InitResultUIController()
    --ResultUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
    if not invalid(self.resultUIController) then
        local resultUIEntity = _EntityService:GetEntityByPath("/ui/ResultUI")
        --ResultUIController를 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등록합니다.
        if invalid(resultUIEntity) then
            local component = resultUIEntity.ResultUIController
            -- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
            if invalid(component) then
                self.resultUIController = component
            end
        end
    end
end
end
end

```

```

@ExecSpace("Client")
method void ShowResultUI(boolean IsClear)
-- 결과 UI를 출력합니다.
self.resultUIController:Show(IsClear)
if IsClear then
--축하 효과음 출력
_SoundService:PlaySound("5d57b617cfc242cca347925e209c8e05", 1.0)
else
--실패 효과음 출력
_SoundService:PlaySound("52dccfb8889b4181b9181780dc7e9040", 1.0)
end
end

@ExecSpace("Client")
method void InitDefaultUIController()
--defaultUIController가 등록되어 있지 않을 경우, 해당 컴포넌트를 등록합니다.
if not isvalid(self.defaultUIController) then
local titleUIEntity = _EntityService:GetEntityByPath("/ui/DefaultUI")
--DefaultUIGroup을 찾는데 성공했을 경우, 해당 엔티티의 컴포넌트를 등록합니다.
if isvalid(titleUIEntity) then
local component = titleUIEntity.DefaultUIController
-- 정상적으로 컴포넌트가 등록되어있는지 검색한 뒤, 이를 할당합니다.
if isvalid(component) then
self.defaultUIController = component
end
end
end

@ExecSpace("Client")
method void UpdateLifeCountUI(number PlayerLife)
-- DefaultUI의 체력 UI를 PlayerLife 수에 맞춰 업데이트합니다.
self.defaultUIController:UpdateLifeUI(PlayerLife)
end

@ExecSpace("Client")
method void UpdateSkillCountUI(number LeftSkillCount)
-- LeftSkillCount의 수에 맞춰 DefaultUI의 스킬 UI를 업데이트합니다.
self.defaultUIController:UpdateSkillUI(LeftSkillCount)
end

end

```

- UI Group의 Controller를 모두 등록하고, 필요한 UI 업데이트에 맞춰 메소드를 제작했습니다.
- GameLogic 또는 PlayerState에서 UI 업데이트 요청이 있을 때, 담당 Controller를 호출하여 해당 작업을 실행합니다.

최소 구현 기준

- UI를 관리하는 Controller와, Controller의 기능을 외부에서 편하게 사용할 수 있도록 UIManageLogic을 구성합니다.
- UI에서 버튼과 같은 요소가 클릭했을 때 상호작용이 일어나야 합니다.

- UI가 게임을 진행하는데 있어 필요한 정보를 전달해야 합니다.

응용 및 확장 방향

- 플레이어가 피격당했을 때, UIManageLogic에게 업데이트를 요청하지 않고도 갱신되도록 Event를 제작합니다.
 - 해당 방식은 추후 Server와 Client간 통신을 제작할 때 동기화의 기초 처리가 됩니다.
- 필요에 따라 새로운 UI를 제작하거나 화면에 이펙트를 추가하여 유저에게 시각적 정보를 직관적으로 전달합니다.

마치며

- UI의 경우 제작하는 게임의 형식, 방향성에 따라 매우 다르게 구성될 수 있습니다.
- 샘플 월드의 경우 전체적인 흐름을 보여드리기 위해 제작된 UI이며 반드시 해당 방식을 따라 할 필요는 없습니다.
- 오히려 여러분들의 아이디어를 적극적으로 채용하고, 이를 녹여낸다면 훌륭한 게임으로 제작될 수 있습니다.